

Service-Induced Congestion in Memory-Constrained LLM Serving

(Authors' names are not included for peer review)

Abstract. In large language model (LLM) serving, each request accumulates persistent memory during service as its key–value cache grows with every generated token. Under high concurrency, aggregate memory usage therefore increases endogenously over time: the service process itself creates future capacity pressure. When memory capacity is exceeded, systems must evict active requests, i.e., discarding their cached state and restarting them later, which wastes computation and reduces throughput. This progressive consumption of resources gives rise to a new form of service-induced congestion. In this work, we develop a discrete-time dynamical model of memory-constrained LLM inference that captures the interaction among admission, memory growth, and throughput under continuous batching. In the saturated-input regime, the system admits both eviction-free fixed points and limit cycles with evictions. For homogeneous workloads, we show that the eviction-free equilibrium is unstable under standard batching dynamics and that the system converges to a unique asymptotically stable worst-case limit cycle, where throughput losses can reach 50%. For heterogeneous workloads, we establish a sharp stability dichotomy. Under a large-prompt scaling regime, the eviction-free equilibrium is asymptotically stable if and only if the decoding lengths are coprime. This result characterizes when workload heterogeneity desynchronizes completion events and helps stabilize the system around the eviction-free equilibrium. More broadly, we identify service-induced congestion as a structural instability mechanism in memory-constrained systems and determine when heterogeneity can help restore stability.

Key words: large language model serving; memory constraints; continuous batching; dynamical systems; admission control

1. Introduction

Large language models (LLMs) now serve as shared online services that must handle high request volumes under strict latency and reliability requirements (Yu et al. 2022, Kwon et al. 2023). These models generate text using autoregressive decoding. Given a prompt, the model produces text one token at a time; each newly generated token is appended to the context and used to condition subsequent token generation. Internally, this process requires storing intermediate attention activations from previous tokens, known as the key–value

(KV) cache. In particular, at each transformer layer, every token produces a key and a value vector, and the collection of these vectors for all prior tokens is cached and reused during decoding. This avoids recomputing attention over the full prefix at each step, reducing the per-token computational complexity from quadratic to linear in the sequence length. However, the KV cache grows linearly with the number of generated tokens, resulting in a progressively increasing GPU memory footprint during inference.

As such, inference for a single request is not a one-shot computation but a sequence of decoding steps that create, update, and retain request-specific state. This behavior contrasts with traditional stateless inference workloads such as image classification or fixed-length embedding generation, where each request is processed in a single forward pass. For example, in an image classification service, an input image is mapped to a label through one feedforward evaluation of the neural network, after which all intermediate activations can be discarded. In LLM serving, by contrast, a request accumulates internal state across many decoding steps and continuously occupies GPU memory throughout its lifetime. Requests therefore consume not only compute capacity but also persistent memory capacity, making memory a first-class scheduling constraint.

Another important characteristic of LLM inference is that modern serving systems do not process requests one at a time. A single decoding step uses only a small fraction of a GPU’s parallel capacity. To achieve high throughput, systems advance many requests concurrently, sharing compute across them. This high concurrency keeps the GPU well utilized, but it also means that the memory states of many requests coexist in the GPU memory.

In many practical serving regimes, GPU memory becomes the binding constraint, especially under high concurrency and long-context workloads. Production systems therefore treat memory pressure and memory-induced preemption as explicit scheduling concerns: vLLM implements swapping and recomputation for evicted KV caches (Kwon et al. 2023), SGLang employs LRU eviction for its radix-tree cache (Zheng et al. 2024), and FastServe (Wu et al. 2023) and TensorRT-LLM (NVIDIA 2023) schedule preemptively to manage memory overflow. Unlike compute resources, which can be time-shared across requests, GPU memory cannot be safely overcommitted. When aggregate memory usage exceeds capacity, the system must evict in-progress requests, discarding previously accumulated computation, which reduces effective throughput.

This memory-driven behavior differs qualitatively from congestion in classical queueing models. In most classical formulations, a job occupies a fixed amount of capacity throughout its lifetime, whether that capacity represents a server, bandwidth, or memory. LLM requests, in contrast, consume increasing capacity: even when a request’s total decoding length is known at admission, its memory footprint grows monotonically during service. A request that fits in memory at admission may become infeasible solely because it continues to run. In this context, eviction is not merely an implementation artifact or rare failure mode. Under sustained load, it becomes unavoidable because memory state grows during service: even a system that never exceeds capacity at admission may evict later as in-progress requests expand. Service itself therefore creates future capacity pressure, giving rise to a new form of congestion.

This structural difference fundamentally changes the role of admission decisions, creating an inherent tension between protecting the system from future memory overflow and maintaining high utilization. A naive myopic admission policy checks only current KV-cache usage: it admits a new request whenever the instantaneous footprint fits within available memory, without reserving capacity for future growth. Such locally feasible decisions can create synchronized memory peaks later, triggering eviction cascades that discard accumulated computation and collapse throughput. At the other extreme, overly conservative policies that reserve memory for worst-case future growth can also perform poorly. Excessive reservation suppresses concurrency and leaves substantial GPU capacity idle, a phenomenon widely observed in practice (Kwon et al. 2023, Zhong et al. 2024). The central challenge is thus how to balance memory protection against utilization when capacity consumption evolves during service.

To analyze these dynamics formally, we introduce a discrete-time queueing system in which jobs have known total sizes but resource footprints grow during service. The resulting model departs from classical queues with fixed per-job capacity consumption and reveals phenomena absent from traditional settings, including multiple equilibria and eviction-driven throughput loss. We first analyze the homogeneous workload case, and then study the heterogeneous setting. We show that homogeneous workloads are prone to synchronized memory growth, which can lead to memory peaks and throughput collapse, whereas some form of heterogeneity in decoding lengths can help sustain throughput by desynchronizing memory growth across requests. Our analysis also provides an analytical basis for studying congestion and efficiency in stateful AI services where resource consumption evolves during service.

1.1. Contributions

We formalize LLM serving with KV-cache memory as a discrete-time dynamical system that captures the interaction among admission, memory growth, and eviction (Section 2). The model isolates a congestion mechanism absent from classical queueing dynamics: resource consumption increases progressively during service. We characterize the long-run behavior of the system in a saturated-input regime, where admission is constrained only by memory feasibility. This regime isolates the intrinsic congestion effects induced by progressive memory growth, independent of stochastic arrival fluctuations, and reveals the fundamental stability limit imposed by the serving architecture itself.

For homogeneous job sizes, we provide a complete characterization of system equilibria (Theorem 1, Section 3). For jobs with decode length l_1 , the dynamics admit l_1 classes of system equilibria: one eviction-free fixed point and $l_1 - 1$ families of limit cycles indexed by their eviction levels. We show that under the baseline continuous batching policy, the eviction-free fixed point is unstable and all intermediate eviction-level cycles are also unstable. The unique asymptotically stable equilibrium is the maximal-eviction limit cycle. In particular, in the saturated-input regime, the system converges to this worst-case limit cycle, leading to throughput losses approaching 50% when decoding lengths are large relative to prompt lengths, compared with the eviction-free equilibrium.

For heterogeneous job sizes, we establish a sharp stability dichotomy governed by number-theoretic structure under a large-prompt scaling (Theorem 2, Section 4). Specifically, the eviction-free equilibrium is asymptotically stable if and only if the decoding lengths are coprime, i.e., $\gcd(l_1^{(1)}, \dots, l_1^{(K)}) = 1$. The underlying mechanism is synchronization: when decoding lengths share a common divisor, completion events align periodically, producing persistent oscillatory modes that drive the system into eviction. When they are coprime, completion phases drift relative to one another, desynchronizing memory release and damping perturbations. Thus, heterogeneity stabilizes the system only when it breaks arithmetic synchronization; not all diversity is beneficial.

Service-induced congestion can lead to eviction, which in turn causes throughput degradation: when a request is evicted, all accumulated computation is discarded, reducing effective throughput and increasing

latency variability. This observation motivates treating eviction-free operation as a natural performance target. Guided by our theoretical analysis, we propose two policies to achieve it: rate-limited admission for settings when the eviction-free equilibrium is unstable, and request mixing for heterogeneous deployments. The former regulates concurrency through admission control, while the latter stabilizes the system via routing diversification. Together, these policies provide practical guidance for memory-aware LLM serving systems.

[JD: Resolved: Discuss numerical experiments here as well.] Simulation experiments confirm the theoretical insights and illustrate the practical impact of the proposed policies (Section 6). In single-class systems, observed trajectories converge to the worst-case limit cycle under baseline admission, and the measured throughput ratio matches the analytical formula within 2% across a range of prompt and decoding lengths. In multi-class systems, when the prompting length is relatively large, coprime decoding-length pairs maintain zero eviction while non-coprime pairs exhibit persistent oscillations. Rate-limited admission eliminates eviction and achieves the theoretical throughput optimum, with the analytically derived threshold x^* confirmed to be sharp. Request mixing across GPUs prevents the cluster-wide eviction spirals caused by type-based routing and maintains near-zero eviction under varying traffic composition. Both policies remain stable under bursty arrivals and dynamic workload transitions.

1.2. Related Work

[JD: Jing will work on the literature review part later.] **LLM serving systems.** Efficient LLM inference has been studied since the deployment of large-scale Transformer models. Yu et al. (2022) introduce continuous batching (iteration-level scheduling), which admits new requests mid-iteration to improve GPU utilization and is now the de facto standard in production systems. Memory management is a central challenge: Kwon et al. (2023) apply virtual memory principles to KV cache allocation via PagedAttention; Sheng et al. (2023) develop hierarchical offloading between GPU, CPU, and disk; and Aminabadi et al. (2022) study CPU/SSD swapping strategies. A complementary line of work reduces per-request KV cache memory through sliding-window attention (Xiao et al. 2023), dynamic token eviction (Zhang et al. 2023), and low-rank compression (DeepSeek-AI 2024). These approaches reduce per-request footprint; our analysis instead

addresses the system-level dynamics arising from aggregate memory constraints, and the phenomena we identify persist regardless of the underlying compression scheme. Recent systems address the prefill-decode dichotomy through chunked prefill (Agrawal et al. 2023), disaggregated serving (Zhong et al. 2024), and pipeline parallelism (Patel et al. 2023). These works schedule within the feasible region; we characterize boundary behavior when aggregate demand approaches or exceeds capacity.

Queueing theory and fluid models. Our work connects to the queueing theory literature on fluid models and stability analysis. Fluid models approximate discrete stochastic systems by continuous dynamics (Mandelbaum et al. 1998, Maglaras 2000), and heavy-traffic analysis characterizes behavior near capacity (Bäuerle 2002, Liu and Whitt 2012). The memory-growing property of LLM inference—where each request’s resource footprint increases over time—distinguishes our setting from classical processor-sharing queues (Kleinrock 1967) and bandwidth-sharing networks (Massoulié and Roberts 2002), in which jobs occupy fixed resources. Classical admission control, beginning with Naor (1969)’s toll-based queue regulation and extended by threshold policies (Stidham 1985, Lippman 1975), assumes that admitted jobs consume fixed capacity throughout service. Extensions to load-dependent service rates (Dong et al. 2015), reusable resource networks (Dong et al. 2024, Ao et al. 2024), and proactive transfer policies (Ao et al. 2025a) relax some of these assumptions but still treat per-job resource consumption as fixed or bounded; our setting requires admission decisions that account for future memory growth. [JD: Resolved: Added admission control citations.] The instability phenomena we characterize relate to limit cycles in dynamical systems (Strogatz 2015), but the GCD-dependent stability condition appears to be new in the queueing literature.

LLM scheduling theory. Recent theoretical works target LLM scheduling directly. Li et al. (2025) develop stochastic processing models with piecewise linear iteration time and prove throughput optimality of work-conserving algorithms. Mitzenmacher and Shahout (2025) survey the intersection of queueing theory and LLM inference, identifying open problems. Most closely related, Ao et al. (2025b) formulate LLM inference as a multi-stage online scheduling problem with memory constraints and derive a fluid-based threshold policy. Our work differs in focus: we characterize discrete-time limit cycles and the role of coprimality in determining stability—phenomena absent from fluid approximations. Jaillet et al. (2025) study online

scheduling with KV cache constraints and provide constant-regret algorithms; [Chen et al. \(2025\)](#) address robust scheduling under output length uncertainty with logarithmic competitive ratios. These competitive ratio analyses complement our equilibrium-focused stability characterization.

Online scheduling and batching. LLM inference scheduling also connects to the online scheduling literature on sequential job assignment ([Devanur and Hayes 2009](#)), simultaneous batching ([Im and Moseley 2015](#)), and resource-constrained scheduling ([Lucier et al. 2012](#)). These works provide competitive ratio guarantees but do not address the dynamic memory growth inherent in LLM serving.

2. Model

2.1. Background: How LLM Inference Uses GPU Memory

Before the formal model, we summarize the inference mechanics that create memory pressure. A large language model generates text one *token* at a time, where a token is the atomic unit of text (roughly a word or word fragment). Each request arrives with a *prompt*, i.e., the user’s input, whose length varies across requests. The model first reads the entire prompt in a single pass (the *prefill* phase) and then enters the *decoding* phase, producing one output token per iteration. Because each new token depends on all preceding tokens (autoregressive generation), the model must store a pair of key and value vectors per layer for every token processed so far, collectively called the *KV cache*. This cache lets the model attend efficiently: without it, the model would reprocess the entire sequence at every step.

The KV cache must reside in GPU high-bandwidth memory (HBM) for fast attention computation. HBM is orders of magnitude smaller than system RAM and, unlike disk-backed virtual memory, cannot be overcommitted: if the aggregate KV cache of all active requests exceeds capacity, the system must evict some requests. In practice, eviction removes a request from the active batch and discards its KV cache, forcing the system to recompute the cache later. Crucially, each request’s KV cache grows with every generated token. A request that fits in memory at admission may no longer fit several tokens later.

Modern LLM servers use *continuous batching* to exploit GPU parallelism. Rather than serving one request to completion before starting the next, the server processes many requests simultaneously. In each iteration, all active requests advance by one decoding step, while new requests may be admitted and completed ones

retired. Batching is essential for high throughput because a single decoding step typically occupies only a small fraction of the GPU’s parallel compute capacity. Advancing many requests in parallel amortizes per-step overheads and keeps GPU cores busy. However, batching also means that every active request maintains its own KV cache in GPU memory. Total memory usage scales with the number of concurrent requests, making memory exhaustion inherent to high-throughput operation.

2.2. System Model

Consider a single GPU with a memory capacity of M tokens serving a stream of heterogeneous requests. Each request arrives with a prompt of length l_0 and requires l_1 decoding tokens; both l_0 and l_1 vary across requests. We abstract away the prefill phase and focus on the decoding stage, where the KV cache grows over time and drives memory pressure. This is justified by the disaggregated serving architecture now adopted in production (Zhong et al. 2024): prefill runs on separate hardware, and the resulting KV cache is transferred to the decoding GPU via high-bandwidth interconnects (e.g., NVLink). The transfer is fast relative to decoding iteration time, so the decoding GPU sees each new request as arriving with its prompt-length KV cache already in place. At decoding stage $j \in \{0, \dots, l_1 - 1\}$, the request occupies $l_0 + 1 + j$ tokens of GPU memory, consisting of the prompt, the j tokens generated so far, and one additional token reserved for the next decoding step.

For analytical tractability, we group requests into $K \geq 1$ classes by their prompt and decoding lengths: type- k requests have prompt length $l_0^{(k)}$ and decoding length $l_1^{(k)}$. Throughout, we assume deterministic lengths within each type to isolate the effect of memory growth on stability. We also assume the GPU memory capacity satisfies $M > \max_k \{l_0^{(k)} + l_1^{(k)}\}$, so that even the largest request can be processed in isolation. In practice, GPU memory is typically much larger than the footprint of a single request, allowing many requests to be served concurrently. In simulations, we assume type- k requests arrive as independent Poisson processes with rate λ_k .

Let $Q_n = (Q_n^{(1)}, \dots, Q_n^{(K)})$ denote the queue of waiting requests at time n , and let

$$X_n = (x_n^{(k,j)} : k \in [K], j \in \{0, \dots, l_1^{(k)} - 1\})$$

denote the vector of active requests by type and decoding stage. A type- k request at stage j occupies

$$w^{(k,j)} = l_0^{(k)} + 1 + j$$

tokens of GPU memory. We adopt the convention $w^{(k,l_1^{(k)})} = 0$: a request that completes its final decoding step releases all memory. The total memory usage at time n , corresponding to the post-decoding (one-step-ahead) state is

$$M_n = \sum_{k=1}^K \sum_{j=0}^{l_1^{(k)}-1} w^{(k,j)} x_n^{(k,j)}.$$

At each iteration n , the system evolves according to the following sequence.

1. *Execute.* Every active request advances one decoding stage and generates one token. Type- k requests at their final stage ($j = l_1^{(k)} - 1$) complete and leave the system, releasing their memory. Define the post-execution state

$$\tilde{x}_n^{(k,j)} = \begin{cases} x_n^{(k,j-1)}, & j = 1, \dots, l_1^{(k)} - 2, \\ 0, & j = 0, \end{cases}$$

and let

$$\tilde{M}_n = \sum_{k=1}^K \sum_{j=0}^{l_1^{(k)}-1} w^{(k,j)} \tilde{x}_n^{(k,j)}$$

denote the total memory usage in the post-execution state.

2. *Arrive.* New requests arrive and join the waiting queue:

$$Q_n \leftarrow Q_n + A_n,$$

where $A_n = (A_n^{(1)}, \dots, A_n^{(K)})$.

3. *Evict.* Each surviving request grew by one token during execution. If the post-execution memory exceeds capacity,

$$\tilde{M}_n > M,$$

the system evicts active requests until feasibility is restored ($\tilde{M}_n \leq M$). Evicted requests forfeit their KV cache and re-enter the queue. Evictions follow a *least-progressed-first* (LPF) rule: requests at smaller (earlier) stages are evicted before those at larger (later) stages, regardless of type. Let $\hat{X}_n$ denote the resulting active state after eviction.

4. *Admit.* Requests from Q_n are admitted into stage 0 subject to the memory constraint:

$$\sum_{k,j} w^{(k,j)} \hat{x}_n^{(k,j)} + \sum_k w^{(k,0)} a_n^{(k)} \leq M,$$

where $a_n^{(k)}$ is the number of type- k requests admitted at time n , and each admission consumes $w^{(k,0)} = l_0^{(k)} + 1$ tokens of memory. (We do not specify how requests are selected from the queue at this stage; this will be addressed later.) Admitted requests enter at stage 0, and the remaining requests stay waiting in the queue.

The resulting state after admission defines X_{n+1} and Q_{n+1} .

The sequence above defines one *decoding iteration*. Completions occur when type- k requests at stage $j = l_1^{(k)} - 1$ advance and leave the system during the execution step. Let D_n denote the number of completions at time n . The throughput is

$$\bar{T} = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^N D_n$$

whenever the limit exists. This per-iteration completion rate reflects the degree of concurrency sustained by the active batch; in the saturated-input regime formalized below, it isolates the memory-limited throughput. When iteration duration is approximately constant under sustained continuous batching, $\bar{T}$ also approximates wall-clock throughput, but our analysis is stated entirely in completions per iteration.

Since eviction discards all computation performed for the evicted request, evicting a later-stage request incurs strictly greater waste than evicting an earlier-stage one. The LPF rule, which evicts requests in increasing order of stage, thus minimizes wasted computation for any given number of evictions. Furthermore, by preserving more advanced requests, LPF increases the likelihood that accumulated work is completed. For these reasons, LPF is a natural eviction policy for memory-constrained serving (Kwon et al. 2023).

The baseline system employs no admission control beyond a one-step memory feasibility check: requests are admitted whenever their anticipated memory footprint (including reserved space for the next decoding step) fits within the available GPU memory. In particular, the admission decision does not reserve capacity for the future growth of KV caches during decoding. This greedy policy reflects common practice in existing serving systems (Kwon et al. 2023, NVIDIA 2023) and isolates the mechanisms that drive throughput loss. Section 5 builds on our analysis to develop improved admission policies.

EXAMPLE 1 (PROTOCOL TRACE). Consider a single class of requests ($K = 1$) with $l_0 = 2$, $l_1 = 3$, and memory capacity $M = 24$. Stage weights are $w^{(j)} = l_0 + 1 + j$, $j = 0, 1, 2$, so stage-0, stage-1, and stage-2 requests occupy 3, 4, and 5 tokens of memory, respectively.

We trace two consecutive iterations starting from $X_n = (x^{(0)}, x^{(1)}, x^{(2)}) = (1, 1, 2)$ and $Q_n = 8$, and assume $A_n = 5$ and $A_{n+1} = 0$.

Iteration n .

1. *Execute.* The two stage-2 requests complete and leave the system, while the remaining requests advance one stage. The post-execution state becomes $\tilde{X}_n = (0, 1, 1)$, with memory usage $\tilde{M}_n = 4 + 5 = 9$.
2. *Arrive.* Five new requests arrive, increasing the queue length to $Q_n \leftarrow 8 + 5 = 13$.
3. *Evict.* Since $\tilde{M}_n = 9 \leq 24$, no eviction occurs.
4. *Admit.* Each stage-0 request requires $w^{(0)} = 3$ tokens of memory. The feasibility condition $9 + 3a \leq 24$ implies $a \leq 5$, so the system admits $a = 5$ requests. The resulting state becomes $X_{n+1} = (5, 1, 1)$, with memory usage $M_{n+1} = 5 \cdot 3 + 4 + 5 = 24$, and queue length $Q_{n+1} = 13 - 5 = 8$.

Iteration $n + 1$.

1. *Execute.* The single stage-2 request completes, while the remaining requests advance one stage, yielding $\tilde{X}_{n+1} = (0, 5, 1)$. The resulting memory usage is $\tilde{M}_{n+1} = 0 \cdot 3 + 5 \cdot 4 + 1 \cdot 5 = 25 > 24$.
2. *Arrive.* No new requests arrive, so the queue length remains $Q_{n+1} \leftarrow Q_{n+1} + A_{n+1} = 8 + 0 = 8$.
3. *Evict.* Since $\tilde{M}_{n+1} > 24$, the system evicts requests until feasibility is restored. Under the LPF rule, requests at smaller stages are evicted first. Evicting one stage-1 request frees $w^{(1)} = 4$ tokens, reducing memory to $25 - 4 = 21 \leq 24$. The post-eviction state is therefore $\hat{X}_{n+1} = (0, 4, 1)$, and the evicted request rejoins the queue, giving $Q_{n+1} \leftarrow 8 + 1 = 9$.
4. *Admit.* After eviction, $24 - 21 = 3$ tokens of memory remain available. Since each stage-0 admission requires 3 tokens, the system admits $a = 1$ request. The resulting state is $X_{n+2} = (1, 4, 1)$, with memory usage $M_{n+2} = 1 \cdot 3 + 4 \cdot 4 + 1 \cdot 5 = 24$, and $Q_{n+2} = 9 - 1 = 8$.

This trace illustrates how admission decisions that are feasible at the time they are made can nevertheless lead to memory overflow in the subsequent iteration.

2.3. Equilibria and Stability

We consider a *saturated-input regime* in which a request is always available for admission. Formally, we replace the exogenous arrival process by the condition $Q_n > 0$ for all $n \geq 0$, so that the admission decision at time n is constrained only by GPU memory feasibility. This decouples memory-driven throughput limits from the stochastic arrival process, isolating the maximum sustainable throughput imposed by memory growth and eviction. This is a theoretical abstraction, not a literal description of day-to-day operation. Real systems may move in and out of this regime. Nevertheless, operators strive to keep GPU memory utilized, since idle memory represents wasted capital (Kwon et al. 2023, NVIDIA 2023). The memory-constrained regime therefore reveals the fundamental limits that memory growth imposes on performance. Numerical experiments in Section 6 confirm that these insights extend beyond this idealized regime.

In the saturated-input regime, the queue never empties, so admissions depend only on the current active state X_n and the memory constraint M . We therefore take X_n as the system state. Under this assumption, the dynamics reduce to a deterministic map

$$X_{n+1} = F(X_n). \quad (1)$$

Our objective is to characterize the macroscopic structure of this map, e.g., its equilibrium behaviors and their dependence on job sizes. In subsequent analysis, i.e., Sections 3 and 4, we analyze a continuous relaxation in which the state variables $X_n^{(k,j)}$ are allowed to take real values. This formulation captures the dominant system dynamics while avoiding integrality artifacts. The resulting approximation error is at most one request per stage. Since each stage typically contains $O(M)$ requests, the relative error is $O(1/M)$ and vanishes as M grows.

For the discrete-time dynamical system (1), two types of equilibria can arise.

DEFINITION 1 (FIXED POINT). A **fixed point** is a state X^* satisfying $F(X^*) = X^*$.

In our model, a fixed point is a steady regime: the number of requests at each stage, total memory usage, and throughput all remain constant, with no eviction.

DEFINITION 2 (LIMIT CYCLE). A **limit cycle** of period $p \geq 2$ is a sequence of states $X_1^*, \dots, X_p^*$, such that

$$F(X_i^*) = X_{i+1}^* \quad \text{for } i = 1, \dots, p-1, \quad F(X_p^*) = X_1^*,$$

and p is the smallest positive integer for which this property holds.

For a limit cycle of period p , the throughput reduces to

$$\bar{T} = \frac{1}{p} \sum_{i=1}^p \sum_{k=1}^K X_i^{*(k, l_1^{(k)} - 1)}$$

where $X_i^{*(k, l_1^{(k)} - 1)}$ denotes the number of type- k requests at stage j in iteration i . For a fixed point, $T = \sum_{k=1}^K X^{*(l_1^{(k)} - 1)}$.

We further distinguish equilibria by their stability:

DEFINITION 3 (STABILITY). Let $\|\cdot\|$ denote any norm on the state space. A fixed point X^* is said to be:

- **Stable** if for every $\epsilon > 0$ there exists $\delta > 0$ such that for any X_0 satisfying $\|X_0 - X^*\| < \delta$, $\|X_n - X^*\| < \epsilon$ for all n .
- **Asymptotically stable** if it is stable and there exists $\delta > 0$ such that for any X_0 satisfying $\|X_0 - X^*\| < \delta$,

$$\lim_{n \rightarrow \infty} \|X_n - X^*\| = 0.$$

For a limit cycle $X_1^*, \dots, X_p^*$, define the cycle set

$$C = \{X_1^*, \dots, X_p^*\}, \quad \text{dist}(X, C) := \min_{Y \in C} \|X - Y\|.$$

The limit cycle is **stable** if for every $\epsilon > 0$ there exists $\delta > 0$ such that for any X_0 satisfying $\text{dist}(X_0, C) < \delta$,

$$\text{dist}(X_n, C) < \epsilon \quad \text{for all } n \geq 0.$$

It is **asymptotically stable** if, in addition,

$$\lim_{n \rightarrow \infty} \text{dist}(X_n, C) = 0.$$

For a limit cycle, stability requires convergence to the orbit C , not to a single point. In particular, asymptotic stability implies that there exists an integer shift $\tau \in \{0, \dots, p-1\}$ such that

$$\|X_n - X_{(n+\tau) \bmod p}^*\| \rightarrow 0.$$

3. Structural Instability of the Homogeneous System

We begin with a homogeneous system ($K = 1$) in which every request has prompt length l_0 and decoding length l_1 . Dropping the type superscript, we write $w_j = l_0 + 1 + j$ for the memory footprint at stage j .

3.1. Eviction-Free Dynamics and Fixed Point

Consider one iteration starting from state $X_n = (x_n^{(0)}, \dots, x_n^{(l_1-1)})$ with total memory usage $M_n = M$. During the *Execute* step, every stage- j request advances to stage $j+1$ and increases its memory footprint by one token. Requests at the final stage $j = l_1 - 1$ complete and depart, releasing their entire footprint, i.e., $w_{l_1-1} = l_0 + l_1$. The net memory freed by the execute step equals $(l_0 + l_1)x_n^{(l_1-1)} - \sum_{j=0}^{l_1-2} x_n^{(j)}$. In the saturated-input regime, the system converts all freed memory into new stage 0 requests, each costing $w_0 = l_0 + 1$ tokens. Therefore, the number of newly admitted (stage 0) requests at the start of iteration $n+1$ satisfies the balance relation

$$(l_0 + 1)x_{n+1}^{(0)} = (l_0 + l_1)x_n^{(l_1-1)} - \sum_{j=0}^{l_1-2} x_n^{(j)}. \quad (2)$$

For $j = 0, 1, \dots, l_1 - 2$, stage advancement gives $x_{n+1}^{(j+1)} = x_n^{(j)}$. Combining these relations yields a linear update $X_{n+1} = B X_n$ where

$$B = \begin{bmatrix} \alpha & \alpha & \cdots & \alpha & \beta \\ 1 & 0 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix}, \quad \alpha = \frac{-1}{l_0 + 1}, \quad \beta = \frac{l_0 + l_1}{l_0 + 1}. \quad (3)$$

The first row encodes the balance equation; subsequent rows encode the stage shift. The coefficient $\beta > 1$ captures a fundamental asymmetry: a completing request frees β admission slots. Note that when $l_1 \gg l_0$, β is large and each completion can trigger a disproportionate admission burst.

Solving $F(X^*) = X^*$ on the memory boundary yields the eviction-free fixed point. Let $\mathbf{1} \in \mathbb{R}^{l_1}$ denote the all-ones vector. Since $\alpha(l_1 - 1) + \beta = 1$, we have $B\mathbf{1} = \mathbf{1}$, and hence any fixed point must be of the form $X^* = x^*\mathbf{1}$. Imposing $M = \sum_{j=0}^{l_1-1} (l_0 + 1 + j)x^*$ yields

$$x^* = \frac{2M}{l_1(2l_0 + l_1 + 1)}. \quad (4)$$

At X^* , each stage holds x^* requests, memory remains at capacity M with no eviction, and throughput equals x^* completions per iteration.

3.2. Equilibria with Eviction

The fixed point X^* is not the only equilibrium. When memory overflows, the dynamics admit periodic regimes in which some stages remain empty at every iteration.

For an integer $i \in \{0, 1, \dots, l_1 - 1\}$, we say the system is at *eviction level* i at time n if exactly i stage coordinates of X_n equal zero and the remaining $l_1 - i$ stage coordinates are strictly positive. In particular, level 0 corresponds to eviction-free operation (no empty stages). An *eviction-level- i limit cycle* is a periodic orbit $\{X_1, \dots, X_p\}$ such that every state on the orbit has exactly i empty coordinates.

EXAMPLE 2 (EXAMPLE EQUILIBRIA). Consider $l_0 = 2$, $l_1 = 3$, memory capacity $M = 24$. The system admits three equilibria corresponding to eviction levels $i = 0, 1, 2$.

Eviction level 0. The fixed point is $X^* = (2, 2, 2)$ and achieves throughput $x^* = 2$ completions per iteration.

Eviction level 1. At eviction level 1, the system admits a period-3 limit cycle

$$\left(\frac{72}{13}, \frac{24}{13}, 0\right) \rightarrow \left(0, \frac{48}{13}, \frac{24}{13}\right) \rightarrow \left(\frac{24}{13}, 0, \frac{48}{13}\right) \rightarrow \left(\frac{72}{13}, \frac{24}{13}, 0\right),$$

with throughput $(24/13 + 48/13)/3 \approx 1.85$. Note that eviction occurs on the first transition.

Eviction level 2. At the maximal eviction level, the system admits the period-3 limit cycle

$$(8, 0, 0) \rightarrow (0, 6, 0) \rightarrow \left(0, 0, \frac{24}{5}\right),$$

with throughput $8/5$, corresponding to a 20% reduction from the fixed-point throughput x^* .

REMARK 1. For $l_1 \geq 4$, limit cycles at a given eviction level i , $2 \leq i \leq l_1 - 2$, need not be unique. Stage j is said to be *occupied* if it contains at least one active request. Different configurations of occupied stages can produce distinct periodic orbits with different throughputs. Figure 1 illustrates this landscape. For a fixed eviction level, some cycles concentrate all occupied stages into a single *contiguous block* of stage indices (contiguous block), while others distribute occupied stages more evenly across the stages (evenly spaced). The contiguous block configurations achieve the lowest throughput, whereas evenly spaced configurations achieve the highest throughput for a fixed eviction level. These limit cycle families are characterized formally in Appendix A.3 (Propositions 2–4).

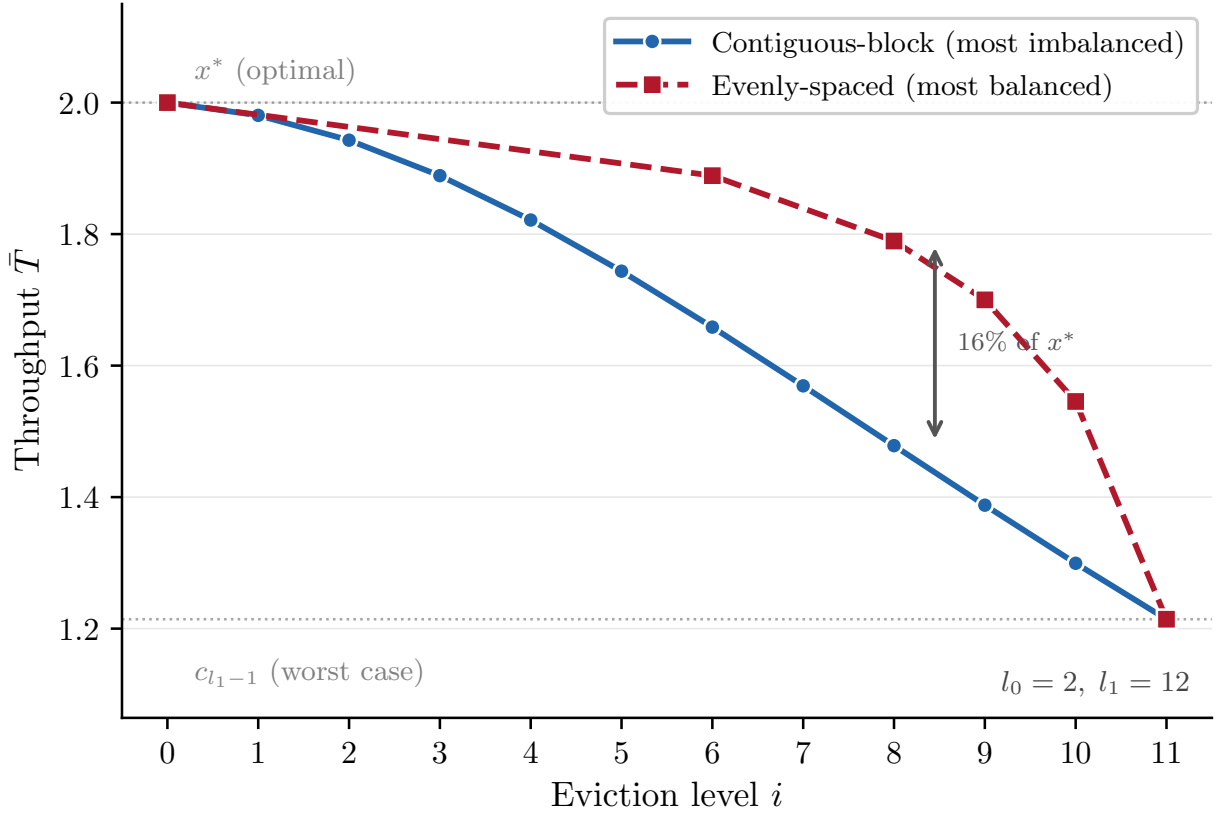


Figure 1 Throughput landscape across eviction levels for $l_0 = 2, l_1 = 12$. The contiguous-block family (Proposition 3) achieves the lowest throughput at each level; the evenly-spaced family (Proposition 4) achieves the highest. All achievable cycle throughputs lie between these two curves. Both families coincide at level 0 (x^* , the eviction-free optimum) and level $l_1 - 1$ (c_{l_1-1} , the worst case). The gap widens at intermediate levels, reaching 16% of x^* at level 8.

At the maximal eviction level $i = l_1 - 1$, exactly one stage is active at each iteration, whose size saturates the memory constraint. The resulting cycle has the form

$$\left(\frac{M}{w_0}, 0, \dots, 0\right) \rightarrow \left(0, \frac{M}{w_1}, 0, \dots, 0\right) \rightarrow \dots \rightarrow \left(0, \dots, 0, \frac{M}{w_{l_1-1}}\right).$$

The corresponding throughput equals

$$\bar{T}_{l_1-1} := \frac{M}{l_1(l_0 + l_1)}.$$

The maximal-eviction cycle leads to the lowest throughput among all equilibria as characterized by the following lemma.

LEMMA 1 (Worst-case throughput). *Let C be any limit cycle and let $\bar{T}(C)$ denote its throughput. Then*

$$\bar{T}(C) \geq \bar{T}_{l_1-1} = \frac{M}{l_1(l_0 + l_1)}.$$

Moreover, equality holds if and only if C is the maximal-eviction cycle (i.e., the cycle with eviction level $l_1 - 1$).

We define the worst-case throughput ratio as

$$\frac{\bar{T}_{l_1-1}}{x^*} = \frac{l_1(2l_0 + l_1 + 1)/2}{l_1(l_0 + l_1)} = \frac{2l_0 + l_1 + 1}{2(l_0 + l_1)}.$$

This ratio lies between $1/2$ and 1 , quantifying the throughput of the maximum eviction equilibrium relative to the eviction-free fixed point. It decreases with the decoding length l_1 and increases with the prompt length l_0 . The ratio approaches $1/2$ as $l_1/l_0 \rightarrow \infty$, indicating the worst-case eviction can halve throughput.

3.3. Eviction Cascade

Although the eviction-free fixed point X^* achieves the maximum throughput, it is unstable. Baseline admission amplifies any small imbalance across stages, eventually triggering eviction. Once eviction begins, the system does not return to eviction-free operation; instead, it cascades through regimes with progressively more empty stages and settles into the worst-case limit cycle at eviction level $l_1 - 1$.

THEOREM 1 (Baseline Instability). *In the saturated-input regime, the system possesses l_1 types of equilibria: one eviction-free fixed point, one eviction-level- $(l_1 - 1)$ limit cycle, and (potentially multiple) eviction-level- i limit cycles for $i = 1, \dots, l_1 - 2$.*

1. *The eviction-free fixed point is not stable.*
2. *Eviction-level- i limit cycles, for $i = 1, \dots, l_1 - 2$, are not stable.*
3. *The eviction-level- $(l_1 - 1)$ limit cycle is asymptotically stable.*

The instability is driven by an amplification mechanism, captured by the *spread*

$$G_n := \max_j x_n^{(j)} - \min_j x_n^{(j)}. \quad (5)$$

Before any eviction happens (i.e., before any stage occupancy has reached zero), the stage shift preserves all non-final coordinates in X_n , up to the deterministic relabeling of stages. For the final and first stage, rewrite the memory balance equation as (2) as

$$x_{n+1}^{(0)} = x_n^{(l_1-1)} + \frac{\sum_{j=0}^{l_1-1} (x_n^{(l_1-1)} - x_n^{(j)})}{l_0 + 1} \quad (6)$$

Let $\bar{x}_n = \frac{1}{l_0+1} \sum_{j=0}^{l_1-1} x_n^{(j)}$. Then if $x_n^{(l_1-1)} \leq \bar{x}_n$, $x_{n+1}^{(0)} \leq x_n^{(l_1-1)}$. If $x_n^{(l_1-1)} > \bar{x}_n$, $x_{n+1}^{(0)} > x_n^{(l_1-1)}$. Thus, G_n is non-decreasing. If the current minimum occupies the final stage, then from (6), we have

$$x_{n+1}^{(0)} \leq x_n^{(l_1-1)} + \frac{G_n}{l_0 + 1}.$$

If the current maximum occupies the final stage, we have

$$x_{n+1}^{(0)} \geq x_n^{(l_1-1)} + \frac{G_n}{l_0 + 1}.$$

Thus, for $G_n \neq 0$, it increases by at least $G_n/(l_0 + 1)$ when the minimum or maximum reaches the final stage, which happens at least once every l_1 steps. Since the memory constraint bounds each stage occupancy by $M/(l_0 + 1)$, the maximum $\max_j x_n^{(j)}$ stays bounded while G_n grows. This implies that the minimum $\min_j x_n^{(j)}$ reaches zero in finite time, i.e., we have the first *support loss* (See Lemma 2 in Appendix A for the precise argument).

After support loss, the physical-stage spread is no longer the right object: eviction changes which physical stages carry mass. To address this, first, instead of tracking physical stages, we track relative positions moving around a circular pipeline. By taking this relative position view, we can show that once a position becomes empty, it stays empty. In addition, we sample only at special times called block-ends. In particular, we define an eviction block as a sequence of iterations where eviction happens, but no completion occurs; and block-end is the first iteration after the eviction block in which a live position (i.e., position with positive mass) completes and new requests are admitted. Sampling the process at block-ends allows us to reduce a complicated process into a clean iterative map. Lastly, we measure imbalance using a normalized *block-end spread*, where each live mass is divided by the predecessor gap, which is the number of stages from the previous live position. The normalization makes the mass more comparable because positions with larger

gaps accumulate more mass during eviction. For the newly defined block-end spread, the same amplification mechanism holds: any positive spread is amplified until another live position reaches zero (see Appendix A for more details).

Since empty relative positions never revive, support losses form a finite cascade. Therefore, every trajectory not lying exactly on a balanced intermediate cycle eventually reaches the single-live-position regime, namely the maximal-eviction cycle. At that level, the memory boundary and LPF eviction damp perturbations within one pipeline cycle, so the maximal-eviction cycle is the unique asymptotically stable cycle.

EXAMPLE 3 (INSTABILITY SPIRAL). Consider the system with $l_0 = 2$, $l_1 = 3$, and memory capacity $M = 24$ (same as Example 1). The eviction-free equilibrium has $x^* = 2$. We perturb this equilibrium by increasing stage 0 slightly, by $\frac{1}{2}$. Specifically, we start from $X_0 = \left(\frac{5}{2}, 2, \frac{17}{10}\right)$. The balance equation (2) yields the following eviction-free trajectory:

n	$X_n = (x_n^{(0)}, x_n^{(1)}, x_n^{(2)})$	G_n
0	$\left(\frac{5}{2}, 2, \frac{17}{10}\right)$	$\frac{4}{5}$
1	$\left(\frac{4}{3}, \frac{5}{2}, 2\right)$	$\frac{7}{6}$
2	$\left(\frac{37}{18}, \frac{4}{3}, \frac{5}{2}\right)$	$\frac{7}{6}$
3	$\left(\frac{82}{27}, \frac{37}{18}, \frac{4}{3}\right)$	$\frac{46}{27}$
4	$\left(\frac{85}{162}, \frac{82}{27}, \frac{37}{18}\right)$	$\frac{407}{162}$
5	$\left(\frac{544}{243}, \frac{85}{162}, \frac{82}{27}\right)$	$\frac{407}{162}$
6	$\left(\frac{6037}{1458}, \frac{544}{243}, \frac{85}{162}\right)$	$\frac{2636}{729}$

At iteration $n = 6$, the execute step shifts the state to $\tilde{X}_6 = (0, x_6^{(0)}, x_6^{(1)}) = \left(0, \frac{6037}{1458}, \frac{544}{243}\right)$, whose post-execution memory usage is

$$\tilde{M}_6 = 4x_6^{(0)} + 5x_6^{(1)} = \frac{20234}{729} > 24.$$

LPF evicts from the least-progressed occupied stage (stage 1), reducing it by $e = (\tilde{M}_6 - 24)/4 = \frac{1369}{1458}$ and leaving no slack for admission: $X_7 = \left(0, \frac{778}{243}, \frac{544}{243}\right)$. This is the first eviction (see also Figure 2).

Continuing from $X_7 = (0, \frac{778}{243}, \frac{544}{243})$, the system enters eviction level 1 (one empty stage per iteration). Over the next nine iterations, LPF evictions progressively deplete the second live position. At $n = 16$, the system

enters the maximal-eviction cycle $(8, 0, 0) \rightarrow (0, 6, 0) \rightarrow (0, 0, 4.8)$, which repeats stably with throughput $\bar{T}_2 = 8/5$, a 20% reduction from $x^* = 2$.

n	X_n (approx.)	level	event
7	(0, 3.20, 2.24)	1	first eviction
10	(0, 2.67, 2.66)	1	second eviction
13	(0, 1.56, 3.55)	1	third eviction
16	(0, 0, 4.80)	2	second position dies
17	(8, 0, 0)	2	worst-case cycle begins
18	(0, 6, 0)	2	...
19	(0, 0, 4.8)	2	...

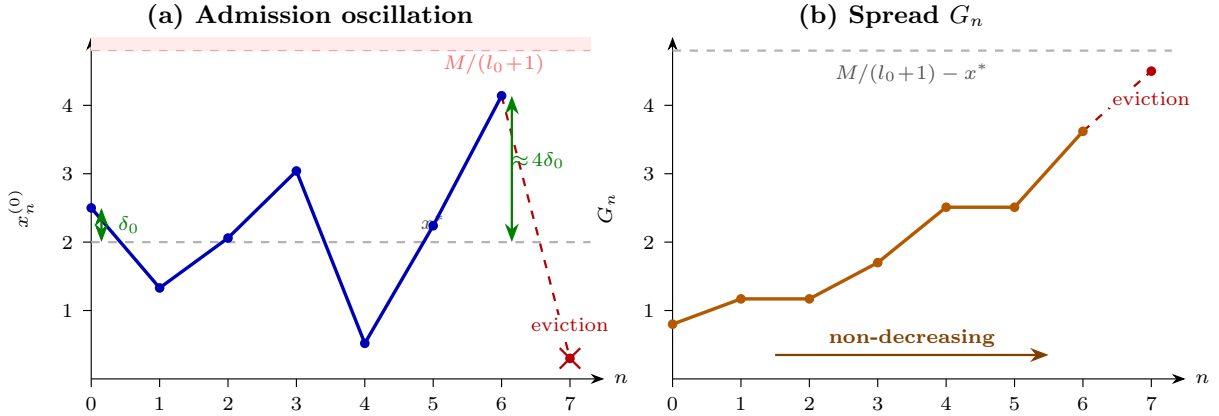


Figure 2 Instability trajectory for $l_0 = 2$, $l_1 = 3$, $M = 24$, starting from a small perturbation ($\delta_0 = 0.5$) above the equilibrium $x^* = 2$. (a) The admission count $x_n^{(0)}$ oscillates with geometrically growing amplitude (spectral radius $|\lambda| = \sqrt{5/3} \approx 1.29$). After 7 iterations, the required admission turns negative—memory is exhausted by surviving requests alone—triggering eviction. (b) The spread $G_n = \max_j \{x_n^{(j)}\} - \min_j \{x_n^{(j)}\}$ is monotonically non-decreasing. The spread’s inevitable growth drives the system to eviction regardless of the perturbation direction.

Above all, the instability is structural: baseline admission converts each completion burst into a disproportionate admission pulse ($\beta > 1$), and progressive memory growth amplifies these pulses as they propagate through the pipeline. The pulses eventually synchronize all requests at a single stage, triggering maximal eviction.

4. Multi-Class Systems and Structural Stability

Real LLM workloads are heterogeneous: short chat completions, long code-generation sessions, and multi-turn reasoning chains coexist on the same GPU pool, each with different prompt and decoding lengths. Variation in decoding lengths can desynchronize completions and mitigate the eviction cascades of Section 3, but whether it does depends on arithmetic as we will show in this section.

4.1. Multi-Class Model and Large-Prompt Regime

Consider $K \geq 2$ request types with decoding lengths $l_1^{(1)} < l_1^{(2)} < \dots < l_1^{(K)}$ and arrival proportions $p_1, \dots, p_K$ ($\sum_{k=1}^K p_k = 1$, and $p_k > 0$ for $k = 1, \dots, K$). Each type k has prompt length $l_0^{(k)}$. As before, we focus on the saturated-input regime: the waiting queue is never empty, and admission is constrained only by memory. Because the backlog is effectively infinite, the system must specify how request types are selected for admission. We impose the natural proportion constraint that $x_n^{(0,k)} = p_k x_n^{(0)}$. This constraint separates memory dynamics from type-selection effects and ensures that any eviction-free operating point, if it exists, is determined by memory balance rather than by admission control.

To isolate the structural effects of heterogeneous decoding lengths, we analyze a prompt-dominated asymptotic regime. Specifically, we consider a sequence of systems in which the decoding lengths remain fixed, while prompt lengths and total memory scale proportionally:

$$l_0^{(k)} = r_k L, \quad M = c L, \quad L \rightarrow \infty, \quad (7)$$

where $r_1, \dots, r_K > 0$ and $c > 0$ are fixed constants. This scaling preserves non-degenerate concurrency while making per-stage memory costs nearly constant across stages. Indeed, for a type k request at stage j , $w_j^{(k)} = l_0^{(k)} + 1 + j = r_k L (1 + O(1/L))$, so the relative variation in memory usage across decoding stages vanishes as L grows. In this limit, the dominant source of dynamics is not the precise stage-dependent memory weight, but the timing of request completions, which is determined by the interaction among the decoding lengths.

This regime also reflects a prompt-dominated setting common in modern LLM serving: multi-turn chat, retrieval-augmented pipelines, and agentic or tool-use workflows often append retrieved documents,

interaction histories, and tool outputs to the prompt (Lewis et al. 2020, Yao et al. 2023, Schick et al. 2023, Zheng et al. 2024), while individual decoding outputs may span only tens to a few hundred tokens.

It is worth noting that large prompts also attenuate the amplification mechanism observed in the homogeneous model. There, eviction cascades are driven by the memory gradient across stages: a completion releases $w_{l_1-1} = l_0 + l_1$ tokens of memory while admissions consume only $w_0 = l_0 + 1$, yielding amplification factor $\beta = w_{l_1-1}/w_0 = 1 + O(l_1/l_0)$. Thus, when prompts dominate memory usage, the stage-to-stage memory gradient becomes small, and eviction cascades cause smaller throughput losses. The eviction-free fixed point nevertheless remains unstable for every finite l_0 , and eviction cascades can still occur.

In the heterogeneous setting, we use the large-prompt regime to remove the confounding effect of stage-dependent memory weights and expose the role of completion timing. The limiting dynamics are then governed by the arithmetic structure of the decoding lengths, leading to the $\gcd(l_1^{(1)}, \dots, l_1^{(K)})$ -based dichotomy derived below. The numerical experiments in Section 6 confirm that these structural insights persist at finite prompt lengths, including regimes where the homogeneous throughput loss is substantial.

We first characterize the eviction-free equilibrium. In the saturated-input regime with proportion constraint, an eviction-free fixed point is one in which: 1) No request is evicted before completing decoding; 2) Memory is exactly saturated; 3) Admissions occur at a constant rate x^* per iteration; and 4) The composition of active jobs reflects the proportions p_k . At such a state, each type- k job occupies stages $0, \dots, l_1^{(k)} - 1$, and there are $p_k x^*$ jobs in each active stage of type k . Memory balance therefore requires

$$\sum_{k=1}^K p_k \sum_{j=0}^{l_1^{(k)}-1} (l_0^{(k)} + 1 + j) x^* = M.$$

Define the total lifetime memory footprint of a type- k job as

$$C_k = \sum_{j=0}^{l_1^{(k)}-1} (l_0^{(k)} + 1 + j) = l_1^{(k)} \left(l_0^{(k)} + \frac{l_1^{(k)} + 1}{2} \right),$$

Then, the eviction-free admission rate is

$$x^* = \frac{M}{\sum_{k=1}^K p_k C_k}. \quad (8)$$

At this fixed point, every iteration admits x^* new jobs, completes x^* jobs, and maintains a constant occupancy profile across decoding stages. Note that this reduces to the single-class fixed point when $K = 1$.

As before, a central question is whether this eviction-free fixed point is dynamically stable, which we study next. Appendix B.10 records a complementary finite- l_0 calculation for the stricter per-iteration mixing protocol and the resulting pulse cycle.

4.2. Heterogeneity, Synchronization, and Stability

Unlike the homogeneous case, heterogeneity introduces additional structure that can fundamentally alter system dynamics. Requests of different types are completed at different speeds due to the heterogeneous decoding lengths, so their completion times may or may not align over repeated iterations. Such alignment (or lack thereof) determines whether memory-release bursts synchronize and trigger eviction cascades.

To understand when heterogeneity mitigates eviction-induced instability, we study perturbations around the eviction-free fixed point. In the saturated-input regime, the admission and execution rules define a deterministic discrete-time dynamical system. Linearizing this system around the eviction-free equilibrium yields a recurrence that describes how small imbalances propagate.

Let $\Delta_n := x_{n+1}^{(0)} - x_n^{(0)}$ denote the change in stage-0 admissions at iteration n . A type- k request admitted m iterations ago occupies $l_0^{(k)} + 1 + m$ memory tokens if $l_1^{(k)} > m$, i.e., it is still active. Thus, conservation of total memory in the eviction-free regime gives the linearized recurrence

$$\sum_{m=0}^{l_1^{(K)}-1} \left(\sum_{k: l_1^{(k)} > m} p_k (l_0^{(k)} + 1 + m) \right) \Delta_{n-m} = 0. \quad (9)$$

The outer sum ranges over possible lags, while the inner sum includes exactly the request types that remain active at that lag. The equality to zero enforces that total memory remains at capacity when no eviction occurs.

By standard linear recurrence theory (Elaydi 2005), perturbations are linear combinations of modes λ^n , where λ is a root of the characteristic polynomial

$$F(\lambda) = \sum_{m=0}^{l_1^{(K)}-1} \left(\sum_{k: l_1^{(k)} > m} p_k (l_0^{(k)} + 1 + m) \right) \lambda^{l_1^{(K)}-1-m}. \quad (10)$$

Each root corresponds to one perturbation mode: modes with $|\lambda| < 1$ decay, while modes with $|\lambda| > 1$ grow. Thus the linear stability of the eviction-free equilibrium is determined by whether all roots lie inside the unit circle.

Under the prompt-dominated scaling (7), dividing F by L and letting $L \rightarrow \infty$ gives the limiting polynomial

$$A(\lambda) = \sum_{m=0}^{l_1^{(K)}-1} \left(\sum_{k: l_1^{(k)} > m} p_k r_k \right) \lambda^{l_1^{(K)}-1-m},$$

whose coefficients depend only on the decoding lengths, arrival proportions, and prompt-length ratios.

Summing geometric series produces the closed form

$$(1 - \lambda) A(\lambda) = -R \lambda^{l_1^{(K)}} + \sum_{k=1}^K p_k r_k \lambda^{l_1^{(K)} - l_1^{(k)}}, \quad (11)$$

where $R = \sum_{k=1}^K p_k r_k$.

Interestingly, the location of the roots depends only on simple arithmetic properties of the decoding lengths, not on the prompt-length ratios r_k or fine-grained weight differences. The key quantity is the greatest common divisor of the decoding lengths. The following theorem formalizes this dichotomy.

THEOREM 2 (GCD Stability Condition). *For $K \geq 2$ request types, let $g = \gcd(l_1^{(1)}, \dots, l_1^{(K)})$. Under the prompt-dominated scaling (7):*

1. *If $g > 1$, then the eviction-free equilibrium is linearly unstable: the linearized dynamics have $g - 1$ eigenvalues with modulus $|\lambda| = 1 + \Theta(1/L)$, while the remaining $l_1^{(K)} - g$ modes lie strictly inside the unit circle. For generic initial perturbations, these unstable modes grow and eventually drive the system into the eviction regime*
2. *If $g = 1$, then the eviction-free equilibrium is globally asymptotically stable: from any initial condition, the system converges to the eviction-free fixed point.*

The proof of Theorem 2 is provided in Appendix B. We highlight the main spectral mechanism here. Let i denote the imaginary unit, i.e., $i^2 = -1$. When $g > 1$, the limiting polynomial $A(\lambda)$ has roots at all nontrivial g -th roots of unity. To see this, consider $\lambda_j = e^{2\pi i j/g} = \cos(2\pi j/g) + i \sin(2\pi j/g)$ for $j = 1, \dots, g-1$, which

are nontrivial g -th root of unity, i.e., $\lambda_j^g = 1$ and $\lambda \neq 1$. Since g divides every decoding length $l_1^{(k)}$, we have $\lambda^{l_1^{(K)} - l_1^{(k)}} = 1$ for all k , which further implies

$$(1 - \lambda)A(\lambda) = -R + R = 0.$$

Because $\lambda_j \neq 1$, it follows that $A(\lambda_j) = 0$. Thus, the nontrivial g -th roots of unity generate unit-circle modes of the limiting linearized dynamics, independently of the prompt-length ratios r_k . For finite L , the characteristic polynomial is an $O(1/L)$ perturbation of the limiting polynomial. The appendix verifies the required nondegeneracy and shows that each unit-circle root of A perturbs to a nearby finite- L root with displacement of order $O(1/L)$. A first-order perturbation calculation further shows that this displacement is outward, so these modes satisfy $|\lambda| = 1 + \Theta(1/L) > 1$. They therefore produce unstable directions in the linearized dynamics.

When $g = 1$, there are no nontrivial common roots of unity. In this case, the same polynomial representation, together with a triangle-inequality argument, shows that all roots lie strictly inside the unit circle. A Lyapunov argument then upgrades this linear contraction to global convergence to the eviction-free equilibrium.

As a special case, a homogeneous system with decoding length l satisfies $\gcd(l, l) = l > 1$, so the above argument recovers the instability result of Theorem 1 through a spectral characterization.

The dichotomy reflects the synchronization structure of completion events. When $g > 1$, completion times share a common period g , so memory releases occur in synchronized bursts. These bursts reinforce periodic oscillations and can push the system into the eviction regime. When $g = 1$, completion times occupy different phases of the cycle. Memory is released more evenly over time, which damps oscillations and stabilizes the eviction-free state. Figure 3 illustrates this contrast: coprime decoding lengths stagger completions, whereas non-coprime lengths create periodic memory-release spikes.

In the heterogeneous non-coprime case, the instability is typically “milder” than in the homogeneous system. By Theorem 2, only the $g - 1$ modes associated with g -periodic oscillations are unstable, while the remaining $l_1^{(K)} - g$ modes are damped. Thus the system may enter the eviction regime, but the oscillations need not escalate into the full synchronization that produces maximal eviction.

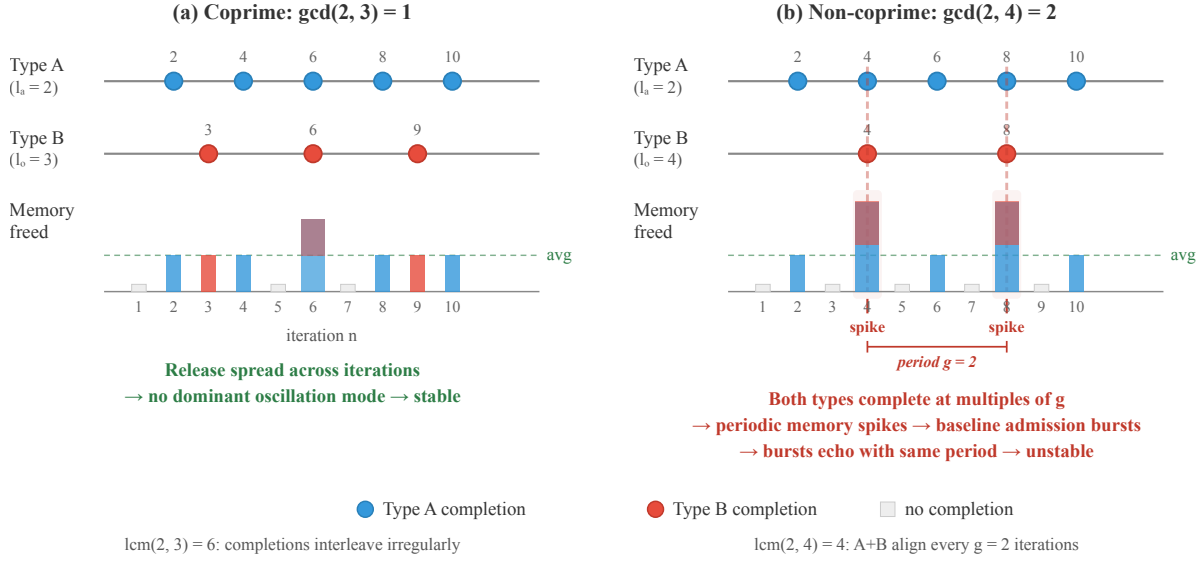


Figure 3 Synchronization mechanism underlying the GCD stability condition. Each dot marks a completion event for Type A (blue, $l_1^{(A)} = 2$) or Type B (red). The bottom row shows the combined memory release per iteration. (a) Coprime decode lengths ($l_1^{(A)} = 2, l_1^{(B)} = 3$): completions are staggered across most iterations, producing relatively even memory release. (b) Non-coprime ($l_1^{(A)} = 2, l_1^{(B)} = 4, g = 2$): both types complete at every even iteration, creating periodic memory-release spikes that drive oscillation.

EXAMPLE 4 (COPRIME DECODE LENGTHS: CONVERGENCE). Let $K = 2$ with $l_1^{(1)} = 2, l_1^{(2)} = 3$, common prompt length $l_0^{(1)} = l_0^{(2)} = l_0 = 50$, arrival proportions $p_1 = p_2 = 1/2$, and $M = 518$, giving equilibrium admission $x^* = 4$.

With $r_1 = r_2 = 1$, the closed form (11) yields

$$A(\lambda) = \lambda^2 + \lambda + \frac{1}{2},$$

whose roots are $\lambda = (-1 \pm i)/2$, so $|\lambda| = 1/\sqrt{2} \approx 0.707$. Since $\gcd(2, 3) = 1$, all roots lie strictly inside the unit circle, and perturbations decay geometrically. Figure 4(a) illustrates this behavior: starting from a 25% over-admission ($x_0^{(0)} = 5$), the system converges to x^* within 2% by iteration 8. Intuitively, no oscillation period can synchronize completions of both types. A period-2 fluctuation in admissions propagates through type-1 completions (delay 2) but is phase-shifted by type-2 completions (delay 3), causing oscillatory modes to interfere destructively.

EXAMPLE 5 (NON-COPRIME DECODE LENGTHS: DIVERGENCE). Let $K = 2$ with $l_1^{(1)} = 2$, $l_1^{(2)} = 4$, common prompt length $l_0^{(1)} = l_0^{(2)} = l_0 = 50$, arrival proportions $p_1 = p_2 = 1/2$. Setting $M = 626$ again yields $x^* = 4$.

Because $\gcd(2, 4) = 2$, the limiting polynomial $A(\lambda)$ places a root at $\lambda = -1$ on the unit circle. For finite $l_0 = 50$, perturbation analysis shows that this root moves outside the unit circle to $|\lambda| \approx 1.019$. Figure 4(b) shows the resulting dynamics: the same initial perturbation now produces oscillations whose amplitude grows over time.

The instability arises from period-2 synchronization. Both type-1 (delay 2) and type-2 (delay 4) completions echo a period-2 fluctuation in admissions, reinforcing the oscillation rather than damping it.

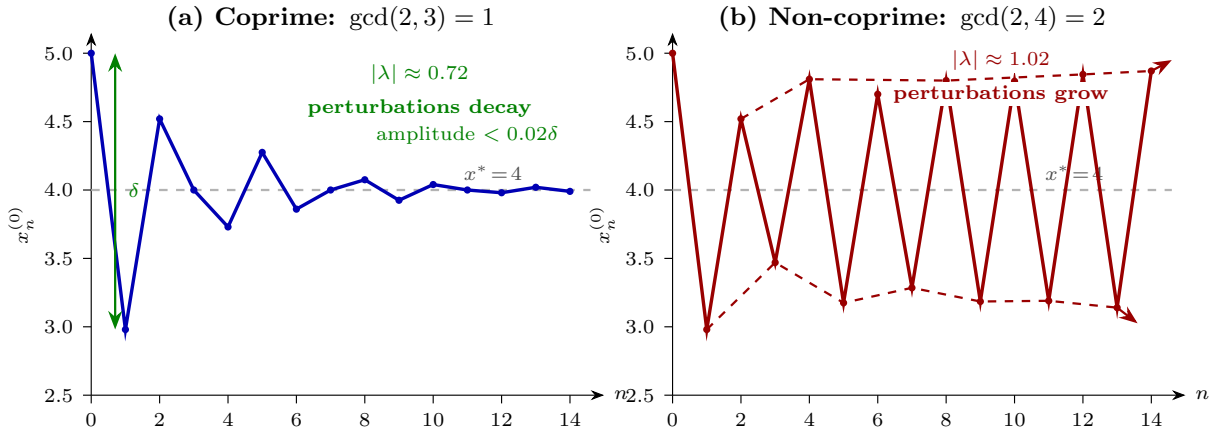


Figure 4 Admission trajectories for two multi-class systems ($l_0 = 50$, $p = q = 1/2$, $x^* = 4$, starting at $x_0^{(0)} = 5$). (a) Coprime decode lengths ($l_A = 2$, $l_B = 3$): the limiting polynomial $A(\lambda) = \lambda^2 + \lambda + \frac{1}{2}$ has roots with modulus $|\lambda| \approx 0.72$; the perturbation decays to within 2% of x^* by iteration 8. (b) Non-coprime decode lengths ($l_A = 2$, $l_B = 4$, $\gcd = 2$): $A(\lambda)$ has a root at $\lambda = -1$ (the non-trivial second root of unity), which drifts to $|\lambda| \approx 1.02$ for finite l_0 (Theorem 2); oscillations grow geometrically (dashed envelope), driving the system toward eviction. Both trajectories are computed from the finite- l_0 recurrence (9).

REMARK 2 (INVARIANCE TO PROMPT-LENGTH RATIOS). The GCD stability condition depends only on the decoding lengths $l_1^{(1)}, \dots, l_1^{(K)}$ and is invariant to the prompt-length ratios $r_1, \dots, r_K$. Intuitively, in the prompt-dominated regime the prompt lengths affect the overall memory scale but not the timing structure

of the decoding pipeline. Consequently, the stability boundary is determined entirely by the arithmetic relationship among the decoding lengths. For finite L , the ratios r_k do influence the growth rate of the unstable modes (through the $\Theta(1/L)$ perturbation) but not whether instability occurs. A detailed perturbation analysis for heterogeneous prompts is provided in Appendix B.

4.3. Finite Prompt Length and Stability Threshold

Theorem 2 characterizes stability in the prompt-dominated limit. We now ask how large the prompt length must be for a finite system to inherit the coprime stability predicted by that limit. For clarity, we focus on two request types with a common prompt length l_0 , coprime decoding lengths: $l_1^{(1)} < l_1^{(2)}$, $\gcd(l_1^{(1)}, l_1^{(2)}) = 1$, and arrival proportions p and $1 - p$.

Let

$$\rho_\infty := \max\{|\alpha| : A(\alpha) = 0\}$$

be the spectral radius of the limiting polynomial (11). By Theorem 2, the coprime condition implies $\rho_\infty < 1$. At finite prompt length, however, the characteristic roots of the linearized system drift outward. The following theorem quantifies this drift. Write $\varepsilon := 1/(l_0 + l_1^{(2)})$ and $\beta = l_1^{(1)}/l_1^{(2)} \in (0, 1)$. We also write the corresponding finite- l_0 characteristic equation as $F_\varepsilon(\lambda)$. It decomposes as $F_\varepsilon(\lambda) = \frac{1}{\varepsilon}A(\lambda) - \lambda A'(\lambda)$.

THEOREM 3 (Spectral drift under finite prompt length). *Assume $\gcd(l_1^{(1)}, l_1^{(2)}) = 1$. Let α be a simple root of the limiting polynomial A , i.e., $A(\alpha) = 0$ and $A'(\alpha) \neq 0$. Then, there exists a unique root branch $\lambda_\alpha(\varepsilon)$ of F_ε with $\lambda_\alpha(0) = \alpha$, and constants $\varepsilon_\alpha > 0$ and $K_\alpha > 0$, such that for all $0 \leq \varepsilon \leq \varepsilon_\alpha$,*

$$|\lambda_\alpha(\varepsilon) - \alpha(1 + \varepsilon) - c_2(\alpha)\varepsilon^2| \leq K_\alpha \varepsilon^3, \text{ where } c_2(\alpha) = \frac{\alpha}{2} \left(\frac{\alpha A''(\alpha)}{A'(\alpha)} + 2 \right). \quad (12)$$

Along coprime families with $l_1^{(2)} \rightarrow \infty$, the two dominant limiting branches are simple. Let $\rho(\varepsilon)$ denote the spectral radius of the corresponding finite- l_0 linearization, F_ε . On each dominant branch, the second-order modulus correction is positive. Moreover, there exist constants $E > 0$, $K_\rho > 0$, and $N_\rho \geq 1$ such that, for all $l_1^{(2)} \geq N_\rho$ and all $0 \leq \varepsilon \leq E(l_1^{(2)})^{-3}$,

$$|\rho(\varepsilon) - \rho_\infty(1 + \varepsilon)| \leq K_\rho l_1^{(2)} \varepsilon^2. \quad (13)$$

Theorem 3 shows that the dominant root satisfies

$$\rho(\varepsilon) = \rho_\infty(1 + \varepsilon) + O(l_1^{(2)} \varepsilon^2). \quad (14)$$

For a finite prompt length, stability is governed by a competition between the limiting spectral gap $1 - \rho_\infty$ and the finite-prompt perturbation ε . To first order, stability requires

$$\rho_\infty(1 + \varepsilon) < 1, \quad (l_0 + l_1^{(2)})(1 - \rho_\infty) \gtrsim 1.$$

Let

$$l_0^{\min} := \min \left\{ l_0 \in \mathbb{Z}_{\geq 0} : \rho((l_0 + l_1^{(2)})^{-1}) < 1 \right\}.$$

We next send $l_1^{(2)}$ to infinity and study how $l_0^{\min}$ scales with $l_1^{(2)}$. The following corollary shows that along coprime families with $l_1^{(2)} \rightarrow \infty$, $(l_0^{\min} + l_1^{(2)})(1 - \rho_\infty) = 1 + o(1)$. The proof in Appendix B.7 also rules out stable islands before this first stable prompt length in the physical range $0 \leq (l_0 + l_1^{(2)})^{-1} \leq (l_1^{(2)})^{-1}$.

COROLLARY 1 (Stability threshold). *Along any coprime family with $l_1^{(2)} \rightarrow \infty$, there exist constants $K_S > 0$ and $N_0 \geq 1$ such that, for all $l_1^{(2)} \geq N_0$, the minimum stable prompt length satisfies*

$$\left| (l_0^{\min} + l_1^{(2)})(1 - \rho_\infty) - 1 \right| \leq K_S (l_1^{(2)})^{-2}.$$

Corollary 1 shows that $l_0^{\min} \sim (1 - \rho_\infty)^{-1}$. The next proposition evaluates the gap $1 - \rho_\infty$ itself.

PROPOSITION 1 (Spectral gap scaling). *Along any coprime family with $l_1^{(2)} \rightarrow \infty$, there exist constants $K_{\text{gap}} > 0$ and $N_{\text{gap}} \geq 1$ such that, for all $l_1^{(2)} \geq N_{\text{gap}}$,*

$$\left| 1 - \rho_\infty - c(\beta, p) (l_1^{(2)})^{-3} \right| \leq K_{\text{gap}} (l_1^{(2)})^{-4}, \text{ where } c(\beta, p) = \frac{2\pi^2 p(1-p)}{((1-p) + p\beta)^3}. \quad (15)$$

From Proposition 1, for large coprime decoding lengths, we have $1 - \rho_\infty \simeq c(\beta, p)(l_1^{(2)})^{-3}$. Combining this with Corollary 1 yields the scaling law

$$l_0^{\min} \sim \frac{((1-p) + p\beta)^3}{2\pi^2 p(1-p)} (l_1^{(2)})^3. \quad (16)$$

The cubic scaling has a direct operational interpretation. Although coprime decoding lengths stabilize the eviction-free fixed point in the prompt-dominated limit, the prompt length required to see this stabilization

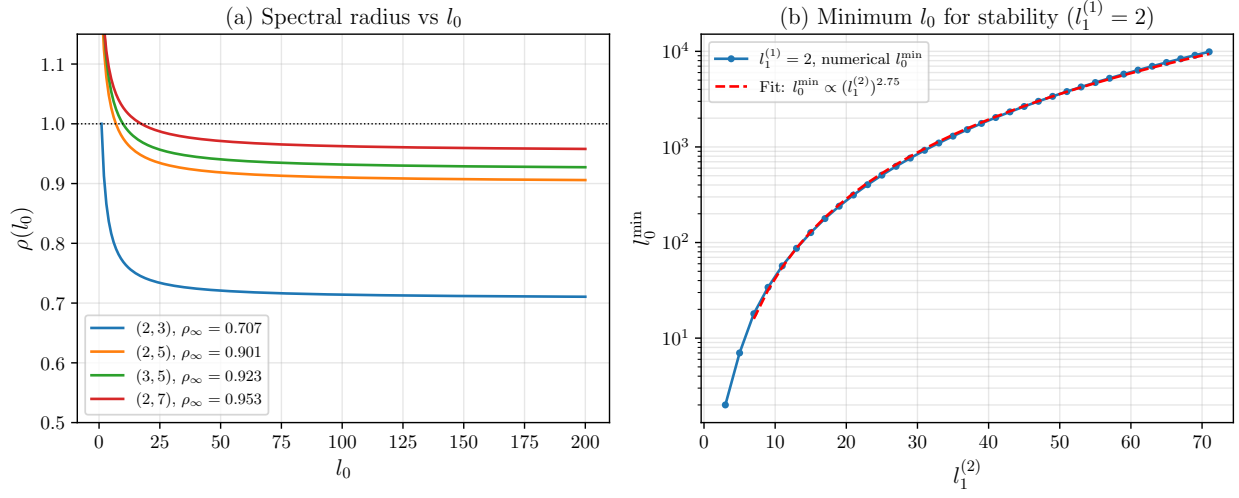


Figure 5 Finite- l_0 spectral radius for coprime pairs ($p = 1/2$). (a) Spectral radius $\rho(l_0)$ of the characteristic polynomial $F(\lambda)$ as l_0 increases, for four coprime pairs $(l_1^{(1)}, l_1^{(2)})$. Each curve converges to the limiting value ρ_∞ (dashed line at $\rho = 1$). The pair $(2, 7)$ requires $l_0 \geq 15$ to enter the stable regime. (b) Minimum prompt length $l_0^{\min}$ for all eigenvalues to lie inside the unit circle ($l_1^{(1)} = 2$). The log-log fit confirms the $\Theta((l_1^{(2)})^3)$ scaling predicted by Corollary 1.

at a finite scale grows cubically with the longer decoding length. Interestingly, the prefactor also depends on the degree of heterogeneity: near-homogeneous workloads, with $\beta \simeq 1$, have a smaller limiting spectral gap and therefore require larger prompts than highly heterogeneous workloads, with $\beta \simeq 0$.

For any fixed coprime pair, the exact finite threshold can be computed by finding the roots of the finite- l_0 characteristic polynomial. Figure 5 compares these numerical thresholds with the asymptotic prediction (16), confirming both the radial spectral drift and the cubic threshold scaling. The proofs of Theorem 3, Corollary 1, and Proposition 1 are given in Appendix B.

5. Eviction-Aware Control Policies

Theorems 1 and 2 uncover a single structural source of instability of the eviction-free equilibrium: positive feedback between memory release and admission under progressive memory growth. How this feedback manifests depends on workload composition. In homogeneous systems, it emerges as admission pulses that amplify through the decoding pipeline; in heterogeneous systems, under the large prompt scaling, arithmetic synchronization determines whether those pulses reinforce or dissipate. Based on these insights, in this section, we propose eviction-aware control policies.

The policies below are intentionally stylized. Each targets a specific manifestation of the underlying feedback mechanism and illustrates how the structural insights developed earlier translate into concrete design principles. They are not intended as complete production schedulers, but rather as minimal interventions that eliminate eviction at its structural source.

5.1. Rate-Limited Admission

The instability results above identify a common mechanism across homogeneous workloads, non-coprime heterogeneous workloads, and coprime workloads below the finite-prompt stability threshold: transient over-admission. When $X_n^{(0)}$ exceeds some eviction-free sustainable rate, the excess mass enters the decoding pipeline, and its memory footprint grows over subsequent stages. The resulting amplification can push the system into eviction and even initiate a support-loss cascade. A natural remedy is therefore to cap admission at the eviction-free equilibrium rate:

$$X_n^{(0)} = \min(Q_n, S_n/(l_0 + 1), x^*),$$

where x^* is the eviction-free equilibrium rate derived in Sections 3 and 4, Q_n is the number of requests waiting in the queue (so admissions cannot exceed available demand), and S_n is the GPU memory available after execution and eviction at iteration n (the term $S_n/(l_0 + 1)$ represents the maximum number of requests that can be admitted without violating the memory constraint). This cap removes the admission pulses that drive the instability while preserving the eviction-free throughput whenever sufficient demand and memory are available.

Our analysis is conducted in the saturated-input regime, where a backlog of requests is always available for admission. In practice, systems rarely operate persistently in this regime. Instead, demand is typically below capacity, although fluctuations and temporary surges may occasionally push the system toward saturation. Nevertheless, the analysis identifies the critical admission boundary that prevents eviction once the system approaches capacity. The proposed policy adapts naturally to non-saturated conditions. When demand is low ($Q_n < x^*$), the threshold is non-binding and requests are admitted immediately, preserving low latency (Appendix C). In particular, the policy does not require waiting to accumulate a large batch of requests,

which would unnecessarily increase latency. When demand surges and the system approaches saturation, the rate cap activates automatically and limits admissions to the sustainable rate x^* , preventing admission pulses and the eviction cascades they would otherwise trigger.

5.2. Request Mixing for Heterogeneous Systems

When heterogeneous request types are available, Theorem 2 suggests a complementary control mechanism that operates at the routing level. Rather than assigning requests to GPUs arbitrarily or grouping identical requests together, the system can route requests so that each GPU processes a diversified mixture of decoding lengths whose greatest common divisor equals one. The motivation follows directly from the instability mechanism identified earlier. In non-coprime systems, decoding completions synchronize at multiples of $g = \gcd(l_1, \dots, l_K)$, producing periodic bursts of memory release that amplify admission fluctuations and trigger eviction. By contrast, when the decoding lengths assigned to a GPU are coprime, completion events occur at desynchronized phases, smoothing memory release over time.

In practice, serving systems often group similar requests together to improve batching efficiency and cache locality (Yu et al. 2022, Zheng et al. 2024). While such specialization simplifies scheduling and improves GPU utilization, it may also concentrate workloads with identical decoding lengths on the same device, creating precisely the synchronization conditions that lead to eviction cycles. Our analysis does not imply that requests with drastically different decoding lengths must be mixed. Stability can often be improved through mild diversification: assign together request types with comparable decoding lengths but avoid nontrivial common periods among their lengths. Such routing preserves most of the batching and memory-efficiency benefits of specialization while reducing the synchronization of completion events that drives eviction-induced instability.

5.3. Design Implications

The structural results above suggest several practical guidelines for the design and monitoring of memory-constrained LLM serving systems.

Arithmetic structure as a stability diagnostic. The stability of the system depends critically on the arithmetic structure of decoding lengths. When requests assigned to a GPU share a common divisor, completion events can synchronize and trigger eviction cycles. In contrast, mixtures of coprime decoding lengths naturally desynchronize completions and stabilize the system. Monitoring the effective greatest common divisor of active workloads therefore provides a simple structural diagnostic for eviction risk.

Engineering stability through mild diversification. Stability can often be achieved with only minor diversification in decoding lengths. For example, if service-level agreements define discrete output-length tiers such as $\{128, 256, 512\}$, all of which share $\text{gcd} = 128$, small adjustments (e.g., $\{127, 255, 509\}$) restore coprimality and eliminate synchronization effects. Because the stability condition is discrete, any perturbation achieving $\text{gcd} = 1$ suffices. This provides a robust and low-cost design lever.

Operational practices that route similar request types to dedicated GPUs can improve batching efficiency and cache locality. At the same time, incorporating a modest level of diversification in the requests assigned to each GPU can enhance system stability by preventing the synchronization of completion events that leads to eviction cycles. This suggests that efficient and stable operation can often be achieved by balancing specialization with mild heterogeneity in decoding lengths across requests.

Eviction as an early warning signal. The transition from zero eviction to persistent eviction is a structural phase transition (Theorem 1), rather than a gradual degradation. Once the sustainable admission boundary is exceeded, the system can converge to an eviction-driven limit cycle with sustained throughput loss. Persistent eviction therefore indicates operation near an unstable equilibrium and can serve as a sensitive indicator of system health.

From a control perspective, this observation also highlights the importance of admission regulation. Monitoring eviction rates provides a practical signal for when admission should be moderated to remain within the stable operating region. Policies such as the rate-limited admission rule described above can then act as safeguards, preventing transient demand surges from pushing the system into the unstable regime.

6. Numerical Experiments

[JD: Things we need to show:

1. Our saturated-input regime analysis has broader implications: Just for our stylized stochastic model (Poisson arrivals), when $\lambda < \bar{T}_{l_0-1}$ (the worst case throughput, the system is stable; when $\lambda > \bar{T}_{l_0-1}$, the system is unstable. Interestingly, the system is unstable even if $\lambda < x^*$.
2. Different aspects of request mixing: Just for our stylized stochastic model, and can have different arrival patterns (e.g., the one Ruicheng is experimenting with). The set of experiments aims to show different things our theory does not fully capture: e.g., even in the non-coprime case, mixing can help.
3. The phenomenon we observe also exists in real systems (Vidur simulator (Agrawal et al. 2024) and real GPU): instability of eviction-free equilibrium. For this, we can use an arbitrary demand.
4. Effectiveness of the proposed remedy in real systems: rate-limited admission control and request mixing under Sarathi (Agrawal et al. 2023).

]

The experiments test the stability mechanisms outside the deterministic saturated-input setting. A model-based simulator, implemented directly from the memory and admission dynamics in Section 2, shows that stochastic arrivals inherit the same worst-cycle stability boundary. Vidur and real-GPU experiments then show that the same synchronization mechanism appears with serving-system batching and memory accounting. Across these settings, request mixing reduces synchronized completions, rate-limited admission prevents eviction cascades, and short arrival bursts expose the time needed to assemble a heterogeneous pool. Appendix D contains additional model-based, Vidur, and real-GPU experiments.

6.1. Model-based simulations show the open-system threshold

The saturated-input analysis predicts that greedy admission loses stability at the worst-cycle throughput, not at the eviction-free fixed point. To test this prediction under open arrivals, the model-based simulator keeps the state variables and memory accounting of Section 2 but feeds the system with a Poisson arrival process. Let Q_n denote the number of requests waiting at iteration n : new arrivals join Q_n , evicted requests rejoin Q_n , and the admission rule draws from Q_n subject to memory feasibility.

The simulations reproduce the mechanism behind Theorem 1. Under greedy admission, trajectories converge to the level- $(l_1 - 1)$ eviction cycle, and small perturbations push the eviction-free fixed point away

from stability. The open-system stability region is therefore governed by $\bar{T}_{l_1-1}$, which can be up to 50% below the eviction-free service rate when $l_1 \gg l_0$.

Figure 6 shows the boundary directly. A Poisson load $\lambda < \bar{T}_{l_1-1}$ keeps Q_n near zero, while any $\lambda > \bar{T}_{l_1-1}$, including the interval $(\bar{T}_{l_1-1}, x^*)$ that the eviction-free fluid calculation deems feasible, produces persistent evictions and a growing Q_n . Thus greedy admission becomes unstable before the nominal eviction-free capacity is reached.

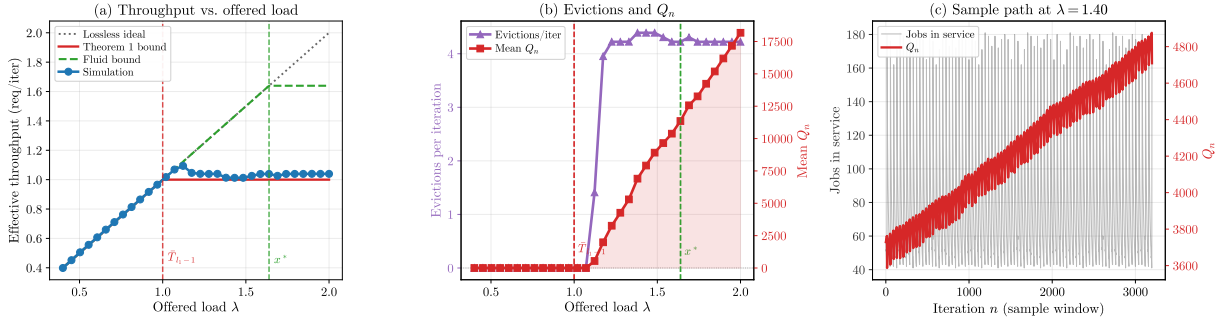


Figure 6 Open-system stability under Poisson arrivals in the model-based simulator. With $M = 2000$, $l_0 = 10$, and $l_1 = 40$, the worst-cycle service rate is $\bar{T}_{l_1-1} = 1.00$, while the eviction-free fluid optimum is $x^* = 1.64$. **(a)** Throughput departs from the lossless line once λ crosses $\bar{T}_{l_1-1}$, despite $\lambda < x^*$. **(b)** Evictions per iteration and mean queue length Q_n are negligible below $\bar{T}_{l_1-1}$, then become positive above it. **(c)** A raw sample path at $\lambda = 1.40 \in (\bar{T}_{l_1-1}, x^*)$ shows persistent evictions and growing Q_n . Detailed settings and additional GCD simulations appear in Appendix D.1.

6.2. Mixing reduces synchronization beyond coprime cases

Theorem 2 predicts a sharp coprime stabilization effect. In operation, however, routing can also partially reduce the common periodicity. Such partial desynchronization can still improve performance by weakening the synchronized completion pulses that trigger eviction.

We use Vidur, a validated large-scale LLM-serving simulator with reported latency and throughput errors below 5% against real systems (Agrawal et al. 2024). Unless otherwise noted, Vidur experiments use the Sarathi scheduler (Agrawal et al. 2023), Meta-Llama-3-8B, and a single NVIDIA A100 GPU. Sarathi greedily admits pending requests when memory is available. Our rate-limited variant adds the global population cap suggested by the eviction-free equilibrium. When resident KV cache later exceeds capacity, Vidur records a

restart, meaning that the request loses its current KV cache and must be recomputed from prefill. We apply the same LPF eviction priority as in the model and report Vidur restarts and evicted-request counts as the system-level counterparts of model evictions. In the coprime case, mixing removes the common period. In the partial-GCD case, mixing reduces the synchronization scale without making the combined GCD equal to one. Both cases improve the operating point, as shown in Figure 7.

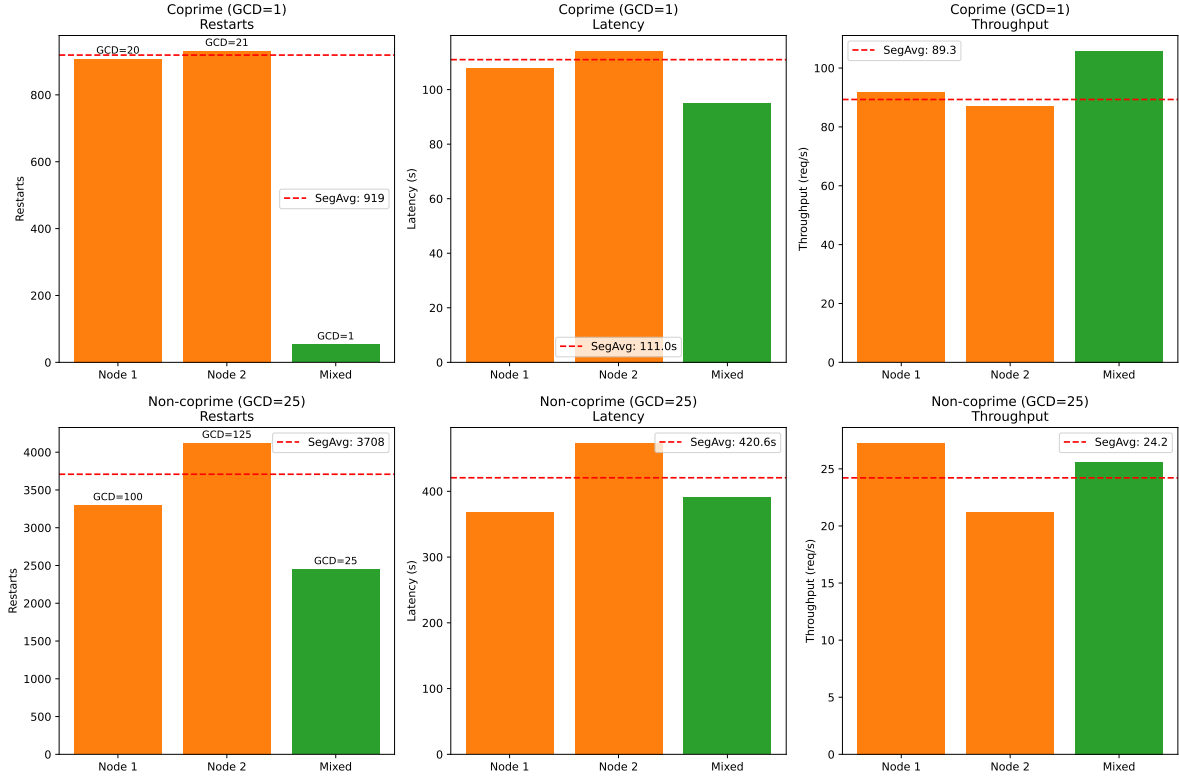


Figure 7 Request mixing with the Sarathi scheduler in Vidur. Segregated routing isolates request classes on separate nodes and creates synchronized memory-release patterns; mixed routing combines the classes and reduces the effective GCD. The coprime case gives the cleanest stabilization; the partial-GCD case shows that mixing can still help without reaching $GCD = 1$.

The performance gains match this mechanism. In the coprime configuration, mixing sharply reduces restarts and improves both latency and throughput. In the partial-GCD configuration, the restart reduction is smaller, but the direction is the same: fewer restarts, lower latency, and higher throughput. The mechanism is arithmetic complementarity. Routing request classes together can reduce the combined GCD even when

Table 1 Request mixing with the Sarathi scheduler in Vidur (Meta-Llama-3-8B, NVIDIA A100 GPU, 20k requests, $l_0 = 512$).

Both coprime (GCD= 1) and non-coprime (GCD= 25) configurations improve eviction, latency, and throughput.

Configuration	GCD	Restarts		Latency (s)		Throughput (req/s)
		Value	Improv.	Value	Improv.	
Node 1	20	908	—	107.8	—	91.7
Node 2	21	931	—	114.1	—	87.0
SegAvg	—	919	—	111.0	—	89.3
Mixed	1	53	−94.2%	95.1	−14.3%	105.8
Node 1	100	3296	—	367.9	—	27.3
Node 2	125	4121	—	473.4	—	21.2
SegAvg	—	3708	—	420.6	—	24.2
Mixed	25	2449	−34.0%	390.4	−7.2%	25.6

each separate node has a large GCD. Appendix D.2 contains the corresponding model-based figures, Vidur memory figures, and partial-GCD results.

The same pattern appears on real GPU hardware. We run SGLang (Zheng et al. 2024), which uses continuous batching and therefore aligns with the model’s per-iteration dynamics, on an NVIDIA A800 80 GB GPU with Qwen2.5-1.5B-Instruct, 60k requests, and prefill length $l_0 = 300$. Node 1 uses decode lengths $D = \{5, 10\}$ at ratio 1:1 (GCD= 5); Node 2 uses $D = \{6, 12\}$ at ratio 1:1 (GCD= 6); the mixed configuration combines all four lengths $D = \{5, 6, 10, 12\}$ at ratio 1:1:1:1 (GCD= 1). Figure 8 shows that the segregated configurations produce periodic eviction bursts, while mixing suppresses them throughout the run. Table 2 quantifies the gains: real evictions fall by 97.7%, mean latency drops by 18.2%, and throughput rises by 30.5% relative to the segregated average.

Table 2 Request mixing on real GPU hardware with SGLang (Zheng et al. 2024). Mixed routing (GCD= 1) nearly eliminates evictions and improves latency and throughput relative to the segregated average.

Configuration	GCD	Evictions		Latency (s)		Throughput (tok/s)
		Value	Improv.	Value	Improv.	
Node 1	5	422	—	5.44	—	2,731
Node 2	6	372	—	5.52	—	3,264
SegAvg	—	397	—	5.48	—	2,998
Mixed	1	9	−97.7%	4.48	−18.2%	3,911

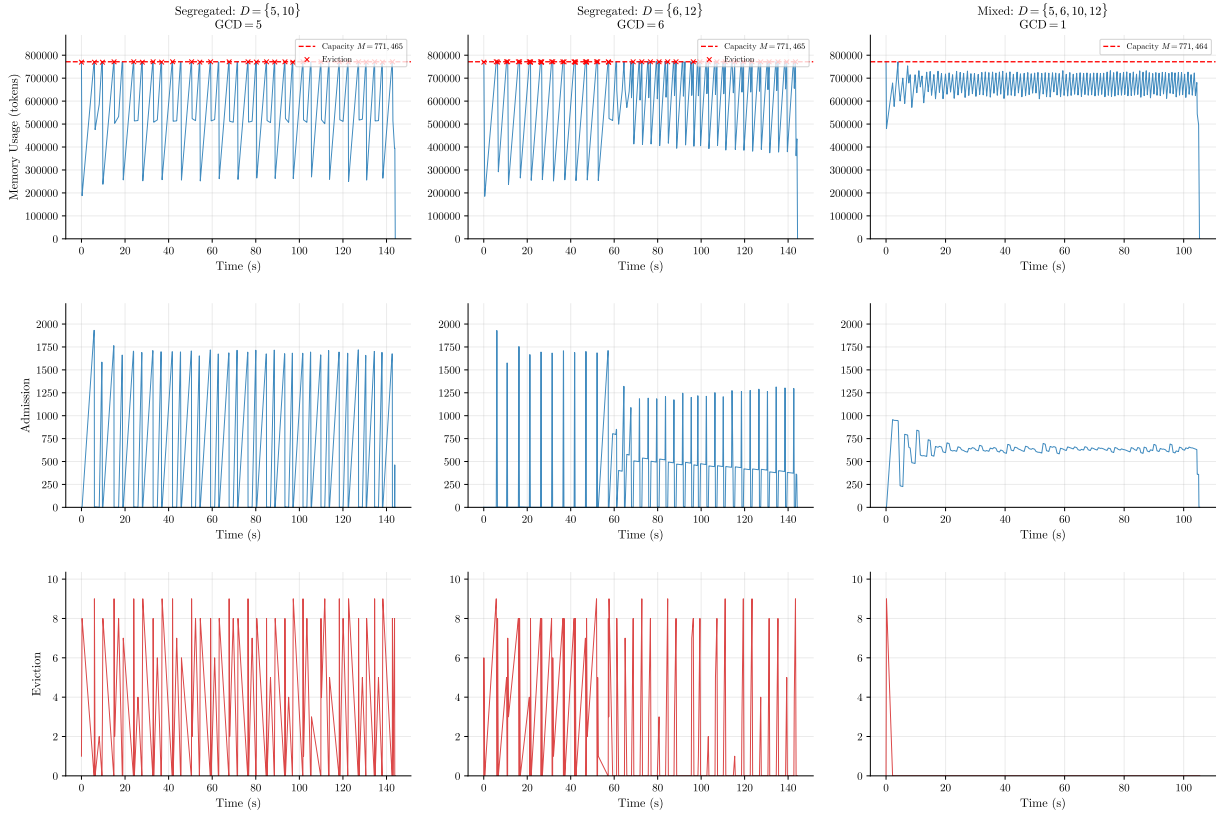


Figure 8 Real admission (top) and real eviction (bottom) over wall-clock time for three decode-length configurations. Segregated routing ($D = \{5, 10\}$ and $D = \{6, 12\}$) produces recurring eviction bursts at fixed intervals; mixed routing ($D = \{5, 6, 10, 12\}$, GCD= 1) eliminates the periodic pattern. x-axis: timestamp (seconds).

6.3. Rate limits remove Vidur restarts

Greedy admission fails because it reacts only to current memory feasibility and ignores future KV cache growth. Rate-limited admission uses x^* as a cap on the active population, preventing the overshoot that triggers eviction cascades.

The Vidur simulator experiment in Table 3 uses decode lengths $\{100, 125, 200, 250\}$ and arrival rate 300 req/s. Sarathi admits every request that fits in available memory, producing periodic memory overflow and eviction cascades. Rate-limited admission uses a per-iteration cap of 5 new requests, approximating the steady-state population implied by x^* . This eliminates eviction, lowers latency by 18.9%, and raises throughput by 28.3%.

Table 3 Rate-limited admission in Vidur with Sarathi, Meta-Llama-3-8B, and an NVIDIA A100 GPU. The workload has 20k requests, $l_0 = 512$, $l_1 \in \{100, 125, 200, 250\}$, and arrival rate 300 req/s.

Scheduler	Restarts	Evicted	Latency (s)	Throughput (req/s)
Sarathi	2,232	2,232	234.2	42.7
Sarathi + rate-limited admission	0	0	189.9	54.8
Improvement	−100.0%	−100.0%	+18.9%	+28.3%

The sample path in Figure 9 makes the mechanism visible. Under Sarathi, memory repeatedly rises above the capacity line and each excess is resolved by forced eviction. Under rate-limited admission, memory remains in a narrow steady-state band and never crosses capacity.

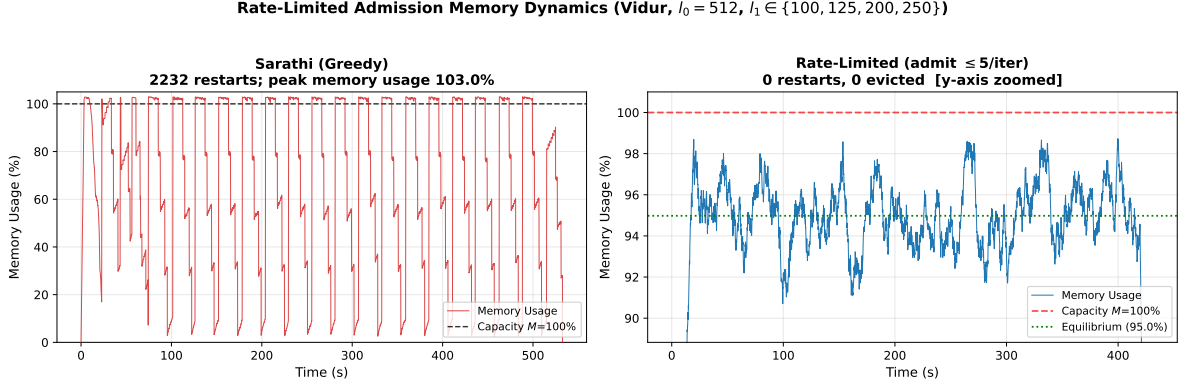


Figure 9 Rate-limited admission stabilizes Vidur memory dynamics. Sarathi exceeds the $M = 100\%$ capacity line in 25.2% of scheduling cycles and incurs 2,232 restarts. Rate-limited admission keeps memory below capacity and eliminates eviction. Additional rate-limit sensitivity results appear in Appendix D.4.

6.4. Burst windows limit mixing

Mixing requires overlap among distinct request types. Under bursty arrivals, the high-rate window may end before the scheduler sees enough types to form the intended mixed pool. This creates a finite-window limitation for mixing.

The model-based ablation in Figure 10 isolates this finite-window mechanism. Even for a favorable coprime type set, burst duration 10 often contains only one or two active types, so the realized GCD rarely equals one. By duration 50 the probability of effective GCD 1 rises to 0.84; by duration 100 it reaches 0.97. Mixing helps once the burst window is long enough to realize the heterogeneous composition. Under shorter transients, rate limiting remains the direct stability mechanism.

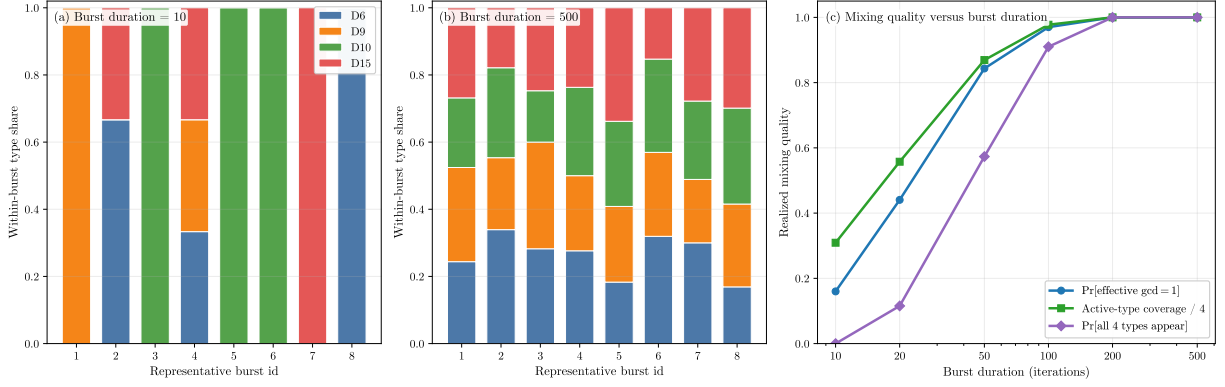


Figure 10 Window-length ablation for finite burst mixing in the model-based simulator. Short bursts may not contain enough active types to realize the intended coprime pool; longer bursts do. Full Vidur and model-based burst sweeps are reported in Appendix D.5.

7. Conclusion

This paper develops a queueing-theoretic framework for analyzing congestion dynamics in memory-constrained LLM serving systems. By isolating the effect of progressive memory accumulation—a feature absent from classical queueing models—we identify structural phenomena that fundamentally alter system behavior under load.

Our analysis yields two main results. First, in single-class systems, baseline admission is inherently unstable: the system possesses multiple equilibria, and only the worst-case eviction cycle is asymptotically stable (Theorem 1). This instability arises from a positive feedback mechanism—admission pulses that propagate and amplify through the decoding pipeline—that is a structural consequence of growing memory footprints. Second, in multi-class systems, the stability landscape is governed by the greatest common divisor of the decoding lengths (Theorem 2): coprime lengths naturally stabilize the system by desynchronizing memory release, while non-coprime lengths permit synchronized peaks that trigger eviction cascades.

These results characterize fundamental mechanisms that any memory-constrained serving system must contend with, independent of specific implementation details. The instability phenomena we identify arise from first principles—the combination of progressive memory accumulation, finite capacity, and work-destroying eviction—and persist across a wide range of system parameters.

The illustrative policies we derive (rate-limited admission and request mixing) demonstrate that the identified mechanisms are actionable: simple, principled interventions can eliminate eviction and restore optimal

throughput. More broadly, the analysis provides a conceptual foundation for reasoning about congestion, stability, and efficiency in stateful AI services where capacity consumption evolves during service.

Future Directions

The deterministic decoding lengths assumed throughout provide analytical tractability but suppress the variability of real LLM outputs. Extending the model to stochastic decoding lengths—and understanding how output length uncertainty shifts the stability boundaries—is an important open question.

On the modeling side, the multi-class analysis currently treats two types with a common prompt length. Generalizing to arbitrary mixtures with continuous length distributions would yield more realistic characterizations, and connecting the discrete-time limit cycle structure to oscillations in fluid models of LLM serving would clarify when each framework is appropriate.

More broadly, the role of coprimality in determining system stability and the fragility of specialization point to unexplored connections between queueing theory, number theory, and AI systems design.

References

- Agrawal A, Kedia N, Mohan J, Panwar A, Kwatra N, Gulavani BS, Ramjee R, Tumanov A (2024) Vidur: A large-scale simulation framework for llm inference. *Proceedings of Machine Learning and Systems*, volume 6.
- Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani BS, Ramjee R (2023) SARATHI: Efficient llm inference by piggybacking decodes with chunked prefills.
- Aminabadi RY, Rajbhandari S, Zhang M, Awan AA, Li C, Li D, Zheng E, Rasley J, Smith S, Ruwase O, He Y (2022) Deepspeed inference: Enabling efficient inference of transformer models at unprecedented scale.
- Ao R, Dong J, Liu A, Copenhaver M (2025a) Proactive transfer admission control for emergency departments. Working paper.
- Ao R, Fu H, Simchi-Levi D (2024) Two-stage online reusable resource allocation: Reservation, overbooking and confirmation call.
- Ao R, Luo G, Simchi-Levi D, Wang X (2025b) Optimizing llm inference: Fluid-guided online scheduling with memory constraints.

-
- Bäuerle N (2002) Asymptotic optimality of tracking policies in stochastic networks. *The Annals of Applied Probability* 12(2):583–600.
- Chen Z, Ye Y, Zhou Z (2025) Adaptively robust llm inference optimization under prediction uncertainty.
- DeepSeek-AI (2024) Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model.
- Devanur NR, Hayes TP (2009) The adwords problem: online keyword matching with budgeted bidders under random permutations. *Proceedings of the 10th ACM Conference on Electronic Commerce*, 71–78.
- Dong J, Feldman P, Yom-Tov GB (2015) Service systems with slowdowns: Potential failures and proposed solutions. *Operations Research* 63(2):305–324.
- Dong J, Hu Y, Wang S (2024) Value of sparse structures in dynamic reusable resource allocation with waiting. Working paper, Columbia University.
- Elaydi SN (2005) *An Introduction to Difference Equations* (Springer), 3rd edition.
- Im S, Moseley B (2015) Competitive scheduling from competitive equilibria. *Proceedings of the 47th Annual ACM Symposium on Theory of Computing*, 220–229.
- Jaillet P, Jiang J, Mellou K, Molinaro M, Podimata C, Zhou Z (2025) Online scheduling for llm inference with kv cache constraints.
- Kleinrock L (1967) Time-shared systems: A theoretical treatment. *Journal of the ACM* 14(2):242–261.
- Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu CH, Gonzalez JE, Zhang H, Stoica I (2023) Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.
- Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih Wt, Rocktäschel T, Riedel S, Kiela D (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.
- Li Y, Dai J, Peng T (2025) Throughput-optimal scheduling algorithms for llm inference and ai agents.
- Lippman SA (1975) Applying a new device in the optimization of exponential queuing systems. *Operations Research* 23(4):687–710.

- Liu Y, Whitt W (2012) A logarithmic safety staffing rule for contact centers with call blending. *Management Science* 58(2):341–357.
- Lucier B, Paes Leme R, Tardos É (2012) On revenue in the generalized second price auction. *Proceedings of the 21st International Conference on World Wide Web*, 361–370.
- Maglaras C (2000) Discrete-review policies for scheduling stochastic networks: trajectory tracking and fluid-scale asymptotic optimality. *The Annals of Applied Probability* 10(3):897–929.
- Mandelbaum A, Massey WA, Reiman MI (1998) Strong approximations for markovian service networks. *Queueing Systems* 30(1):149–201.
- Massoulié L, Roberts J (2002) Bandwidth sharing: objectives and algorithms. *IEEE/ACM Transactions on Networking* 10(3):320–328.
- Mitzenmacher M, Shahout R (2025) Queueing, predictions, and llms: Challenges and open problems.
- Naor P (1969) The regulation of queue size by levying tolls. *Econometrica* 37(1):15–24.
- NVIDIA (2023) Tensorrt-llm: A tensorrt toolbox for optimized large language model inference. <https://github.com/NVIDIA/TensorRT-LLM>.
- Patel P, Choukse E, Zhang C, Shah A, Goiri Í, Maleki S, Bianchini R (2023) Splitwise: Efficient generative LLM inference using phase splitting.
- Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T (2023) Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, volume 36.
- Sheng Y, Zheng L, Yuan B, Li Z, Ryabinin M, Chen B, Liang P, Ré C, Stoica I, Zhang C (2023) Flexgen: High-throughput generative inference of large language models with a single gpu. *International Conference on Machine Learning*, 31094–31116.
- Stidham S Jr (1985) Optimal control of admission to a queueing system. *IEEE Transactions on Automatic Control* 30(8):705–713.
- Strogatz SH (2015) *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering* (Westview Press), 2nd edition.

-
- Wu B, Zhong Y, Zhang Z, Liu S, Liu F, Sun Y, Huang G, Liu X, Jin X (2023) Fast distributed inference serving for large language models.
- Xiao G, Tian Y, Chen B, Han S, Lewis M (2023) Efficient streaming language models with attention sinks.
- Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023) ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*.
- Yu GI, Jeong JS, Kim GW, Kim S, Chun BG (2022) Orca: A distributed serving system for transformer-based generative models. *OSDI*.
- Zhang Z, Sheng Y, Zhou T, Chen T, Zheng L, Cai R, Song Z, Tian Y, Ré C, Barrett C, Wang Z, Chen B (2023) H2o: Heavy-hitter oracle for efficient generative inference of large language models.
- Zheng L, Yin L, Xie Z, Sun C, Huang J, Yu CH, Cao S, Kozyrakis C, Stoica I, Gonzalez JE, Barrett C, Sheng Y (2024) SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*.
- Zhong Y, Liu S, Chen J, Hu J, Zhu Y, Liu X, Jin X, Zhang H (2024) Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving.

Appendix A: Proof of Theorem 1

We prove Theorem 1: under baseline admission at the memory boundary, the eviction-free fixed point and all intermediate eviction-level cycles are unstable, while the level- $(l_1 - 1)$ maximal-eviction cycle is asymptotically stable.

A.1. Stability Analysis

We split the proof into a pre-eviction stage, where the state is described by physical stages, and a post-eviction stage, where the correct state description is a block-end sampled chain of relative positions.

Definition (Eviction-free spread). In the eviction-free regime, define

$$G_n := \max_j x_n^{(j)} - \min_j x_n^{(j)}.$$

Write

$$\bar{x}_n := \max_j x_n^{(j)}, \quad \underline{x}_n := \min_j x_n^{(j)}.$$

LEMMA 2 (Monotonicity before eviction). *While the trajectory remains eviction-free, $G_{n+1} \geq G_n$.*

Proof. Under eviction-free operation, the memory boundary is preserved and the stage-0 admission satisfies

$$(l_0 + 1)x_{n+1}^{(0)} = (l_0 + l_1)x_n^{(l_1-1)} - \sum_{j=0}^{l_1-2} x_n^{(j)}. \quad (17)$$

Rewriting gives

$$x_{n+1}^{(0)} = x_n^{(l_1-1)} + \frac{l_1 x_n^{(l_1-1)} - \sum_{j=0}^{l_1-1} x_n^{(j)}}{l_0 + 1}. \quad (18)$$

For $j \geq 1$, the shift relation $x_{n+1}^{(j)} = x_n^{(j-1)}$ preserves all non-final stage values. Therefore the only new coordinate is $x_{n+1}^{(0)}$.

If the maximum is attained at some stage $j \leq l_1 - 2$, then the same value reappears at stage $j + 1$, so $\bar{x}_{n+1} \geq \bar{x}_n$. If the maximum is attained at the final stage, then $x_n^{(l_1-1)} \geq \sum_j x_n^{(j)} / l_1$, and (18) gives

$$x_{n+1}^{(0)} \geq x_n^{(l_1-1)} = \bar{x}_n.$$

Hence $\bar{x}_{n+1} \geq \bar{x}_n$.

The minimum is similar. If it is attained at some stage $j \leq l_1 - 2$, it shifts unchanged to stage $j + 1$, so $\underline{x}_{n+1} \leq \underline{x}_n$. If it is attained at the final stage, then $x_n^{(l_1-1)} \leq \sum_j x_n^{(j)} / l_1$, and (18) gives

$$x_{n+1}^{(0)} \leq x_n^{(l_1-1)} = \underline{x}_n.$$

Hence $\underline{x}_{n+1} \leq \underline{x}_n$, and so $G_{n+1} \geq G_n$. $\square$

Lemma A.2 (Finite-time first support loss). If $G_0 > 0$, then the trajectory cannot remain eviction-free forever. In particular, the eviction-free fixed point is unstable.

Proof. Suppose the system stays eviction-free. When the maximum is at the final stage, $\bar{x}_n = x_n^{(l_1-1)}$, we have

$$\sum_{j=0}^{l_1-1} x_n^{(j)} \leq (l_1 - 1)\bar{x}_n + \underline{x}_n = l_1 \bar{x}_n - G_n.$$

Substituting into (18) gives

$$x_{n+1}^{(0)} \geq \bar{x}_n + \frac{G_n}{l_0 + 1}.$$

Thus each time the current maximum reaches the final stage, the next admission exceeds the current maximum by at least $G_n/(l_0 + 1) \geq G_0/(l_0 + 1)$.

The maximum reaches the final stage at least once every l_1 steps: every old coordinate shifts one stage older per iteration, and if a larger new admission appears it starts at stage 0 and reaches the final stage after at most $l_1 - 1$ further shifts. Therefore, if the trajectory stayed eviction-free and $G_0 > 0$, the spread would increase by at least $G_0/(l_0 + 1)$ at least once every l_1 steps. This would force $G_n \rightarrow \infty$.

But $G_n \leq \bar{x}_n$, and the memory constraint implies

$$\bar{x}_n(l_0 + 1) \leq \sum_{j=0}^{l_1-1} (l_0 + 1 + j)x_n^{(j)} = M.$$

So $G_n \leq M/(l_0 + 1)$, a contradiction. Hence the eviction-free regime must break down in finite time. Since the fixed point is the unique eviction-free state with $G_0 = 0$, it is unstable. $\square$

After the first support loss, physical-stage indices lose their tracking power: eviction reshuffles which stages carry mass at each iteration. The right coordinate system tracks *relative positions* in the circular pipeline, samples the state only at *block-ends* (the iterations where live positions complete), and normalizes masses by *predecessor gaps* (the number of stages each live position absorbs).

Relative positions. Fix labels $P_0, \dots, P_{l_1-1}$ on the circular pipeline. At time n , position P_r occupies stage $r + n \pmod{l_1}$. Let $z_n^{(r)}$ denote the mass carried by P_r at n , so

$$x_n^{(j)} = z_n^{((j-n) \bmod l_1)}.$$

When a cohort completes and the released memory funds a new admission at stage 0, the label is preserved. Thus, the labels track relative cohorts rather than physical stage indices.

Lemma A.3 (Irreversibility of lost positions). If $z_{n_0}^{(r)} = 0$ for some position P_r , then $z_n^{(r)} = 0$ for every $n \geq n_0$.

Proof. Zero mass shifts as zero, so P_r remains empty until it next reaches the final stage. Starting from the boundary M , every other live position increases its footprint by one token during execute, while P_r 's completion releases zero memory. The post-execution memory is therefore

$$\tilde{M}_n = M + \sum_{q \neq r} z_n^{(q)} > M.$$

LPF eviction restores the total to exactly M , leaving no slack for admission: the new stage-0 batch is zero. That batch is exactly the mass that would have been assigned to P_r after wrap-around, so $z_{n+1}^{(r)} = 0$. Repeating the argument each time P_r returns to stage 0 shows that P_r never revives. $\square$

Define the *live set* as

$$L_n := \{r : z_n^{(r)} > 0\}. \quad (19)$$

Once a relative position becomes empty, it never revives: when that dead position later cycles back to stage 0, it carries no mass and therefore frees no completion memory, while the surviving positions continue to grow their footprints. No new mass enters the dead position. Hence $L_{n+1} \subseteq L_n$, and the live set can only shrink.

Eviction blocks and block-ends. Consider a trajectory with $m = |L_n| \geq 2$ live positions. Within each period of l_1 iterations, the dynamics decompose into two phases. An *eviction block* is a maximal contiguous run of iterations during which eviction removes excess requests but no completions occur (i.e., only dead positions pass stage $l_1 - 1$). The *block-end* is the first iteration after the eviction block in which a live position completes and new requests are admitted. At block-end, the live/dead pattern returns to the same configuration it had at the previous block-end, making block-ends the natural sampling points for the dynamics (Figure 11a).

EXAMPLE 6 (BLOCK DECOMPOSITION). Consider the adjacent level-2 orbit for $l_0 = 2$, $l_1 = 4$, $M = 63$:

$$(17, 3, 0, 0) \rightarrow (0, 12, 3, 0) \rightarrow (0, 0, 9, 3) \rightarrow (3, 0, 0, 9) \rightarrow (17, 3, 0, 0).$$

The transitions $n = 0 \rightarrow 1 \rightarrow 2$ form the eviction block: at each step, execute-and-shift produces a memory overshoot corrected by evicting from the most recently admitted stage (5 requests at $n=0 \rightarrow 1$, then 3 at $n=1 \rightarrow 2$). No completions occur because only dead positions pass stage 3. The block-ends are $n = 2$ and $n = 3$, where live positions complete and new requests enter.

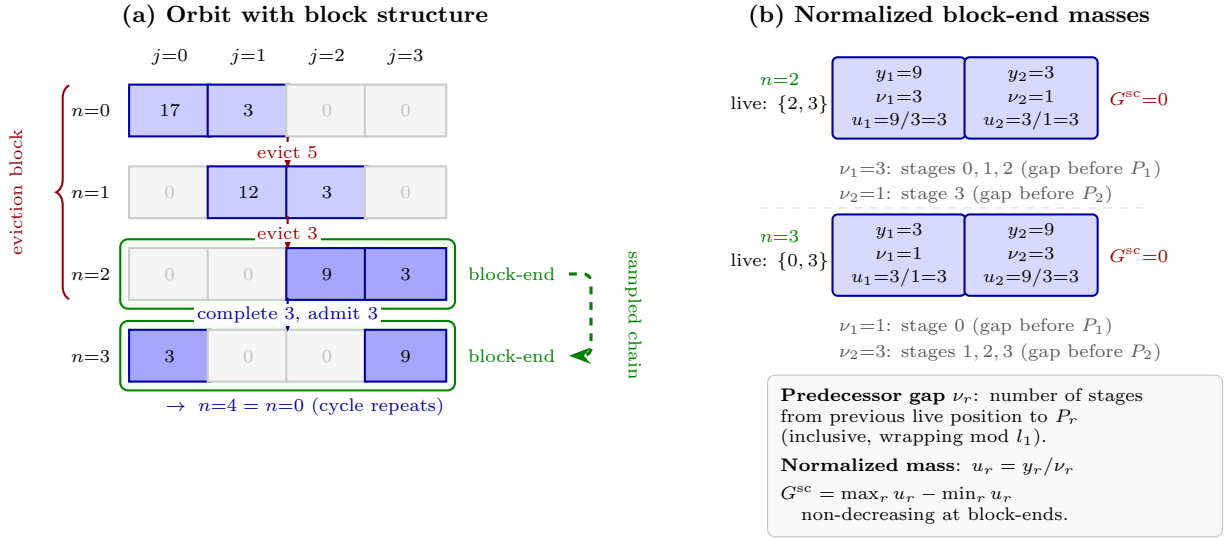


Figure 11 Block decomposition for the level-2 adjacent orbit ($l_0 = 2$, $l_1 = 4$, $M = 63$). (a) The orbit consists of an *eviction block* ($n = 0 \rightarrow 2$), during which eviction removes excess requests but no completions occur, followed by a *renewal phase* ($n = 2 \rightarrow 0$) in which completions free memory and new requests are admitted. The block-end phases (green borders) are the states immediately after the eviction block settles. (b) At each block-end, the predecessor gap ν_r counts the stages between consecutive live positions. The normalized mass $u_r = y_r / \nu_r$ adjusts for these gaps; when all normalized masses agree ($G^{\text{sc}} = 0$), the orbit is balanced.

At $n = 2$ (live stages $\{2, 3\}$): predecessor gaps $\nu = (3, 1)$, masses $(9, 3)$, normalized masses $u = (9/3, 3/1) = (3, 3)$, spread $G^{\text{sc}} = 0$. At $n = 3$ (live stages $\{0, 3\}$): gaps swap to $\nu = (1, 3)$, masses $(3, 9)$, normalized $u = (3/1, 9/3) = (3, 3)$, spread $G^{\text{sc}} = 0$. The orbit is balanced.

Predecessor gaps. Fix an interval over which the live-position set is unchanged, with $m \geq 2$ live positions listed in circular order as $r_0, \dots, r_{m-1}$. The *predecessor gap* of the h -th live position is

$$g_h := (r_h - r_{h-1}) \bmod l_1, \quad h = 0, \dots, m-1,$$

with indices modulo m . Thus g_h is the number of service stages crossed from the previous live position to the current one, with wrap-around modulo l_1 . In particular, $g_h \geq 1$ and $\sum_h g_h = l_1$.

Write $S_h := \sum_{t=0}^h g_t$ for the cumulative gap. At a block-end, the h -th live position occupies physical stage $S_h - 1$, so the live positions sit at stages

$$g_0 - 1, \quad g_0 + g_1 - 1, \quad \dots, \quad l_1 - 1.$$

The oldest live position is y_{m-1} ; it sits at the final stage $l_1 - 1$ and is about to complete. A *block-end sampled state* is described by the live-mass vector $Y = (y_0, \dots, y_{m-1})$. Here y_h is the number of

requests located at the h -th live position in the circular ordering, equivalently, the mass at physical stage $S_h - 1$. At stage $S_h - 1$, each request occupies $l_0 + S_h$ tokens. The memory boundary in these coordinates is therefore

$$M = \sum_{h=0}^{m-1} (l_0 + S_h) y_h. \quad (20)$$

Proposition A.4 (Exact block-end map). Starting from a block-end sampled state $Y = (y_0, \dots, y_{m-1})$, the next state of the embedded block-end chain, conditional on the same support pattern, is $Y' = (y'_0, \dots, y'_{m-1})$ with

$$y'_{h+1} = y_h, \quad h = 0, \dots, m-2,$$

and

$$y'_0 = \frac{(l_0 + l_1)y_{m-1} - g_{m-1} \sum_{h=0}^{m-2} y_h}{l_0 + g_{m-1}}. \quad (21)$$

Proof. The next sampling epoch occurs after exactly g_{m-1} physical iterations. At the current block-end, the oldest live position P_{m-1} sits at stage $l_1 - 1$. After one execute step it completes, releasing $(l_0 + l_1)y_{m-1}$ tokens. Meanwhile, every other live position advances one stage, increasing total memory by one token per request, while dead positions passing through stage $l_1 - 1$ release no memory. The system then admits a new batch at stage 0 and traverses $g_{m-1} - 1$ pure-evict iterations, until the next live position reaches the final stage. During these pure-evict iterations the new batch ages by one stage at a time, and LPF eviction trims exactly the memory overflow created by that aging.

The old non-final live masses shift one slot in circular order:

$$y'_{h+1} = y_h, \quad h = 0, \dots, m-2.$$

In the new sampled state, every live stage has advanced by g_{m-1} positions. The live stages are therefore

$$g_{m-1} - 1, \quad g_{m-1} + g_0 - 1, \quad \dots, \quad l_1 - 1,$$

with memory weights $l_0 + g_{m-1}, l_0 + g_{m-1} + g_0, \dots, l_0 + l_1$. The new boundary equation is

$$M = (l_0 + g_{m-1})y'_0 + \sum_{h=0}^{m-2} (l_0 + g_{m-1} + S_h)y_h.$$

Subtracting the old boundary (20) eliminates the constant terms and yields

$$(l_0 + g_{m-1})y'_0 = (l_0 + l_1)y_{m-1} - g_{m-1} \sum_{h=0}^{m-2} y_h,$$

which is (21). $\square$

Normalized masses. Define the *normalized mass* of position i by

$$u_i := y_i / g_i. \quad (22)$$

This normalization accounts for the length of the predecessor gap: a live position preceded by more dead stages absorbs more memory growth during the eviction block, so u_i measures mass per unit gap (Figure 11b).

Corollary A.5 (Normalized block-end map). In normalized coordinates, the embedded update is

$$u'_{h+1} = u_h, \quad h = 0, \dots, m-2,$$

and

$$u'_0 = u_{m-1} + \frac{\sum_{h=0}^{m-2} g_h (u_{m-1} - u_h)}{l_0 + g_{m-1}}. \quad (23)$$

Proof. After rotation, the gap vector becomes $(g_{m-1}, g_0, g_1, \dots, g_{m-2})$, so $g'_{h+1} = g_h$ for $h = 0, \dots, m-2$ and $g'_0 = g_{m-1}$. The shift relation $y'_{h+1} = y_h$ gives $u'_{h+1} = y_h / g'_{h+1} = y_h / g_h = u_h$. For the new admission, dividing (21) by $g'_0 = g_{m-1}$ and substituting $y_h = g_h u_h$ yields (23). $\square$

Next, define the *block-end spread* as

$$G^{\text{sc}} := \max_i u_i - \min_i u_i. \quad (24)$$

Lemma A.6 (Spread monotonicity on a fixed support pattern). Let

$$G^{\text{sc}}(Y) := \max_h u_h - \min_h u_h$$

be the block-end sampled spread. Then

$$G^{\text{sc}}(Y') \geq G^{\text{sc}}(Y).$$

Moreover, if $G^{\text{sc}}(Y) > 0$ and the oldest normalized coordinate u_{m-1} is an extremum, then the inequality is strict.

Proof. Let

$$\underline{u} := \min_h u_h, \quad \bar{u} := \max_h u_h.$$

The shifted coordinates $u_0, \dots, u_{m-2}$ reappear unchanged in Y' , so only the loss of the oldest value u_{m-1} can reduce the spread.

If u_{m-1} is neither the unique maximum nor the unique minimum, then both $\underline{u}$ and $\bar{u}$ remain among the shifted coordinates, so the spread cannot decrease.

If $u_{m-1} = \bar{u}$, then every term $u_{m-1} - u_h$ in (23) is nonnegative, hence $u'_0 \geq \bar{u}$. The old minimum remains among the shifted coordinates, so

$$G^{\text{sc}}(Y') \geq \bar{u} - \underline{u} = G^{\text{sc}}(Y).$$

If $G^{\text{sc}}(Y) > 0$, then some $u_h < \bar{u}$, so the numerator in (23) is strictly positive and $u'_0 > \bar{u}$; thus the spread strictly grows.

The case $u_{m-1} = \underline{u}$ is symmetric: every term in the numerator of (23) is nonpositive, so $u'_0 \leq \underline{u}$, while the old maximum remains among the shifted coordinates. Therefore the spread again cannot decrease, and it is strict when $G^{\text{sc}}(Y) > 0$. $\square$

Corollary A.7 (Strict growth within bounded time). Fix a support pattern with $m \geq 2$. If $G^{\text{sc}}(Y) > 0$ and the support pattern does not change, then G^{sc} strictly increases at least once every m sampled steps.

Proof. Take a coordinate attaining either the current maximum or the current minimum. Under the sampled dynamics, old coordinates shift one slot older at each sampled step. Hence, unless strict growth has already occurred, that extremal coordinate reaches the oldest slot $m - 1$ within at most $m - 1$ sampled steps. The following sampled step is then covered by Lemma A.6 and forces strict growth. $\square$

Lemma A.8 (Finite-time support loss on a fixed pattern). Fix a support pattern with $m \geq 2$ and suppose $G^{\text{sc}}(Y_0) > 0$ at the initial sampled state. Then some live coordinate reaches zero, and the support strictly shrinks, within finitely many sampled steps.

Proof. By Lemma A.6, G^{sc} is non-decreasing, and it can strictly increase only at a step where the departing coordinate u_{m-1} equals the current maximum or minimum. By Corollary A.7, such a strict-growth step occurs within every m sampled steps.

We show the growth is multiplicative. At any strict-growth step where $u_{m-1} = \max_h u_h =: \bar{u}$, equation (23) gives

$$u'_0 = \bar{u} + \frac{\sum_{h=0}^{m-2} g_h(\bar{u} - u_h)}{l_0 + g_{m-1}} \geq \bar{u} + \frac{G^{\text{sc}}}{l_0 + l_1},$$

since the minimum-attaining term contributes at least G^{sc} with $g_h \geq 1$, and $l_0 + g_{m-1} \leq l_0 + l_1$. The old minimum persists among slots $1, \dots, m-1$, so

$$G^{\text{sc}}(Y') \geq u'_0 - \underline{u} \geq G^{\text{sc}}(Y) \left(1 + \frac{1}{l_0 + l_1}\right).$$

The case $u_{m-1} = \min_h u_h$ is symmetric: every discrepancy term is non-positive, so $u'_0 \leq \underline{u} - G^{\text{sc}}/(l_0 + l_1)$ while the old maximum persists; the same multiplicative bound holds.

Hence after every m sampled steps, the spread grows by at least the factor $(1 + 1/(l_0 + l_1))$. Since $u_h \in [0, M/(l_0 + 1)]$ for all h , the spread is bounded: $G^{\text{sc}} \leq M/(l_0 + 1)$. Multiplicative growth of a bounded quantity cannot continue indefinitely on a fixed support. Thus some coordinate must reach zero, and the support strictly shrinks, within at most

$$m \left\lceil \frac{\ln(M/((l_0 + 1) G_0^{\text{sc}}))}{\ln(1 + 1/(l_0 + l_1))} \right\rceil$$

sampled steps. When $u_h = 0$, the mass $y_h = g_h u_h = 0$ and the position is dead. $\square$

The *normalized mass* of position i is

$$u_i := y_i / g_i, \quad (25)$$

and the *block-end spread* is

$$G^{\text{sc}} := \max_i u_i - \min_i u_i. \quad (26)$$

The normalization adjusts for the fact that a live position flanked by more dead stages absorbs proportionally more memory during the eviction block, so u_i measures mass per unit gap (Figure 11b).

Theorem A.1 (Position-tracking cascade). Every trajectory either stays exactly on an equilibrium orbit of some eviction level or eventually reaches the level- $(l_1 - 1)$ maximal-eviction cycle. In particular, every trajectory that is not captured by an intermediate equilibrium orbit cascades to maximal eviction.

Proof. By Lemma A.2, every non-equilibrium eviction-free trajectory undergoes a first support loss in finite time.

Now fix an interval on which the live-position set is constant and has size $m \geq 2$. By Lemma A.3, dead positions do not revive. Therefore the trajectory on this interval is governed by the exact block-end sampled chain above.

If $G^{\text{sc}} = 0$ at one sampled phase, then all normalized coordinates are equal. Equation (23) preserves this equality, so the entire sampled chain remains balanced. This is exactly the balanced sampled representative of the corresponding eviction-level- $(l_1 - m)$ equilibrium orbit.

Suppose instead that $G^{\text{sc}} > 0$ on the sampled chain. Since $u_h \in [0, M/(l_0 + 1)]$, the spread is bounded. By Lemma A.8, the spread grows multiplicatively by a factor $\geq (1 + 1/(l_0 + l_1))$ every m

sampled steps. A bounded quantity cannot grow multiplicatively forever, so some live coordinate reaches zero in finite time.

Each such support loss strictly decreases the number of live positions, and Lemma A.3 rules out revival. After finitely many reductions, either the trajectory lands exactly on a balanced intermediate orbit, or only one live position remains. When $m = 1$, the memory boundary forces the deterministic cycle

$$\left(\frac{M}{w_0}, 0, \dots, 0\right) \rightarrow \left(0, \frac{M}{w_1}, 0, \dots, 0\right) \rightarrow \dots \rightarrow \left(0, \dots, 0, \frac{M}{w_{l_1-1}}\right),$$

which is the maximal-eviction cycle. $\square$

Lemma A.2 already proves part (1) of Theorem 1. Theorem A.1 gives part (2): if C is an eviction-level- i orbit with $1 \leq i \leq l_1 - 2$, then its block-end sampled representative has $G^{\text{sc}} = 0$, but any sufficiently small perturbation that makes the normalized sampled masses unequal produces $G^{\text{sc}} > 0$, and Theorem A.1 then forces another support loss. So intermediate cycles are not stable.

Figure 12 illustrates the full cascade for Example 3 ($l_0 = 2, l_1 = 3, M = 24$). The three phases are visible: (i) growing oscillation in the eviction-free regime ($n = 0-6$), (ii) intermittent evictions of increasing severity as the system cascades through levels ($n = 7-16$), and (iii) convergence to the stable level-2 cycle ($n \geq 17$) with throughput $c_2 = 8/5$ per iteration.

Higher decoding lengths: $l_1 = 4$. With $l_0 = 2, l_1 = 4$, and $M = 48$ (weights $w = [3, 4, 5, 6]$, $x^* = 8/3$), the cascade exhibits *four* phases (Figure 13): (i) growing oscillation ($n = 0-8$), (ii) level-1 evictions at $n = 9, 13$, (iii) level-2 and level-3 evictions at $n = 17, 20, 21$, and (iv) convergence to the stable level-3 cycle ($n \geq 22$) with throughput $c_3 = 2$, a 25% loss relative to x^* . The additional decoding stage creates a richer transient: the system traverses two intermediate eviction levels before reaching the worst case.

Cycle multiplicity at intermediate levels. A qualitatively new phenomenon for $l_1 \geq 4$ is that intermediate eviction levels admit multiple distinct periodic orbits with different throughputs; see Figure 1 in Section 3 for an illustration. At level-2 (two stages empty per cycle), two orbit types coexist: an *opposite* (maximally separated) orbit (period 2, throughput = $12/5$, 90% of x^*) and an *adjacent* orbit (period 4, throughput = $16/7$, 86% of x^*). Both are unstable, and the baseline cascade bypasses them to the worst-case level-3 cycle ($c_3 = 2$, 75% of x^*).

Corollary (Stability of Maximum-Eviction Cycle). The level- $(l_1 - 1)$ cycle is the unique stable equilibrium under baseline admission.

Proof. At level- $(l_1 - 1)$ eviction, the system state is $X = [0, \dots, 0, x^{(l_1-1)}]$ with $x^{(l_1-1)} = M/(l_0 + l_1)$. The dynamics form a deterministic cycle:

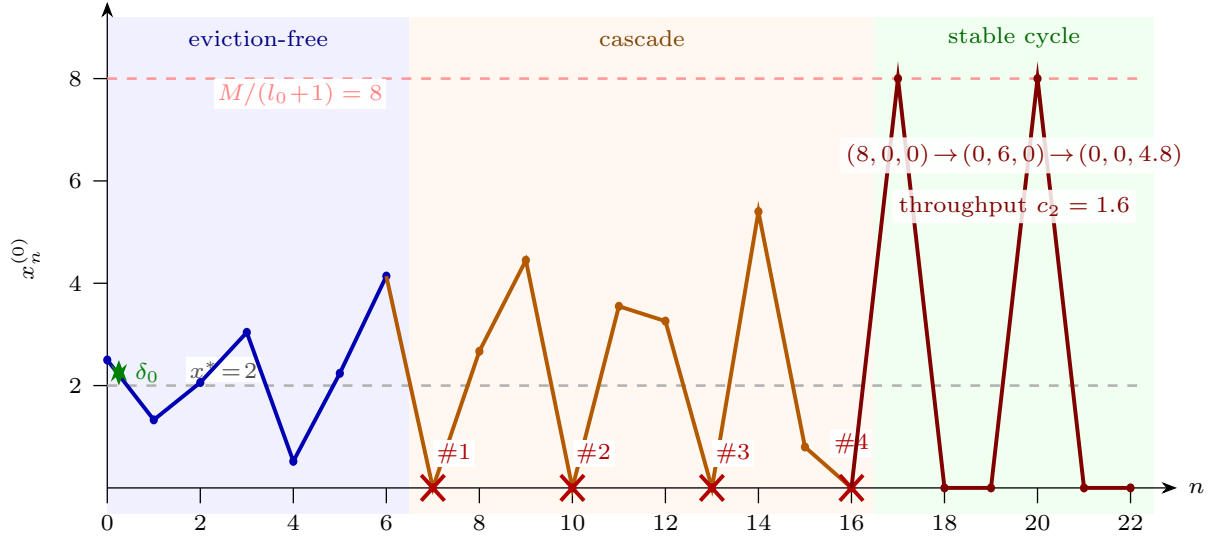


Figure 12 Full cascade trajectory for Example 3 ($l_0 = 2$, $l_1 = 3$, $M = 24$). Starting from $\delta_0 = 0.5$ above equilibrium, the system passes through three phases: (i) growing oscillation in the eviction-free regime ($n = 0$ – 6), (ii) intermittent evictions of increasing severity (#1–#4) as the system cascades through levels ($n = 7$ – 16), and (iii) convergence to the stable level-2 cycle ($n \geq 17$) with throughput $c_2 = 1.6$ per iteration.

1. All $x^{(l_1-1)}$ requests complete, releasing memory $(l_0 + l_1)x^{(l_1-1)} = M$.
2. New admission: $x^{(0)} = M/(l_0 + 1)$ (baseline system fills all available memory).
3. After $l_1 - 1$ steps: requests reach stage $l_1 - 1$ with count $x^{(l_1-1)} = M/(l_0 + l_1)$.

Stability analysis. Consider a perturbation $\tilde{x}^{(l_1-1)} = x^{(l_1-1)} + \delta$ for small $|\delta|$.

Case 1: $\delta > 0$ (excess requests at final stage). The released memory is $(l_0 + l_1)(x^{(l_1-1)} + \delta) = M + (l_0 + l_1)\delta$. The unconstrained admission would be:

$$\tilde{x}^{(0)} = \frac{M + (l_0 + l_1)\delta}{l_0 + 1} = x^{(0)} + \frac{l_0 + l_1}{l_0 + 1}\delta > x^{(0)}.$$

However, the memory constraint $\tilde{x}^{(0)}(l_0 + 1) \leq M$ limits admission to $\tilde{x}^{(0)} = M/(l_0 + 1) = x^{(0)}$. After $l_1 - 1$ steps, these $x^{(0)}$ requests reach stage $l_1 - 1$, yielding $\tilde{x}^{(l_1-1)} = x^{(l_1-1)}$. The perturbation is eliminated in exactly one cycle.

Case 2: $\delta < 0$ (deficit at final stage). The released memory is $M + (l_0 + l_1)\delta < M$. The baseline system admits:

$$\tilde{x}^{(0)} = \frac{M + (l_0 + l_1)\delta}{l_0 + 1} = x^{(0)} + \frac{l_0 + l_1}{l_0 + 1}\delta.$$

Let $\gamma = (l_0 + l_1)/(l_0 + 1) > 1$. The initial perturbation at stage 0 is $\gamma\delta < \delta < 0$.

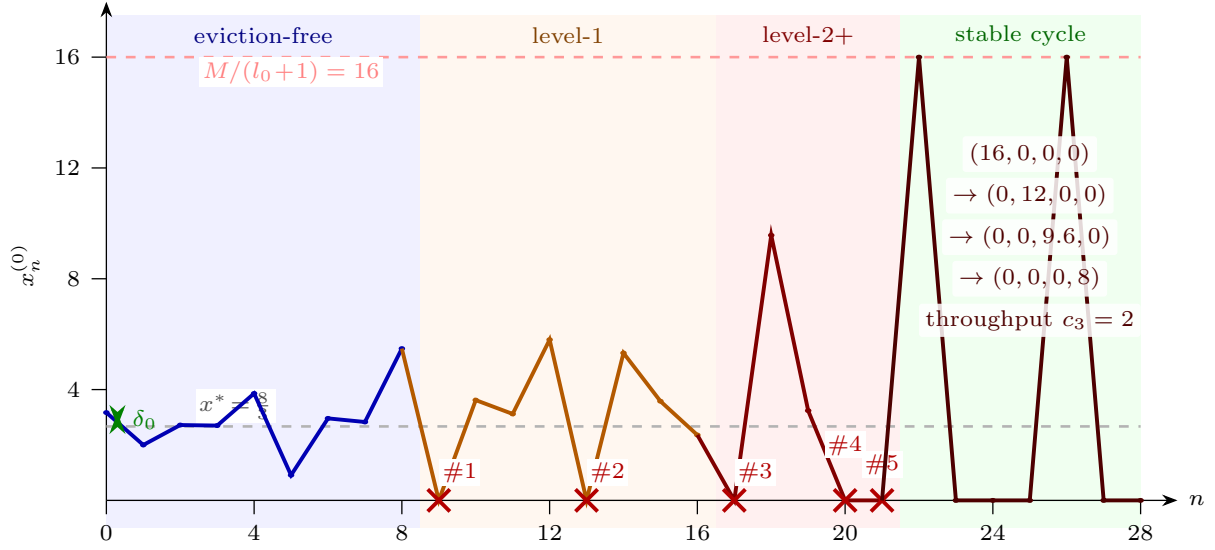


Figure 13 Full cascade trajectory for $l_0 = 2$, $l_1 = 4$, $M = 48$ (weights $w = [3, 4, 5, 6]$, $x^* = 8/3$). Starting from $\delta_0 = 0.5$ above equilibrium, the system passes through *four* phases: (i) growing oscillation in the eviction-free regime ($n = 0-8$), (ii) level-1 evictions at $n = 9, 13$ ($n = 9-16$), (iii) level-2 and level-3 evictions at $n = 17, 20, 21$ ($n = 17-21$), and (iv) convergence to the stable level-3 cycle ($n \geq 22$) with throughput $c_3 = 2$ per iteration—a 25% loss from $x^* = 8/3$.

Perturbation propagation. At level- $(l_1 - 1)$, the unperturbed cycle has the following structure. At step $k = 0, 1, \dots, l_1 - 1$, the batch occupies only stage k with count $x^{(k)} = M/(l_0 + 1 + k)$, since each request at stage k consumes $(l_0 + 1 + k)$ tokens and the total must equal M . The transition from step k to step $k + 1$ involves shifting all requests to the next stage (increasing each request's footprint by 1 token) and evicting the excess: the unperturbed eviction count at step k is

$$x^{(k)} - x^{(k+1)} = \frac{M}{l_0 + 1 + k} - \frac{M}{l_0 + 2 + k} = \frac{M}{(l_0 + 1 + k)(l_0 + 2 + k)}.$$

With perturbed initial count $\tilde{x}^{(0)} = x^{(0)} + \gamma\delta$ where $\gamma\delta < 0$, we track the perturbation $\epsilon_k := \tilde{x}^{(k)} - x^{(k)}$ through the shift steps, with $\epsilon_0 = \gamma\delta < 0$. At step k , the perturbed count is $\tilde{x}^{(k)} = x^{(k)} + \epsilon_k$. After shifting to stage $k + 1$, the memory usage becomes:

$$(l_0 + 2 + k)\tilde{x}^{(k)} = (l_0 + 2 + k)(x^{(k)} + \epsilon_k) = M \cdot \frac{l_0 + 2 + k}{l_0 + 1 + k} + (l_0 + 2 + k)\epsilon_k.$$

In the unperturbed system, the post-shift memory exceeds capacity by $M/(l_0 + 1 + k)$, requiring eviction. For the perturbed system, eviction still occurs provided the post-shift memory exceeds M , i.e.:

$$(l_0 + 2 + k)\epsilon_k > -\frac{M}{l_0 + 1 + k}, \quad \text{equivalently} \quad |\epsilon_k| < \frac{M}{(l_0 + 1 + k)(l_0 + 2 + k)} = x^{(k)} - x^{(k+1)}.$$

That is, the perturbation magnitude must be smaller than the unperturbed eviction count at step k . For sufficiently small $|\delta|$ (and hence small $|\epsilon_0| = |\gamma\delta|$), this condition holds at step 0. When eviction still occurs, the baseline system sets the post-eviction count to exactly fill capacity:

$$\tilde{x}^{(k+1)} = \frac{M}{l_0 + 2 + k} = x^{(k+1)}, \quad \text{giving} \quad \epsilon_{k+1} = 0.$$

The first eviction step *eliminates the perturbation entirely*. From step 1 onward, $\tilde{x}^{(k)} = x^{(k)}$ for all $k \geq 1$, so the perturbed trajectory rejoins the unperturbed orbit. In particular, $\tilde{x}^{(l_1-1)} = x^{(l_1-1)} = M/(l_0 + l_1)$.

Validity condition. The above argument requires $|\epsilon_0| = |\gamma\delta| < x^{(0)} - x^{(1)} = M/[(l_0 + 1)(l_0 + 2)]$. Substituting $\gamma = (l_0 + l_1)/(l_0 + 1)$, this gives $|\delta| < M/[(l_0 + l_1)(l_0 + 2)]$, which defines the basin of attraction for the local stability analysis.

Combined return map. In both cases:

- Case 1 ($\delta > 0$): The memory constraint caps admission at $x^{(0)} = M/(l_0 + 1)$, eliminating the perturbation in one cycle.
- Case 2 ($\delta < 0$): The reduced admission enters the shift phase with a deficit, but eviction at the first shift step resets the count to the unperturbed value, eliminating the perturbation in one cycle.

In both cases, a perturbation at stage $l_1 - 1$ is eliminated within a single cycle of l_1 steps. The return map is therefore a contraction: the spectral radius of the linearized map is zero (the perturbation vanishes in finite time, not merely asymptotically).

General perturbations. The analysis above perturbs only the final-stage coordinate at the completion phase. For full asymptotic stability, consider a general feasible perturbation near an arbitrary phase $X_k = (0, \dots, 0, M/w_k, 0, \dots, 0)$ of the cycle:

$$\tilde{X}_k = (\epsilon_0, \dots, \epsilon_{k-1}, M/w_k - \delta, \epsilon_{k+1}, \dots, \epsilon_{l_1-1}),$$

where $\epsilon_j \geq 0$ are small and $\delta = \sum_{j \neq k} w_j \epsilon_j / w_k$ ensures the memory boundary.

Case $k < l_1 - 1$ (shift phase). After one execute step, every request advances one stage: the main batch moves to stage $k+1$ with memory $(M/w_k - \delta)w_{k+1} \approx Mw_{k+1}/w_k > M$, so overflow occurs regardless of the small perturbation. LPF evicts from the lowest occupied stage upward: first, all perturbation mass at stages $1, \dots, k$ is removed (total memory $O(\epsilon)$). This is far short of resolving the $O(1)$ overflow, so LPF next trims the main batch at stage $k+1$ itself, reducing it to $\approx M/w_{k+1}$ (the unperturbed value for phase X_{k+1}), minus an $O(\epsilon)$ correction from surviving perturbation at

stages $k+2, \dots, l_1-1$. The same mechanism repeats at each subsequent step: the advancing main batch “steamrolls” through the pipeline, and LPF eviction at each stage first clears any perturbation mass behind the main batch, then trims the main batch itself to restore feasibility. Perturbation mass at stages ahead of the main batch simply shifts forward and completes when it reaches stage l_1-1 ; the released memory is $O(\epsilon)$ and is absorbed by the next eviction step. After at most l_1-1-k steps, the main batch reaches stage l_1-1 and all perturbation mass behind it has been evicted.

Case $k = l_1-1$ (completion phase). After one execute step, the main batch at stage l_1-1 completes and exits the system, freeing $\approx M$ units of memory. The remaining perturbation mass $(\epsilon_0, \dots, \epsilon_{l_1-2})$ shifts to stages $1, \dots, l_1-1$, consuming $O(\epsilon)$ total memory. Admission then fills stage 0 to approximately M/w_0 , yielding a state that is an $O(\epsilon)$ perturbation of phase X_0 . The shift-phase steamroller (Case $k=0 < l_1-1$) now applies, eliminating all perturbation within a further l_1-1 steps.

In both cases, a general perturbation near *any* phase of the maximal-eviction cycle is eliminated within at most l_1 steps. Hence the maximal-eviction cycle is asymptotically stable for all sufficiently small feasible perturbations, not just scalar ones.

Stability classification. The level- $(l_1 - 1)$ cycle is:

- **Asymptotically stable:** Both positive and negative perturbations are eliminated within one cycle of l_1 steps. The memory constraint provides a natural “ceiling” that prevents overshoot (Case 1), while eviction during the shift phase corrects undershoot (Case 2).
- **Unique stable attractor under baseline admission:** Starting from any state that is not captured by another equilibrium orbit, Theorem A.1 shows that the trajectory eventually reaches level- $(l_1 - 1)$ eviction. Once there, both types of perturbation are corrected, so the system converges to the unique level- $(l_1 - 1)$ stable orbit.

The level- $(l_1 - 1)$ cycle is the unique stable attractor because: (i) the eviction-free fixed point is unstable by Lemma A.2, and any intermediate cycle is unstable since any perturbation creates $G_n > 0$ and therefore triggers another support loss by Theorem A.1; and (ii) every trajectory not captured by an intermediate equilibrium orbit eventually enters the basin of attraction of the level- $(l_1 - 1)$ cycle. $\square$

A.2. Summary

The proof establishes:

1. The eviction-free spread G_n is non-decreasing (Lemma A.1).
2. For non-equilibrium initial conditions ($G_0 > 0$), the maximum $\bar{x}_n$ grows by at least $G_0/(l_0 + 1)$ each time it reaches stage $l_1 - 1$, while G_n remains bounded by $M/(l_0 + 1)$. This contradiction forces a first support loss in finite time (Lemma A.2).

3. After the first eviction, relative-position tracking becomes the right state description. Lemma A.3 shows that once a position reaches zero it never revives, and Proposition A.4 gives the exact block-end sampled map on any fixed support pattern.
4. On the block-end sampled chain, the normalized spread G^{sc} is non-decreasing (Lemma A.6) and strictly increases within bounded time whenever it is positive (Corollary A.7). Thus the only obstruction to further cascade is landing exactly on a balanced intermediate equilibrium orbit; otherwise another live position dies in finite time and the support shrinks again (Theorem A.1).
5. The level- $(l_1 - 1)$ cycle is the unique stable attractor: both positive and negative perturbations are eliminated within one cycle, and all other equilibria are unstable.

This completes the proof of Theorem 1. Q.E.D.

A.3. Equilibrium Characterization

The following propositions characterize the families of limit cycles at intermediate eviction levels. These results are stated here and proved in the remainder of this subsection.

PROPOSITION 2 (Two-stage cycles (eviction level $l_1 - 2$)). *Assume $l_1 \geq 2$ and $M \geq w_{l_1-1} = l_0 + l_1$. At eviction level $l_1 - 2$ (exactly two occupied stages at each iteration), the dynamics admit an explicit adjacent limit cycle of period l_1 whose throughput is*

$$\bar{T}_{\text{adj}} = \frac{M}{w_{l_1-1}^2 - w_0 w_{l_1-2}}. \quad (27)$$

If, in addition, l_1 is even with $l_1 = 2d$, the dynamics admit an explicit opposite (maximally separated) limit cycle of period d with throughput

$$\bar{T}_{\text{opp}} = \frac{M}{d(w_{d-1} + w_{l_1-1})}. \quad (28)$$

PROPOSITION 3 (Contiguous-block cycles (most imbalanced)). *Fix an eviction level $i \in \{0, 1, \dots, l_1 - 1\}$ and define*

$$c_i := \frac{M}{(i+1)w_i + \sum_{j=i+1}^{l_1-1} w_j}. \quad (29)$$

When $i = 0$, we have $c_0 = x^$ (the eviction-free fixed point throughput). When $i = l_1 - 1$, $c_{l_1-1} = M/[l_1 w_{l_1-1}]$ (the maximal-eviction throughput). For every $i \geq 1$, the execute-evict-admit map admits an eviction-level- i limit cycle whose time-averaged throughput equals c_i and whose occupied stages form a contiguous block at every iteration. Moreover, $\{c_i\}_{i=0}^{l_1-1}$ is strictly decreasing:*

$$c_i > c_{i+1}, \quad i = 0, 1, \dots, l_1 - 2. \quad (30)$$

PROPOSITION 4 (Evenly spaced cycles (most balanced)). *Let $k \in \{2, 3, \dots, l_1\}$ be a divisor of l_1 and write $l_1 = kd$. There exists an eviction-level- $(l_1 - k)$ limit cycle with k occupied stages per iteration whose occupied stage indices are evenly spaced by d . Its throughput equals*

$$\bar{T}_{\text{bal}}(k) = \frac{M}{d \sum_{q=1}^k w_{qd-1}}. \quad (31)$$

Moreover, $\bar{T}_{\text{bal}}(k)$ is strictly increasing in k (and hence strictly decreasing in the eviction level $l_1 - k$).

Note that, across levels, the ordering need not be monotone: a cycle at a higher eviction level can outperform one at a lower level when the lower-level cycle clusters its active stages into consecutive positions.

EXAMPLE 7 (CROSS-LEVEL NON-MONOTONICITY). Let $l_0 = 2$, $l_1 = 10$, so $w_j = 3 + j$.

1. *Contiguous-block, level 4* (six occupied stages in positions 0–5). By Proposition 3,

$$c_4 = \frac{M}{5w_4 + \sum_{j=5}^9 w_j} = \frac{M}{5 \cdot 7 + (8+9+10+11+12)} = \frac{M}{85}.$$

2. *Balanced, level 5* (five occupied stages at positions 1, 3, 5, 7, 9; $k = 5$, $d = 2$). By Proposition 4,

$$\bar{T}_{\text{bal}}(5) = \frac{M}{2(w_1 + w_3 + w_5 + w_7 + w_9)} = \frac{M}{2(4+6+8+10+12)} = \frac{M}{80}.$$

Since $c_4 = M/85 < M/80 = \bar{T}_{\text{bal}}(5)$, the level-4 cycle achieves *lower* throughput despite having one more occupied stage. Contiguous clustering concentrates requests in adjacent heavy stages ($5w_4 = 35$ dominates the denominator), while even spacing samples stages of mixed weight ($w_1 = 4$ through $w_9 = 12$), yielding a smaller memory-time total.

Proof of Proposition 2. We construct each orbit and verify that it is invariant under the execute-evict-admit map.

Adjacent orbit. Define $b := \bar{T}_{\text{adj}}$ as in (27) and set

$$a_r := \frac{M - w_{r+1}b}{w_r}, \quad r = 0, 1, \dots, l_1 - 2, \quad a_{l_1-1} := \frac{M - w_0b}{w_{l_1-1}} = (l_1 - 1)b,$$

where the last equality uses $w_{l_1-1} = w_0 + l_1 - 1$. Consider the length- l_1 orbit with states

$$X_r = (0, \dots, 0, \underbrace{a_r}_{\text{stage } r}, \underbrace{b}_{\text{stage } r+1}, 0, \dots, 0), \quad r = 0, 1, \dots, l_1 - 2, \quad X_{l_1-1} = (\underbrace{b}_{\text{stage } 0}, 0, \dots, 0, \underbrace{a_{l_1-1}}_{\text{stage } l_1-1}).$$

Each X_r saturates memory: $w_r a_r + w_{r+1} b = M$ (and $w_0 b + w_{l_1-1} a_{l_1-1} = M$).

For $r = 0, \dots, l_1 - 3$, applying *Execute* to X_r produces two nonzero stages $r + 1$ and $r + 2$ with values (a_r, b) . Because $a_r > a_{r+1}$, the post-execute memory exceeds M and LPF evicts from stage $r + 1$ until it equals a_{r+1} , yielding exactly X_{r+1} and leaving no slack for admission.

At $r = l_1 - 2$, *Execute* completes the b requests at stage $l_1 - 1$ and shifts a_{l_1-2} to stage $l_1 - 1$, leaving post-execute memory $w_{l_1-1} a_{l_1-2} < M$. No eviction occurs and admission fills stage 0 with

$$\frac{M - w_{l_1-1} a_{l_1-2}}{w_0} = b,$$

where the equality is equivalent to the definition of b in (27). This yields X_{l_1-1} .

Finally, applying *Execute* to X_{l_1-1} completes the a_{l_1-1} requests at stage $l_1 - 1$ and shifts b to stage 1; no eviction occurs and admission fills stage 0 with $(M - w_1 b)/w_0 = a_0$, returning to X_0 . Hence $\{X_0, \dots, X_{l_1-1}\}$ is a limit cycle.

Only X_{l_1-2} and X_{l_1-1} have a nonzero final stage, so the total completions over one period are $b + a_{l_1-1} = b + (l_1 - 1)b = l_1 b$, yielding $\bar{T}_{\text{adj}} = b$.

Opposite orbit (even $l_1 = 2d$). Let $b := M/(w_{d-1} + w_{l_1-1})$ and define

$$a_r := \frac{M - w_{r+d} b}{w_r}, \quad r = 0, 1, \dots, d - 1.$$

Consider the d -periodic orbit with states

$$Y_r = (0, \dots, 0, \underbrace{a_r}_{\text{stage } r}, 0, \dots, 0, \underbrace{b}_{\text{stage } r+d}, 0, \dots, 0), \quad r = 0, 1, \dots, d - 1.$$

Each Y_r lies on the memory boundary since $w_r a_r + w_{r+d} b = M$.

For $r = 0, \dots, d - 2$, *Execute* shifts (a_r, b) to stages $(r + 1, r + d + 1)$; since $a_r > a_{r+1}$, memory overflows and LPF evicts from stage $r + 1$ until it equals a_{r+1} , producing Y_{r+1} with no slack for admission.

At $r = d - 1$, the batch b is at the final stage $l_1 - 1 = 2d - 1$ and completes under *Execute*; the remaining batch $a_{d-1} = b$ shifts to stage d . Post-execute memory is $w_d b < M$, so no eviction occurs and admission fills stage 0 with $a_0 = (M - w_d b)/w_0$, producing Y_0 . Thus $\{Y_0, \dots, Y_{d-1}\}$ is a limit cycle. Exactly b requests complete over d iterations, giving $\bar{T}_{\text{opp}} = b/d$, which is (28). $\square$

Proof of Proposition 3. Fix i and write $k := l_1 - i$ for the number of occupied stages. Let $b := c_i$ as in (29). Define a length- l_1 orbit $\{X_0, \dots, X_{l_1-1}\}$ as follows. For $r = 0, 1, \dots, i - 1$, set

$$X_r = (0, \dots, 0, \underbrace{a_r}_{\text{stage } r}, \underbrace{b}_{\text{stages } r+1, \dots, r+k-1}, 0, \dots, 0),$$

where a_r is chosen so that X_r lies on the memory boundary:

$$w_r a_r + b \sum_{j=r+1}^{r+k-1} w_j = M. \quad (32)$$

For $r = i, i+1, \dots, l_1 - 1$, define X_r as the cyclic shift of a length- k contiguous block with one larger leading entry:

$$X_r = (\underbrace{b}_{\text{stages } 0, \dots, r-i-1}, 0, \dots, 0, \underbrace{(i+1)b}_{\text{stage } r}, \underbrace{b}_{\text{stages } r+1, \dots, l_1-1}).$$

(Equivalently, X_r has support $\{r, r+1, \dots, l_1-1, 0, 1, \dots, r-i-1\}$, a contiguous block of length k in cyclic order.) Each X_r saturates memory by construction: for $r < i$ this is (32), and for $r \geq i$ it follows from (29).

We now verify invariance under the execute-evict-admit map.

Case 1: $r < i$. The block $\{r, \dots, r+k-1\}$ does not include the final stage $l_1 - 1$, so no completion occurs. After *Execute*, a_r shifts to stage $r+1$ and the b -mass shifts to stages $r+2, \dots, r+k$, giving post-execute memory

$$w_{r+1} a_r + b \sum_{j=r+2}^{r+k} w_j = \left(w_r a_r + b \sum_{j=r+1}^{r+k-1} w_j \right) + a_r + (k-1)b > M.$$

Since $r+k \leq l_1 - 1$, the post-execute support is contained in stages $\{r+1, \dots, r+k\}$ and LPF evicts only from stage $r+1$ until feasibility is restored. The resulting stage- $r+1$ value is exactly a_{r+1} (defined by (32) with $r+1$), producing X_{r+1} with no slack for admission.

Case 2: $i \leq r \leq l_1 - 2$. Stage $l_1 - 1$ is occupied by b and completes under *Execute*. The total number of active requests in X_r equals $(i+1)b + (k-1)b = l_1 b$, so $(l_1 - 1)b$ survive. Since $w_{l_1-1} = w_0 + l_1 - 1$, the post-execute memory is

$$M + (l_1 - 1)b - w_{l_1-1}b = M - w_0 b < M.$$

No eviction occurs and admission fills the slack $w_0 b$ with exactly b new stage-0 requests, yielding X_{r+1} .

Case 3: $r = l_1 - 1$. Stage $l_1 - 1$ holds $(i+1)b$ and completes under *Execute*. The remaining stages $1, \dots, k-1$ each contain b , so the post-execute state has memory $b \sum_{j=1}^{k-1} w_j < M$ and admission fills stage 0 with

$$a_0 = \frac{M - b \sum_{j=1}^{k-1} w_j}{w_0},$$

which equals the value defined by (32) at $r = 0$. This returns to X_0 and closes the orbit.

Over one period, the final stage holds b for $l_1 - i - 1$ iterations ($r = i, \dots, l_1 - 2$) and $(i + 1)b$ for one ($r = l_1 - 1$). The total completions therefore equal

$$(l_1 - i - 1)b + (i + 1)b = l_1 b,$$

so $\bar{T} = b = c_i$.

Finally, to prove (30), the denominator in (29) satisfies

$$\left((i + 2)w_{i+1} + \sum_{j=i+2}^{l_1-1} w_j \right) - \left((i + 1)w_i + \sum_{j=i+1}^{l_1-1} w_j \right) = (i + 1)(w_{i+1} - w_i) = i + 1 > 0,$$

so $c_{i+1} < c_i$. □

Proof of Proposition 4. Let

$$b := \frac{M}{\sum_{q=1}^k w_{qd-1}}.$$

For $r = 0, 1, \dots, d - 1$, define

$$a_r := \frac{M - b \sum_{q=1}^{k-1} w_{r+qd}}{w_r}, \quad Y_r = (0, \dots, 0, \underbrace{a_r}_{\text{stage } r}, 0, \dots, 0, \underbrace{b}_{\text{stages } r+d, r+2d, \dots, r+(k-1)d}, 0, \dots, 0),$$

where indices are in $\{0, \dots, l_1 - 1\}$ and $r + (k - 1)d \leq l_1 - 1$ since $r \leq d - 1$. Each Y_r lies on the memory boundary by construction.

The sequence $\{a_r\}_{r=0}^{d-1}$ is strictly decreasing. Since $w_{r+qd} = w_r + qd$,

$$a_r = \frac{M - b \sum_{q=1}^{k-1} (w_r + qd)}{w_r} = \frac{M - bd \frac{k(k-1)}{2}}{w_r} - (k-1)b,$$

which decreases strictly with $w_r = w_0 + r$.

For $r = 0, \dots, d - 2$, *Execute* shifts $(a_r, b, \dots, b)$ to stages $(r + 1, r + 1 + d, \dots, r + 1 + (k - 1)d)$. Because $a_r > a_{r+1}$, memory overflows and LPF evicts from the lowest occupied stage $r + 1$ until it equals a_{r+1} , yielding exactly Y_{r+1} with no slack for admission.

At $r = d - 1$, stage $l_1 - 1 = (d - 1) + (k - 1)d$ holds b and completes. Since $b = M / \sum_{q=1}^k w_{qd-1}$, the boundary condition implies $a_{d-1} = b$. The remaining $k - 1$ batches all have size b and shift to stages $d, 2d, \dots, (k - 1)d$, producing post-execute memory $b \sum_{q=1}^{k-1} w_{qd} < M$. No eviction occurs and admission fills stage 0 with

$$a_0 = \frac{M - b \sum_{q=1}^{k-1} w_{qd}}{w_0},$$

returning to Y_0 . Thus $\{Y_0, \dots, Y_{d-1}\}$ is a limit cycle with throughput b/d , which is (31).

To prove monotonicity in k , substitute $w_{qd-1} = w_0 + qd - 1$ and $d = l_1/k$ into (31):

$$d \sum_{q=1}^k w_{qd-1} = d \left(k(w_0 - 1) + d \frac{k(k+1)}{2} \right) = l_1(w_0 - 1) + \frac{l_1^2}{2} \frac{k+1}{k}.$$

The right-hand side is strictly decreasing in k because $(k+1)/k$ decreases with k , so $\bar{T}_{\text{bal}}(k)$ is strictly increasing in k . $\square$

A.4. Worst-Case Limit Cycle

The proof of Lemma 1 relies on a *waste decomposition*. Let

$$W = \sum_{j=0}^{l_1-1} w_j = l_1(2l_0 + l_1 + 1)/2$$

denote the total memory-time cost required to process a request through all l_1 decoding stages. Over a limit cycle of period p , the memory constraint implies $\sum_j w_j x_n^{(j)} = M$ for each iteration n . Summing this identity over the p iterations of the cycle and decomposing the total memory-time usage by request outcome yields

$$pM = p\bar{T}(C)W + E(C), \quad (33)$$

where $E(C) \geq 0$ denotes the total memory-time consumed by requests that are eventually evicted. Rearranging gives

$$\bar{T}(C) = x^* - \frac{E(C)}{pW},$$

where $x^* = M/W$ is the throughput of the eviction-free fixed point. Thus, any limit cycle with eviction satisfies $\bar{T}(C) < x^*$. The throughput loss relative to the fixed point is proportional to the waste $E(C)$. The maximal-eviction cycle maximizes this waste, since requests move through the decoding pipeline as a single synchronized batch and eviction trims the batch at every stage. This regime yields throughput $\bar{T}_{l_1-1} = M/[l_1(l_0 + l_1)]$.

Proof of Lemma 1. Let $C = \{X_1, \dots, X_p\}$ be a limit cycle of period p on the memory boundary ($M_n = M$ at every iteration). Write $X_n = (x_n^{(0)}, \dots, x_n^{(l_1-1)})$ and let $y_n := x_n^{(l_1-1)}$ denote the number of completions at iteration n , so that $\bar{T}(C) = \frac{1}{p} \sum_{n=1}^p y_n$. By the waste decomposition (33), $\bar{T}(C) = x^* - E(C)/(pW) \leq x^*$, with equality only when no eviction occurs ($E = 0$).

We now lower bound $\bar{T}(C)$ by a direct counting argument. Define

$$m := \frac{M}{w_{l_1-1}} = \frac{M}{l_0 + l_1}.$$

Fix any starting index $t \in \{1, \dots, p\}$ and consider the l_1 consecutive iterations $t, t+1, \dots, t+l_1-1$ (with indices taken modulo p). We claim that at least m requests complete over this block:

$$\sum_{r=0}^{l_1-1} y_{t+r} \geq m. \quad (34)$$

To see this, observe that at time t the total number of active requests is at least m because

$$M = \sum_{j=0}^{l_1-1} w_j x_t^{(j)} \leq w_{l_1-1} \sum_{j=0}^{l_1-1} x_t^{(j)}.$$

Consider (in the continuous relaxation) any subpopulation of exactly m active requests at time t with the *largest* stage indices.¹ Call these the *protected* requests. At any future iteration before they complete, each protected request occupies at most w_{l_1-1} tokens, so the protected subpopulation uses at most $m w_{l_1-1} = M$ tokens at all times (including immediately after *Execute*). Evicting *only* non-protected requests therefore suffices to restore feasibility after any *Execute* step: if we hypothetically removed every non-protected request, the remaining protected requests would still fit in memory.

Under LPF, evictions occur in increasing order of stage index, so protected requests can only be evicted after all less-progressed (non-protected) requests have been removed. At the cutoff stage that may contain both protected and non-protected mass, the amount to remove never exceeds the non-protected mass, because the protected mass alone is feasible; since within-stage requests are indistinguishable in the fluid model, we couple tie-breaking so that non-protected mass is evicted first. Consequently, the protected subpopulation is never evicted and advances one stage per iteration until completion. Because each request traverses l_1 stages, all protected requests complete within the next l_1 iterations, proving (34).

Summing (34) over $t = 1, \dots, p$ and noting that each completion y_n appears in exactly l_1 of the resulting window sums gives

$$l_1 \sum_{n=1}^p y_n \geq pm.$$

Dividing by pl_1 yields

$$\bar{T}(C) = \frac{1}{p} \sum_{n=1}^p y_n \geq \frac{m}{l_1} = \frac{M}{l_1(l_0 + l_1)} = \bar{T}_{l_1-1}.$$

¹ If the cutoff falls within a stage, take an arbitrary m -mass subpopulation from that stage; requests within a stage are indistinguishable in the continuous model.

We next characterize when equality holds. Define the total number of active requests at time n by

$$N_n := \sum_{j=0}^{l_1-1} x_n^{(j)}.$$

Since $w_j \leq w_{l_1-1}$ for all j , the memory constraint implies

$$N_n \geq \frac{M}{w_{l_1-1}} = m, \quad (35)$$

with equality if and only if $x_n^{(0)} = \dots = x_n^{(l_1-2)} = 0$ (i.e., all active requests are at stage $l_1 - 1$).

We claim that if there exists some index t with $y_t > 0$ and $N_t > m$, the corresponding length- l_1 completion window is *strictly* larger than m :

$$\sum_{r=0}^{l_1-1} y_{t+r} > m. \quad (36)$$

Indeed, set

$$\delta := \min\{y_t, N_t - m\},$$

which is strictly positive under $y_t > 0$ and $N_t > m$. Consider the subpopulation of size $m + \delta$ consisting of the active requests at time t with the largest stage indices (breaking within-stage ties arbitrarily). This enlarged protected subpopulation includes all stage- $(l_1 - 1)$ requests, so exactly y_t of them complete at iteration t . Immediately after the *Execute* step at iteration t , the number of remaining enlarged protected requests equals

$$(m + \delta) - y_t \leq m,$$

where we used $\delta \leq y_t$. Since every request occupies at most w_{l_1-1} tokens, these remaining protected requests fit in memory: their total footprint is at most $m w_{l_1-1} = M$ (and thereafter it only decreases as further completions occur). Consequently, evicting only non-protected requests suffices to restore feasibility after any subsequent *Execute* step. Under LPF, non-protected requests (which are weakly less progressed at time t and remain so under stage advancement) are evicted before protected ones, and within-stage tie-breaking can again be coupled so that non-protected mass is removed first whenever the eviction front reaches the cutoff stage. Hence none of the $m + \delta$ enlarged protected requests are ever evicted, and all complete within the next l_1 iterations. This proves (36).

If C is not the maximal-eviction cycle, it never visits a state with $N_t = m$. Indeed, if $N_t = m$ for some t , then by the equality case of (35) we must have $X_t = (0, \dots, 0, m)$. Starting from this state, the execute-evict-admit map deterministically generates the maximal-eviction cycle, so C

itself would be maximal, a contradiction. Hence $N_t > m$ for all t on any non-maximal limit cycle. By (34), there exists t with $y_t > 0$; applying (36) at such t gives one strict window sum. Summing window sums over $t = 1, \dots, p$ then yields $l_1 \sum_{n=1}^p y_n > pm$, i.e., $\bar{T}(C) > m/l_1$.

Finally, the maximal-eviction cycle achieves $\bar{T} = m/l_1$, so equality holds if and only if C is the maximal-eviction cycle. $\square$

Appendix B: Proof of Theorem 2: GCD Stability Condition

We prove the complete stability characterization: (i) if $\gcd(l_A, l_B) > 1$, the eviction-free equilibrium is unstable; (ii) if $\gcd(l_A, l_B) = 1$, the system converges globally to equilibrium from any initial condition.

B.1. System Dynamics and Characteristic Polynomial

Consider two request types A and B with common prompt length l_0 but different decoding lengths $l_A < l_B$. Let $p = \lambda_A/(\lambda_A + \lambda_B)$ and $q = 1 - p$.

The token balance at time n yields the recurrence:

$$(l_0 + 1)X_{n+1}^{(0)} = (l_0 + l_B)q \cdot X_{n-l_B+1}^{(0)} + [(l_0 + l_A)p - q]X_{n-l_A+1}^{(0)} - \sum_{i=n-l_A+2}^n X_i^{(0)} - q \sum_{i=n-l_B+2}^{n-l_A} X_i^{(0)}.$$

Subtracting consecutive equations and defining $D_n = X_{n+1}^{(0)} - X_n^{(0)}$:

$$(l_0 + 1)D_n + \sum_{m=1}^{l_A-1} (l_0 + m + 1)D_{n-m} + q \sum_{m=l_A}^{l_B-1} (l_0 + m + 1)D_{n-m} = 0.$$

The characteristic polynomial is:

$$F(\lambda) = (l_0 + 1)\lambda^{l_B-1} + \sum_{m=1}^{l_A-1} (l_0 + m + 1)\lambda^{l_B-1-m} + q \sum_{m=l_A}^{l_B-1} (l_0 + m + 1)\lambda^{l_B-1-m}. \quad (37)$$

B.2. Limiting Polynomial and Key Decomposition

As $l_0 \rightarrow \infty$, define the limiting polynomial:

$$A(\lambda) = \lim_{l_0 \rightarrow \infty} \frac{F(\lambda)}{l_0 + 1} = \lambda^{l_B-1} + \sum_{m=1}^{l_A-1} \lambda^{l_B-1-m} + q \sum_{m=l_A}^{l_B-1} \lambda^{l_B-1-m}.$$

Summing geometric series yields the closed form:

$$(1 - \lambda)A(\lambda) = -\lambda^{l_B} + p\lambda^{l_B-l_A} + q. \quad (38)$$

The characteristic polynomial admits a decomposition that separates leading-order root structure from the $O(1/l_0)$ correction. Define

$$B(\lambda) = F(\lambda) - (l_0 + 1)A(\lambda) = \sum_{m=1}^{l_A-1} m\lambda^{l_B-1-m} + q \sum_{m=l_A}^{l_B-1} m\lambda^{l_B-1-m}.$$

We claim that $B(\lambda) = (l_B - 1)A(\lambda) - \lambda A'(\lambda)$. To verify, compute:

$$\begin{aligned} \lambda A'(\lambda) &= (l_B - 1)\lambda^{l_B-1} + \sum_{m=1}^{l_A-1} (l_B - 1 - m)\lambda^{l_B-1-m} + q \sum_{m=l_A}^{l_B-1} (l_B - 1 - m)\lambda^{l_B-1-m} \\ &= (l_B - 1)A(\lambda) - B(\lambda). \end{aligned}$$

Therefore:

$$F(\lambda) = (l_0 + 1)A(\lambda) + B(\lambda) = (l_0 + l_B)A(\lambda) - \lambda A'(\lambda). \quad (39)$$

This decomposition reduces the stability problem to analyzing the roots of $A(\lambda)$ and their perturbation under the $O(1/l_0)$ correction $\lambda A'(\lambda)$.

B.3. Root Structure of the Limiting Polynomial

We establish four lemmas characterizing the roots of $A(\lambda)$.

LEMMA 3 (Boundedness). *All roots of $(1 - \lambda)A(\lambda) = 0$ satisfy $|\lambda| \leq 1$.*

Proof. From (38), any root $\lambda \neq 1$ satisfies $\lambda^{l_B} = p\lambda^{l_B-l_A} + q$. Taking absolute values and applying the triangle inequality:

$$|\lambda|^{l_B} = |p\lambda^{l_B-l_A} + q| \leq p|\lambda|^{l_B-l_A} + q.$$

Define $g(x) = x^{l_B} - px^{l_B-l_A} - q$ for $x \geq 0$. First, $g(1) = 1 - p - q = 0$. Second, for $x > 1$:

$$g'(x) = l_B x^{l_B-1} - p(l_B - l_A)x^{l_B-l_A-1} = x^{l_B-l_A-1} [l_B x^{l_A} - p(l_B - l_A)].$$

Since $x > 1$ implies $x^{l_A} > 1$, the bracket exceeds $l_B - p(l_B - l_A) = ql_B + pl_A > 0$, so $g'(x) > 0$ for $x > 1$. By the mean value theorem, $g(x) > g(1) = 0$ for all $x > 1$.

Now suppose $|\lambda| > 1$. Then $g(|\lambda|) > 0$, i.e., $|\lambda|^{l_B} > p|\lambda|^{l_B-l_A} + q$. This contradicts the bound $|\lambda|^{l_B} \leq p|\lambda|^{l_B-l_A} + q$. Hence $|\lambda| \leq 1$. $\square$

LEMMA 4 (Unit Circle Structure). *If $|\lambda| = 1$ and $(1 - \lambda)A(\lambda) = 0$, then $\lambda^g = 1$ where $g = \gcd(l_A, l_B)$.*

Proof. For $|\lambda| = 1$, we have $|\lambda|^{l_B} = 1$ and $|p\lambda^{l_B-l_A} + q| \leq p + q = 1$ (by triangle inequality). Since $(1 - \lambda)A(\lambda) = 0$ requires equality in (38), we need $|\lambda|^{l_B} = |p\lambda^{l_B-l_A} + q|$, i.e., $1 = |p\lambda^{l_B-l_A} + q|$.

Since $|p\lambda^{l_B-l_A}| = p > 0$ and $|q| = q > 0$, triangle-inequality equality requires $\lambda^{l_B-l_A}$ to be a positive real of modulus one, hence $\lambda^{l_B-l_A} = 1$. Then $\lambda^{l_B} = p + q = 1$, so

$$\begin{aligned}\lambda^{l_B} &= 1, \\ \lambda^{l_B-l_A} &= 1.\end{aligned}$$

By the definition of GCD, $g = \gcd(l_A, l_B) = \gcd(l_B, l_B - l_A)$. Since $\lambda^{l_B} = 1$ and $\lambda^{l_B-l_A} = 1$, we have $\lambda^{\gcd(l_B, l_B-l_A)} = 1$, i.e., $\lambda^g = 1$ (by Bézout's identity: there exist integers a, b such that $g = a \cdot l_B + b \cdot (l_B - l_A)$). $\square$

LEMMA 5 (Exclusion of Unity). $\lambda = 1$ is not a root of $A(\lambda)$.

Proof. Direct substitution into the definition of $A(\lambda)$:

$$\begin{aligned}A(1) &= 1^{l_B-1} + \sum_{m=1}^{l_A-1} 1^{l_B-1-m} + q \sum_{m=l_A}^{l_B-1} 1^{l_B-1-m} \\ &= 1 + (l_A - 1) \cdot 1 + q \cdot (l_B - l_A) \cdot 1 \\ &= l_A + q(l_B - l_A) \\ &= l_A + (1 - p)(l_B - l_A) \quad (\text{since } q = 1 - p) \\ &= l_A + l_B - l_A - p(l_B - l_A) \\ &= l_B - pl_B + pl_A \\ &= (1 - p)l_B + pl_A \\ &= ql_B + pl_A > 0.\end{aligned}$$

Since $p, q > 0$ and $l_A, l_B \geq 1$, we have $A(1) > 0$, so $\lambda = 1$ is not a root of $A(\lambda)$. $\square$

LEMMA 6 (Non-Trivial g -th Roots). If $g = \gcd(l_A, l_B) > 1$, every non-trivial g -th root of unity $\omega = e^{2\pi i k/g}$ for $k = 1, \dots, g - 1$ is a root of $A(\lambda)$.

Proof. Write $l_A = ag$ and $l_B = bg$ with $\gcd(a, b) = 1$. For any ω with $\omega^g = 1$ and $\omega \neq 1$: $\omega^{l_A} = (\omega^g)^a = 1$, $\omega^{l_B} = (\omega^g)^b = 1$, and $\omega^{l_B-l_A} = (\omega^g)^{b-a} = 1$. Substituting into (38): $(1 - \omega)A(\omega) = -1 + p + q = 0$. Since $\omega \neq 1$, we have $1 - \omega \neq 0$, and therefore $A(\omega) = 0$. $\square$

Combining these lemmas: when $g = 1$, Lemmas 4 and 5 imply that all roots of $A(\lambda)$ lie strictly inside the unit circle; when $g > 1$, Lemma 6 shows there are exactly $g - 1$ roots on the unit circle,

located at the non-trivial g -th roots of unity. Figure 14 in Section 4 visualizes this structure: panel (a) shows the coprime case with all roots inside the unit circle, while panel (b) shows the non-coprime case with a root at $\lambda = -1$ drifting outside.

EXAMPLE 8 (ROOT STRUCTURE VERIFICATION). For the coprime case ($l_A = 2, l_B = 3, p = q = 1/2$), the limiting polynomial is

$$A(\lambda) = \lambda^2 + \lambda + \frac{1}{2}, \quad \text{roots } \lambda = \frac{-1 \pm i}{2}, \quad |\lambda| = \frac{1}{\sqrt{2}} \simeq 0.707.$$

All roots lie strictly inside the unit circle (Lemma 3), $A(1) = 5/2 > 0$ (Lemma 5), and since $\gcd(2, 3) = 1$ no root reaches the boundary (Lemma 4).

For the non-coprime case ($l_A = 2, l_B = 4, \gcd = 2$):

$$A(\lambda) = \lambda^3 + \lambda^2 + \frac{1}{2}\lambda + \frac{1}{2} = (\lambda + 1)\left(\lambda^2 + \frac{1}{2}\right).$$

The root $\lambda = -1$ sits on the unit circle, as Lemma 6 predicts: $\omega = e^{i\pi} = -1$ is the unique non-trivial g -th root of unity for $g = 2$. The remaining roots $\lambda = \pm i/\sqrt{2}$ satisfy $|\lambda| = 1/\sqrt{2} < 1$. Thus only the GCD-predicted root reaches the boundary, and it is exactly the root that the IFT will push outside the unit circle for finite l_0 (Theorem 4).

B.4. Instability for Non-Coprime Case ($g > 1$)

THEOREM 4 (Instability). Let $\gcd(l_A, l_B) = g > 1$. For sufficiently large l_0 , the characteristic equation $F(\lambda) = 0$ has $g - 1$ roots with $|\lambda| > 1$.

Proof. Using decomposition (39), define:

$$G(\lambda, \epsilon) = A(\lambda) - \epsilon \lambda A'(\lambda), \quad \epsilon = \frac{1}{l_0 + l_B}.$$

For $\epsilon > 0$: $G(\lambda, \epsilon) = 0 \iff F(\lambda) = 0$ (after dividing (39) by $l_0 + l_B$).

At each non-trivial g -th root ω , the Implicit Function Theorem applies: $G(\omega, 0) = A(\omega) = 0$ by Lemma 6, and $\frac{\partial G}{\partial \lambda}(\omega, 0) = A'(\omega) \neq 0$, as we now verify.

To compute $A'(\omega)$, differentiate (38) with respect to λ :

$$-A(\lambda) + (1 - \lambda)A'(\lambda) = -l_B \lambda^{l_B-1} + p(l_B - l_A) \lambda^{l_B-l_A-1}.$$

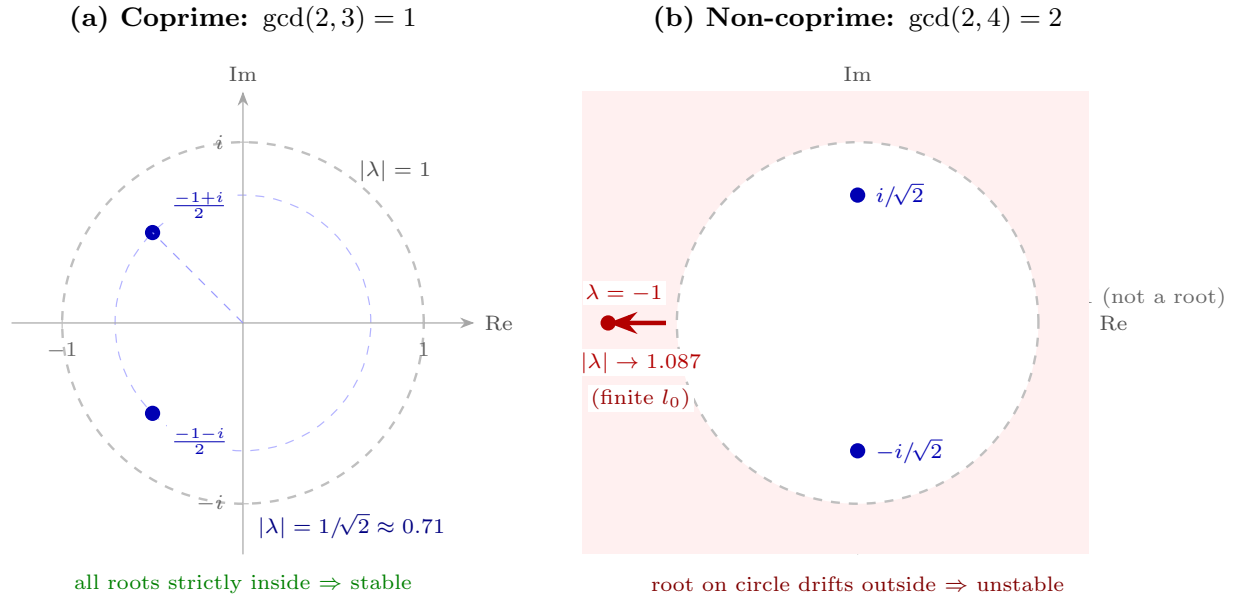


Figure 14 Root structure of the limiting polynomial $A(\lambda)$ for the two-class examples ($p = q = 1/2$). (a) Coprime decode lengths ($l_1^{(1)} = 2, l_1^{(2)} = 3$): the polynomial $A(\lambda) = \lambda^2 + \lambda + \frac{1}{2}$ has roots at $(-1 \pm i)/2$ with $|\lambda| = 1/\sqrt{2} \approx 0.71$, strictly inside the unit circle (dashed blue). The eviction-free equilibrium is asymptotically stable. (b) Non-coprime decode lengths ($l_1^{(1)} = 2, l_1^{(2)} = 4, g = 2$): the polynomial factors as $A(\lambda) = (\lambda + 1)(\lambda^2 + \frac{1}{2})$, placing a root at $\lambda = -1$ on the unit circle (red dot). For finite $l_0 = 10$, the Implicit Function Theorem shows this root drifts to $|\lambda| \approx 1.087$ (arrow), establishing instability. The two interior roots (blue dots) at $\pm i/\sqrt{2}$ remain harmless. The root $\lambda = 1$ is never a root of $A(\lambda)$ (Lemma iii).

At a non-trivial g -th root ω where $A(\omega) = 0$ and $\omega^{l_B} = \omega^{l_B - l_A} = 1$ (by Lemma 6):

$$\begin{aligned}
 (1 - \omega)A'(\omega) &= -l_B \omega^{l_B - 1} + p(l_B - l_A) \omega^{l_B - l_A - 1} \\
 &= -l_B \omega^{-1} + p(l_B - l_A) \omega^{-1} \quad (\text{since } \omega^{l_B} = \omega^{l_B - l_A} = 1) \\
 &= \omega^{-1} [-l_B + p(l_B - l_A)] \\
 &= \omega^{-1} [-l_B + p l_B - p l_A] \\
 &= -\omega^{-1} [(1 - p)l_B + p l_A] \\
 &= -\omega^{-1} [q l_B + p l_A].
 \end{aligned}$$

Since $p, q > 0$ and $l_A, l_B \geq 1$, we have $q l_B + p l_A > 0$. Also $\omega \neq 1$ (non-trivial root), so $1 - \omega \neq 0$. Therefore:

$$A'(\omega) = \frac{-\omega^{-1}(p l_A + q l_B)}{1 - \omega} \neq 0.$$

By IFT, there exists a smooth function $\lambda(\epsilon)$ with $\lambda(0) = \omega$ and $G(\lambda(\epsilon), \epsilon) = 0$. Differentiating $G(\lambda(\epsilon), \epsilon) = 0$ with respect to ϵ at $\epsilon = 0$:

$$\lambda'(0) = -\frac{\partial G / \partial \epsilon}{\partial G / \partial \lambda} \Big|_{(\omega, 0)} = -\frac{-\omega A'(\omega)}{A'(\omega)} = \omega.$$

Taylor expansion gives $\lambda(\epsilon) = \omega(1 + \epsilon) + r(\epsilon)$ where $r(\epsilon) = O(\epsilon^2)$. Let $z(\epsilon) = \omega^{-1}r(\epsilon)$; then $z(\epsilon) = O(\epsilon^2)$ and:

$$|\lambda(\epsilon)| = |1 + \epsilon + z(\epsilon)|.$$

For sufficiently small $\epsilon > 0$, there exists a constant $C > 0$ such that $|z(\epsilon)| \leq C\epsilon^2$. Therefore:

$$|\lambda(\epsilon)| \geq 1 + \epsilon - C\epsilon^2 > 1.$$

Applying this argument to each of the $g - 1$ distinct non-trivial g -th roots of unity yields $g - 1$ distinct roots of $F(\lambda) = 0$ with $|\lambda| > 1$ for all sufficiently large l_0 . $\square$

B.5. Local Stability for Coprime Case ($g = 1$)

We first establish a lemma on root simplicity, then use it for the stability proof.

LEMMA 7 (Simple Roots). *For all but at most finitely many values of $p \in (0, 1)$, all roots of $A(\lambda)$ are simple.*

Proof. Let $P(\lambda) = (1 - \lambda)A(\lambda) = -\lambda^{l_B} + p\lambda^{l_B - l_A} + q$. Since $A(1) = ql_B + pl_A > 0$ (Lemma 5), the root $\lambda = 1$ of $P(\lambda)$ is simple. Hence $A(\lambda)$ has a multiple root only if $P(\lambda)$ has a multiple root at some $\lambda_0 \neq 1$.

Let $\lambda_0 \neq 1$ satisfy $P(\lambda_0) = P'(\lambda_0) = 0$. Note $\lambda_0 \neq 0$ since $P(0) = q > 0$. From $P'(\lambda_0) = 0$:

$$l_B \lambda_0^{l_B - 1} = p(l_B - l_A) \lambda_0^{l_B - l_A - 1} \implies \lambda_0^{l_A} = \frac{p(l_B - l_A)}{l_B}.$$

Substituting into $P(\lambda_0) = 0$ and using $\lambda_0^{l_B} = \lambda_0^{l_A} \lambda_0^{l_B - l_A}$ gives:

$$-\lambda_0^{l_A} \lambda_0^{l_B - l_A} + p \lambda_0^{l_B - l_A} + q = 0 \implies \lambda_0^{l_B - l_A} = -\frac{ql_B}{pl_A}.$$

Equating $\lambda_0^{l_A(l_B - l_A)} = (\lambda_0^{l_A})^{l_B - l_A} = (\lambda_0^{l_B - l_A})^{l_A}$ and taking absolute values yields the necessary condition:

$$\left(\frac{p(l_B - l_A)}{l_B} \right)^{l_B - l_A} = \left(\frac{ql_B}{pl_A} \right)^{l_A} \iff \frac{(1 - p)^{l_A}}{p^{l_B}} = \frac{(l_B - l_A)^{l_B - l_A} l_A^{l_A}}{l_B^{l_B}} =: C.$$

Define $h(p) = (1-p)^{l_A}/p^{l_B}$. Then

$$\frac{d}{dp} \log h(p) = -\frac{l_A}{1-p} - \frac{l_B}{p} < 0 \quad \text{for } p \in (0, 1),$$

so $h(p)$ is strictly decreasing on $(0, 1)$. Therefore $h(p) = C$ has at most one solution $p^* \in (0, 1)$, and the set of p for which $A(\lambda)$ has multiple roots is finite. For generic p , all roots of $A(\lambda)$ are simple. $\square$

THEOREM 5 (Local Stability). *Let $\gcd(l_A, l_B) = 1$. For every $p \in (0, 1)$ and for sufficiently large l_0 , all roots of $F(\lambda) = 0$ satisfy $|\lambda| < 1$.*

Proof. When $g = 1$, Lemmas 4 and 5 imply all roots of $A(\lambda)$ satisfy $|\alpha_j| < 1$. Let $r = \max_j |\alpha_j| < 1$, and choose ρ with $r < \rho < 1$. On the circle $|\lambda| = \rho$, the polynomial A has no zeros, so

$$m_\rho := \min_{|\lambda|=\rho} |A(\lambda)| > 0, \quad M_\rho := \max_{|\lambda|=\rho} |\lambda A'(\lambda)| < \infty.$$

Using decomposition (39) with $\epsilon = 1/(l_0 + l_B)$, the finite equation is

$$A(\lambda) - \epsilon \lambda A'(\lambda) = 0.$$

For all sufficiently large l_0 , $\epsilon M_\rho < m_\rho$ on $|\lambda| = \rho$. Rouché's theorem then implies that $A - \epsilon \lambda A'$ and A have the same number of roots in $|\lambda| < \rho$. Since A has degree $l_B - 1$, this accounts for all roots of $F(\lambda) = 0$, and they all satisfy $|\lambda| < \rho < 1$. $\square$

EXAMPLE 9 (IFT ROOT DRIFT). For the two-class examples ($l_A = 2$, $l_B = 3$ or 4 , $p = q = 1/2$), Figure 15 plots the spectral radius $|\lambda_{\max}|$ of $F(\lambda)$ as l_0 increases. In the non-coprime case ($l_A = 2$, $l_B = 4$, $\gcd = 2$), the root at $\omega = -1$ drifts to $|\lambda| \simeq 1 + 1/(l_0 + 4)$, remaining above 1 for all finite l_0 . In the coprime case ($l_A = 2$, $l_B = 3$), $|\lambda_{\max}| \rightarrow 1/\sqrt{2} \simeq 0.707$ as $l_0 \rightarrow \infty$, with a safety margin that persists at every finite l_0 .

B.6. Global Convergence for Coprime Case

Local stability (Theorem 5) shows the equilibrium is stable to small perturbations. We now prove global convergence: the system reaches equilibrium from *any* initial condition, even when eviction occurs during transients. Figure 4(a) in Section 4 illustrates this convergence for the coprime case $\gcd(2, 3) = 1$.

Throughout this section, we work in the asymptotic regime $l_0 \rightarrow \infty$: memory scales as $M = \beta l_0$ for fixed $\beta > 0$; decoding lengths l_A, l_B are constants; and $\gcd(l_A, l_B) = 1$. All asymptotic notation is with respect to $l_0 \rightarrow \infty$. As l_0 grows, eviction perturbations become $O(1/l_0)$ while state variables remain $O(1)$, so the contractive linear dynamics dominate.

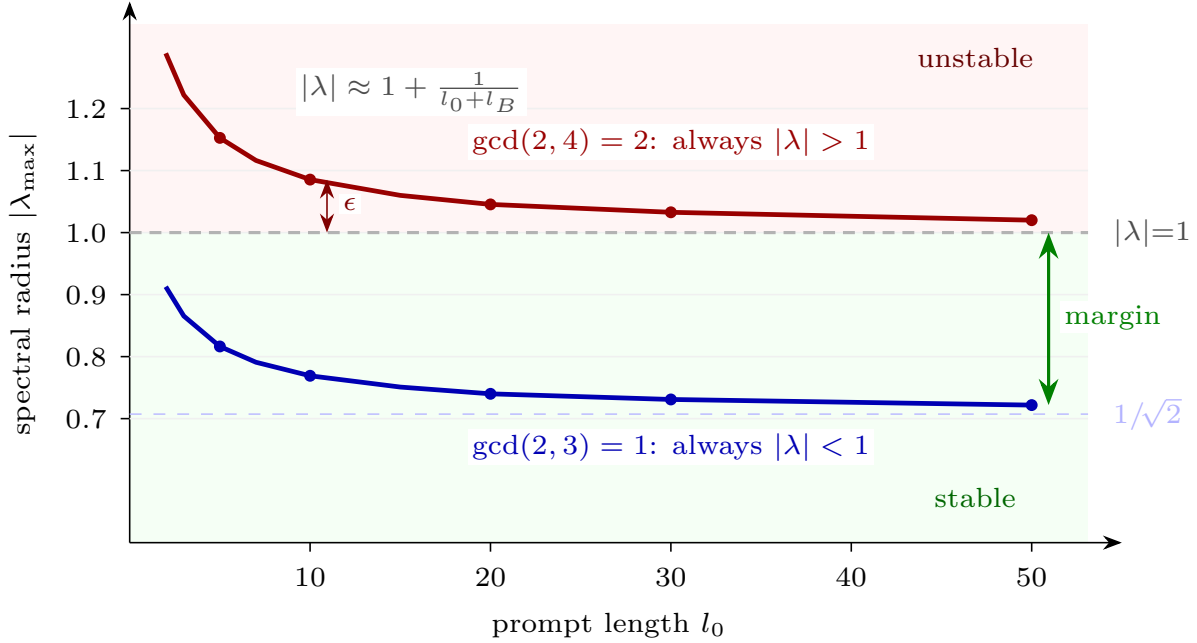


Figure 15 Spectral radius of $F(\lambda)$ versus prompt length l_0 ($p = q = 1/2$). Non-coprime case ($l_1^{(1)} = 2, l_1^{(2)} = 4$, red): the root at $\lambda = -1$ of $A(\lambda)$ drifts to $|\lambda| > 1$ for every finite l_0 (Theorem 4). Coprime case ($l_1^{(1)} = 2, l_1^{(2)} = 3$, blue): all roots remain strictly inside the unit circle (Theorem 5), with a safety margin that persists as $l_0 \rightarrow \infty$. Both curves approach their limiting values (1 and $1/\sqrt{2}$) as $l_0 \rightarrow \infty$, confirming the IFT first-order approximation $|\lambda| \approx |\alpha|(1 + 1/(l_0 + l_1^{(2)}))$.

B.6.1. Normalized System We write $X_{n,s}$ for the number of requests at stage s at time n (equivalently $x_n^{(s)}$ in the earlier notation). The original state $X_n \in \mathbb{R}^{l_B}$ has non-uniform equilibrium: $X_s^* = x^*$ for $s < l_A$ and $X_s^* = qx^*$ for $s \geq l_A$, where x^* denotes the equilibrium admission rate. (At equilibrium, a fraction p of admissions are type A and q are type B. For $s < l_A$, both types contribute, giving $px^* + qx^* = x^*$ requests at stage s . For $s \geq l_A$, only type-B requests remain, giving qx^* .) Define the normalized system:

$$Y_{n,s} = \begin{cases} X_{n,s} & s < l_A \\ X_{n,s}/q & s \geq l_A \end{cases}$$

with weights $V_s = l_0 + s + 1$ for $s < l_A$ and $V_s = q(l_0 + s + 1)$ for $s \geq l_A$. The equilibrium becomes uniform: $Y^* = [x^*, \dots, x^*]^\top$.

The equilibrium admission rate x^* satisfies:

$$x^* = \frac{M}{\sum_s V_s} = \frac{\beta l_0}{\sum_{s=0}^{l_A-1} (l_0 + s + 1) + q \sum_{s=l_A}^{l_B-1} (l_0 + s + 1)}.$$

The denominator expands to $(l_A + q(l_B - l_A))l_0 + \Theta(1) = (pl_A + ql_B)l_0 + \Theta(1)$. Letting $\kappa = pl_A + ql_B$, we have:

$$x^* = \frac{\beta l_0}{\kappa l_0 + \Theta(1)} = \frac{\beta}{\kappa} + O(1/l_0) = \Theta(1).$$

B.6.2. Linear Dynamics and Lyapunov Function The deviation $y_n = Y_n - Y^*$ evolves under the transition matrix $\Phi \in \mathbb{R}^{l_B \times l_B}$:

$$\Phi = \begin{pmatrix} -V_1/V_0 & -V_2/V_0 & \cdots & -V_{l_B-1}/V_0 & 0 \\ 1 & 0 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}.$$

Since the last column of Φ is zero, $\lambda = 0$ is an eigenvalue. The remaining eigenvalues are determined by the $(l_B - 1) \times (l_B - 1)$ leading principal submatrix, which is a companion matrix whose characteristic polynomial is $F(\lambda)/(l_0 + 1)$. Hence the nonzero eigenvalues of Φ are exactly the $l_B - 1$ roots of $F(\lambda) = 0$.

LEMMA 8 (Spectral Bound). *When $\gcd(l_A, l_B) = 1$, there exists l_0^* such that for $l_0 \geq l_0^*$, all eigenvalues of Φ satisfy $|\lambda_j| \leq \rho < 1$ for some fixed ρ independent of l_0 .*

Proof. The proof of Theorem 5 gives a radius $\rho < 1$ and an l_0^* such that every root of $F(\lambda) = 0$ lies in $|\lambda| < \rho$ for $l_0 \geq l_0^*$. These roots are exactly the nonzero eigenvalues of Φ , and the remaining eigenvalue is 0. $\square$

LEMMA 9 (Lyapunov Matrix Bounds). *For $l_0 \geq l_0^*$ (Lemma 8), let $\Phi = \Phi(l_0)$ and let $P = P(l_0)$ be the unique positive definite solution to the discrete Lyapunov equation $\Phi^\top P \Phi - P = -I$. There exists $C_P > 0$, independent of l_0 , such that $I \preceq P \preceq C_P I$.*

Proof. Since all eigenvalues of Φ satisfy $|\lambda_j| < 1$, the series $P = \sum_{k=0}^{\infty} (\Phi^k)^\top \Phi^k$ converges to the unique solution of $\Phi^\top P \Phi - P = -I$. The identity $P - \Phi^\top P \Phi = I$ gives $P \succeq I$. For the upper bound, note that the entries of $\Phi(l_0)$ are ratios V_s/V_0 , so $\Phi(l_0)$ depends continuously on $1/(l_0 + 1)$ and the family $\{\Phi(l_0) : l_0 \geq l_0^*\}$ has compact closure.

Vectorizing the Lyapunov equation gives:

$$(I - \Phi^\top \otimes \Phi^\top) \text{vec}(P) = \text{vec}(I).$$

Since $\rho(\Phi) \leq \rho < 1$ (Lemma 8), we have $\rho(\Phi^\top \otimes \Phi^\top) = \rho(\Phi)^2 \leq \rho^2 < 1$, so $I - \Phi^\top \otimes \Phi^\top$ is invertible for all $l_0 \geq l_0^*$. The inverse depends continuously on Φ , hence $\|(I - \Phi^\top \otimes \Phi^\top)^{-1}\|_2$ is

bounded on the compact closure. Consequently $\|P\|_2$ is uniformly bounded for $l_0 \geq l_0^*$, implying $P \preceq C_P I$ for some constant C_P independent of l_0 . $\square$

The Lyapunov function $L(y) = y^\top P y$ satisfies $\|y\|_2^2 \leq L(y) \leq C_P \|y\|_2^2$ and $L(\Phi y) - L(y) = -\|y\|_2^2$.

EXAMPLE 10 (TRANSITION MATRIX AND LYAPUNOV BOUNDS). Continuing the coprime example ($l_A = 2$, $l_B = 3$, $l_0 = 10$, $p = q = 1/2$, $M = 118$), the normalized weights are $V_0 = 11$, $V_1 = 12$, $V_2 = 13/2$, giving $\sum V_s = 59/2$ and equilibrium $x^* = 2 \cdot 118/59 = 4$. The transition matrix is

$$\Phi = \begin{pmatrix} -12/11 & -13/22 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix},$$

a companion matrix whose nonzero eigenvalues satisfy $\lambda^2 + \frac{12}{11}\lambda + \frac{13}{22} = 0$, equivalently $F(\lambda)/(l_0 + 1) = 0$. The roots are $\lambda = (-12 \pm i\sqrt{142})/22$ with modulus $|\lambda| = \sqrt{13/22} \approx 0.769$, confirming $\rho < 1$ (Lemma 8).

Solving the Lyapunov equation $\Phi^\top P \Phi - P = -I$ yields $C_P = \|P\|_2 \approx 10.8$. The lower bound $P \succeq I$ is tight: $\lambda_{\min}(P) = 1$ (the $k = 0$ term in the series $P = \sum_k (\Phi^k)^\top \Phi^k$ already contributes I). The resulting convergence rate $1 - 1/C_P \approx 0.907$ per step is conservative compared to the observed spectral rate $\rho^2 \approx 0.59$ (see Example 11).

B.6.3. Eviction Bounds

LEMMA 10 (Physical Bounds). *The total number of in-flight requests is $O(1)$, and the number of evicted requests per step is $O(1/l_0)$.*

Proof. Let $N_n = \sum_{s=0}^{l_B-1} X_{n,s}$ be the total number of (actual) requests in the system at time n . Each request occupies at least $l_0 + 1$ tokens, so:

$$N_n \leq \frac{M}{l_0 + 1} = \frac{\beta l_0}{l_0 + 1} = O(1).$$

In one iteration, each in-flight request generates one new token during decoding, so the post-shift token increase is at most $N_n = O(1)$. Completions can only reduce memory, and the admission controller never increases memory beyond M (it admits at most the available slack). Therefore, the number of tokens that must be removed by eviction in any step is at most the $O(1)$ overflow created by the shift.

Since evicting one request frees at least $l_0 + 1$ tokens, the number of evicted requests per step is at most $O(1)/(l_0 + 1) = O(1/l_0)$. $\square$

The actual evolution is $y_n = \Phi y_{n-1} + e_n$, where $e_n \in \mathbb{R}^{l_B}$ captures the deviation caused by eviction.

LEMMA 11 (Perturbation Bound). *There exists a constant $C_e > 0$, independent of l_0 , such that $\|e_n\|_2 \leq C_e/l_0$ for all n .*

Proof. The perturbation $e_n = y_n - \Phi y_{n-1}$ arises when eviction truncates the linear update. At stage 0, eviction replaces the ideal admission $\tilde{a}_n < 0$ with zero, so $|e_{n,0}| = |\tilde{a}_n| = (E^S - M)/V_0 = O(1)/(l_0 + 1) = O(1/l_0)$ by Lemma 10. At stages $s \geq 1$, the eviction removes $O(1/l_0)$ requests in total (Lemma 10), giving $\sum_{s \geq 1} |(e_n)_s| \leq O(1/l_0)/\min(1, q) = O(1/l_0)$. Hence $\|e_n\|_2 \leq \|e_n\|_1 = O(1/l_0)$. $\square$

Figure 16 illustrates the mechanism behind Lemmas 10–11. Panel (a) contrasts token growth per step ($O(1)$) with the memory freed by a single eviction ($\geq l_0 + 1$): at $l_0 = 100$, one eviction clears roughly 9 times the overflow. Panel (b) plots the resulting upper bound on evicted requests per step as a function of l_0 . This bound is uniform over all time steps n and ensures $\|e_n\|_2 \leq C_e/l_0$, which is small enough for the Lyapunov contraction to dominate the perturbation when l_0 is large.

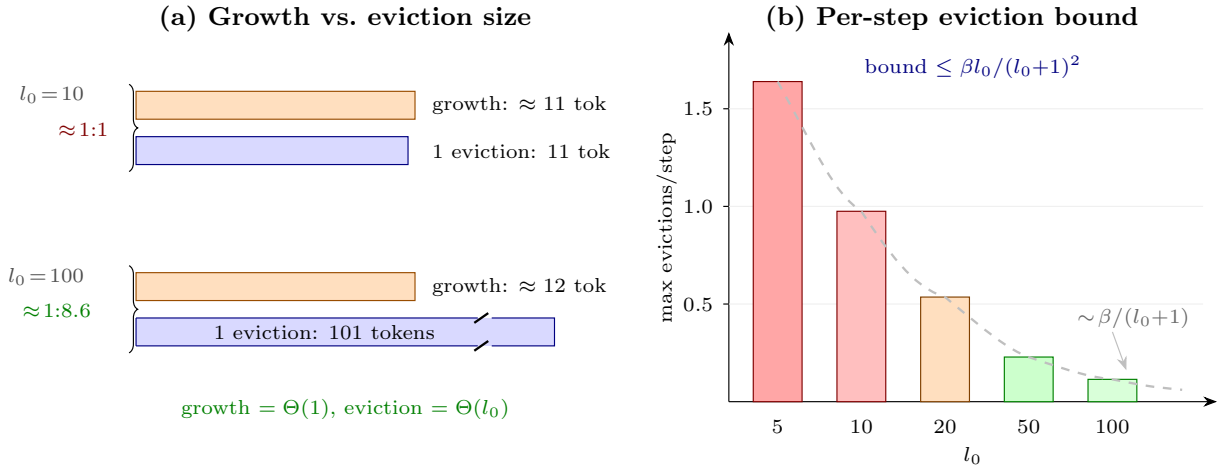


Figure 16 Eviction perturbation bound (Lemmas 10–11). (a) Token growth per step ($\Theta(1)$) versus memory freed by one eviction ($\geq l_0 + 1$). At $l_0 = 100$, one eviction clears ≈ 9 times the overflow, so each step triggers at most a fraction of an eviction. (b) Upper bound on evicted requests per step as a function of l_0 ($\beta = 11.8$). The bound $\beta l_0 / (l_0 + 1)^2 = O(1/l_0)$ is uniform over all time steps n ; it guarantees $\|e_n\|_2 \leq C_e/l_0$ for a constant C_e independent of n .

B.6.4. Convergence Proof

THEOREM 6 (Global Convergence). *Let $\gcd(l_A, l_B) = 1$ and $M = \beta l_0$. For every $p \in (0, 1)$ and sufficiently large l_0 , the system converges exponentially to equilibrium from any initial state.*

Proof. We construct the neighborhood explicitly. Define:

$$\delta = \frac{\beta}{4C_2\kappa} > 0, \quad \text{where } C_2 = \sqrt{l_B} \cdot \max_s V_s / l_0.$$

Since $\max_s V_s \leq l_0 + l_B$, we have $C_2 \leq \sqrt{l_B}(1 + l_B/l_0)$ and in particular $C_2 \leq 2\sqrt{l_B}$ for large l_0 . Thus $\delta = \Theta(1)$ and is effectively independent of l_0 . Define the Lyapunov sublevel set $\mathcal{N} = \{y : L(y) < \delta^2\}$. Since $L(y) \geq \|y\|_2^2$ (Lemma 9), every $y \in \mathcal{N}$ satisfies $\|y\|_2 < \delta$.

We first show that no eviction occurs inside $\mathcal{N}$. The post-shift memory E^S (before admission) is:

$$E^S = \sum_{s=0}^{l_B-2} V_{s+1} Y_{n,s} = \sum_{s=0}^{l_B-2} V_{s+1} (x^* + y_{n,s}).$$

At equilibrium, $E^{S,*} = \sum_{s=0}^{l_B-2} V_{s+1} x^* = M - V_0 x^*$. The deviation satisfies:

$$|E^S - E^{S,*}| = \left| \sum_{s=0}^{l_B-2} V_{s+1} y_{n,s} \right| \leq \max_s V_s \cdot \sqrt{l_B} \|y_n\|_2 \leq C_2 l_0 \|y_n\|_2.$$

When $\|y_n\|_2 \leq \delta$:

$$\begin{aligned} E^S - M &\leq -V_0 x^* + C_2 l_0 \delta \\ &= -\frac{(l_0 + 1)\beta}{\kappa + O(1/l_0)} + C_2 l_0 \cdot \frac{\beta}{4C_2\kappa} \\ &\leq -\frac{\beta l_0}{2\kappa} + \frac{\beta l_0}{4\kappa} = -\frac{\beta l_0}{4\kappa} < 0. \end{aligned}$$

Hence, no eviction occurs inside $\mathcal{N}$ for sufficiently large l_0 .

Outside $\mathcal{N}$, the Lyapunov energy strictly decreases. The one-step change is:

$$\begin{aligned} \Delta L_n &= L(y_n) - L(y_{n-1}) = (\Phi y_{n-1} + e_n)^\top P (\Phi y_{n-1} + e_n) - y_{n-1}^\top P y_{n-1} \\ &= y_{n-1}^\top (\Phi^\top P \Phi - P) y_{n-1} + 2e_n^\top P \Phi y_{n-1} + e_n^\top P e_n \\ &= -\|y_{n-1}\|_2^2 + R_n, \end{aligned}$$

where the residual term is $R_n = 2e_n^\top P \Phi y_{n-1} + e_n^\top P e_n$. By Lemmas 9 and 11:

$$|R_n| \leq 2\|e_n\|_2 \cdot \|P\|_2 \cdot \|\Phi\|_2 \cdot \|y_{n-1}\|_2 + \|P\|_2 \|e_n\|_2^2 \leq \frac{C_3}{l_0} \|y_{n-1}\|_2 + \frac{C_4}{l_0^2},$$

where we used $\|P\|_2 \leq C_P$ (Lemma 9) and $\|e_n\|_2 \leq C_e/l_0$ (Lemma 11). Moreover, for all sufficiently large l_0 , the coefficients satisfy $|V_s/V_0| \leq 2$, so $\|\Phi\|_2 \leq \|\Phi\|_F \leq \sqrt{5(l_B - 1)} =: C_\Phi$. Thus $C_3 = 2C_P C_\Phi C_e$ and $C_4 = C_P C_e^2$.

Outside $\mathcal{N}$, $L(y_{n-1}) \geq \delta^2$, so $\|y_{n-1}\|_2^2 \geq L(y_{n-1})/C_P \geq \delta^2/C_P$. Hence:

$$\Delta L_n \leq -\frac{\delta^2}{C_P} + \frac{C_3 M_y}{l_0} + \frac{C_4}{l_0^2},$$

where $M_y = \sup_n \|y_n\|_2 = O(1)$. Indeed, $\sum_s X_{n,s} \leq M/(l_0 + 1) = O(1)$ implies $\|Y_n\|_2 \leq \|Y_n\|_1 \leq \frac{1}{\min(1,q)} \sum_s X_{n,s} = O(1)$, while $\|Y^*\|_2 = \sqrt{l_B} x^* = \Theta(1)$, so $\|y_n\|_2 \leq \|Y_n\|_2 + \|Y^*\|_2 = O(1)$ uniformly in n . Choose $l_0^{(3)}$ such that $(C_3 M_y + C_4)/l_0 \leq \delta^2/(2C_P)$ for $l_0 \geq l_0^{(3)}$. Then $\Delta L_n \leq -\delta^2/(2C_P) < 0$.

The system enters $\mathcal{N}$ in finite time: the initial Lyapunov value satisfies $L(y_0) \leq C_P M_y^2$, and the guaranteed decrease of $\delta^2/(2C_P)$ per step gives entry within $T^* = \lceil 2C_P^2 M_y^2 / \delta^2 \rceil$ steps. Once inside, the system remains in $\mathcal{N}$: since $e_n = 0$ inside $\mathcal{N}$ (no eviction occurs there), $L(y_n) = L(y_{n-1}) - \|y_{n-1}\|_2^2 < L(y_{n-1})$, so the sublevel set is invariant.

Finally, the convergence inside $\mathcal{N}$ is exponential. With $e_n = 0$, the dynamics reduce to $y_n = \Phi y_{n-1}$. Using $L(\Phi y) - L(y) = -\|y\|_2^2$ and $L(y) \leq C_P \|y\|_2^2$:

$$L(y_n) = L(y_{n-1}) - \|y_{n-1}\|_2^2 \leq \left(1 - \frac{1}{C_P}\right) L(y_{n-1}).$$

Iterating gives $L(y_n) \leq (1 - 1/C_P)^{n-n_0} L(y_{n_0})$, hence $\|y_n\|_2 \rightarrow 0$ geometrically and the convergence is exponential. $\square$

The following example traces each phase of the convergence mechanism on a concrete instance.

EXAMPLE 11 (LYAPUNOV CONVERGENCE ILLUSTRATION). Using the coprime parameters of Example 4 ($l_A = 2$, $l_B = 3$, $l_0 = 10$, $p = q = 1/2$, $M = 118$), the transition matrix $\Phi \in \mathbb{R}^{3 \times 3}$ has spectral radius $\rho \simeq 0.769$, the discrete Lyapunov equation $\Phi^\top P \Phi - P = -I$ gives $C_P \simeq 10.8$, and the eviction-free threshold is $\delta \simeq 0.57$ (so $\delta^2 \simeq 0.32$).

Near equilibrium, the post-shift memory $E^{S,*} = V_1 x^* + V_2 x^* = 74$ consumes 63% of $M = 118$, leaving $V_0 x^* = 44$ units for new admissions. Under the worst perturbation $\|y\| = \delta$, the memory deviation is at most $C_2 l_0 \delta \simeq 12$ units, so the admission margin remains 32 units above zero (Figure 17).

Starting from $y_0 = (1, 0, 0)^\top$, the energy $L(y_0) = 8.70$ decreases by $\|y_{n-1}\|_2^2$ at each step. The guaranteed decrease outside $\mathcal{N}$ exceeds $\delta^2/(2C_P) \simeq 0.015$ per step, but the actual decreases range from 0.19 to 2.55. The energy crosses δ^2 at $T^* = 8$ (Figure 18).

Inside $\mathcal{N}$, no eviction occurs, so the pure linear dynamics $y_n = \Phi y_{n-1}$ take over. The trajectory remains in $\mathcal{N}$ and converges geometrically:

$$L(y_n) \leq (1 - 1/C_P)^{n-n_0} L(y_{n_0}).$$

This gives a theoretical rate of $1 - 1/C_P \simeq 0.907$ per step, while the observed decay is closer to $\rho^2 \simeq 0.59$. The state spirals to the origin in the (y_0, y_1) -plane (Figure 19).

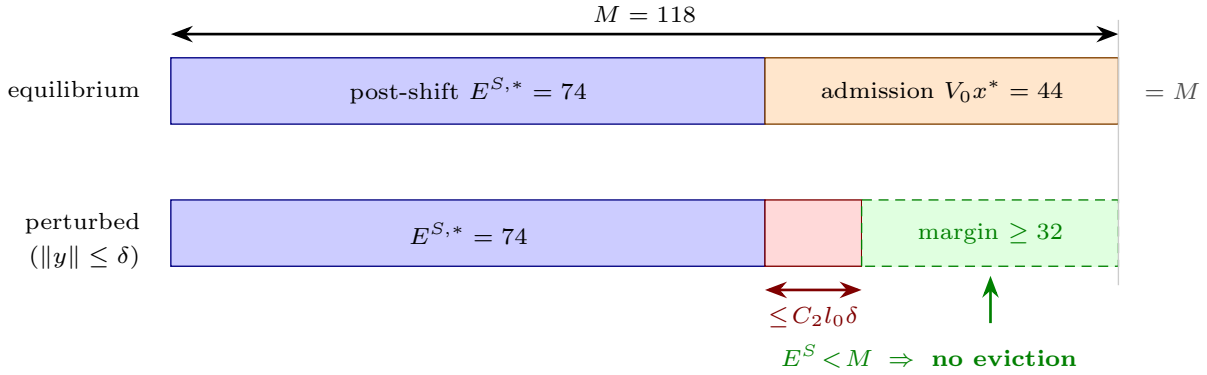


Figure 17 Step 1 illustration (Example 11): post-shift memory budget. At equilibrium, admitting $x^* = 4$ requests fills memory to exactly M . Under worst-case perturbation $\|y\| \leq \delta$, the deviation $C_2 l_0 \delta \approx 12$ leaves a margin of 32 units, so no eviction is triggered inside $\mathcal{N}$.

The mechanism behind this convergence is the scaling of eviction perturbations. When $M = \beta l_0$, the number of in-flight requests is $O(1)$, so each perturbation is $O(1/l_0)$. As $l_0 \rightarrow \infty$, these perturbations vanish relative to the $O(1)$ state variables, and the contractive linear dynamics dominate.

B.7. Finite-Prompt Spectral Drift and Stability Threshold

This subsection proves the finite-prompt statements used in Section 4.3. We use the shorthand $a := l_A$, $n := l_B$, $q := 1 - p$, and $\beta_n := a/n$. The proof has three parts. First, we quantify how each limiting root moves when $\varepsilon = (l_0 + n)^{-1}$ is positive. Second, we identify the limiting root closest to the unit circle. Third, we balance these two effects to obtain the finite- l_0 stability threshold.

We write

$$P_n(\lambda) := \lambda^n - p\lambda^{n-a} - q.$$

Since $A(1) = pa + qn > 0$, the unit root of P_n is removed by the $(1 - \lambda)$ factor in (38). Thus the roots of A are exactly the non-unit roots of P_n .

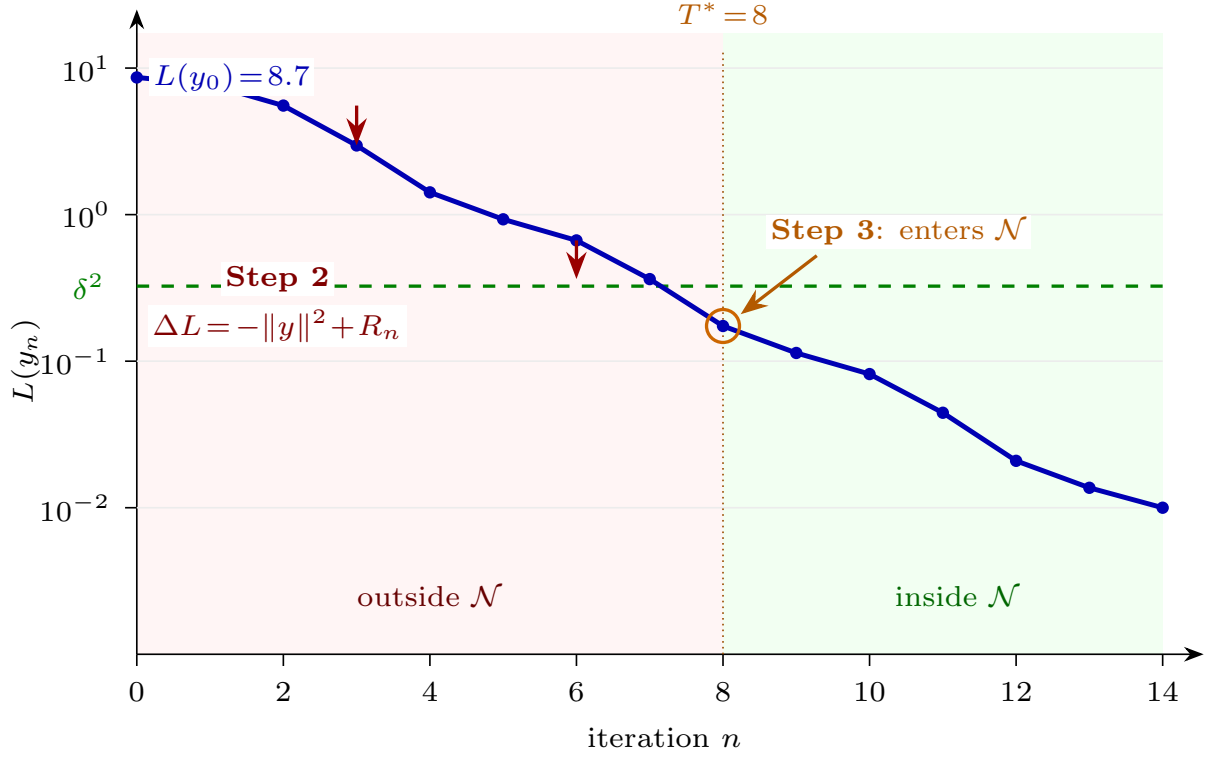


Figure 18 Steps 2–3 illustration (Example 11): energy descent on log scale. Outside $\mathcal{N}$ (red background), the Lyapunov energy $L(y_n)$ decreases at each step (Step 2): the actual decreases (0.19–2.55) far exceed the guaranteed minimum $\delta^2/(2C_P) \approx 0.015$. The energy crosses δ^2 at $T^* = 8$ (Step 3, orange dot).

LEMMA 12 (Near-unit logarithmic chart). Fix $p \in (0, 1)$. There exist constants $\delta, \eta, c > 0$, independent of the coprime pair $1 \leq a < n$, such that every root $\lambda \neq 1$ of P_n with $|\lambda| \geq e^{-\eta/n}$ can be represented as

$$\lambda = \omega e^{-\mu/n}, \quad \omega^n = 1, \quad \Im \mu \in [-\pi, \pi],$$

after choosing an n -th root ω nearest to the argument of λ . If $x \in (-\pi, \pi]$ is defined by $\omega^{-a} = e^{ix}$, then

$$H(\mu, x, \beta_n) = 0, \quad \beta_n = a/n,$$

where

$$H(\mu, x, \beta) := e^{-\mu} - p e^{ix} e^{-(1-\beta)\mu} - q.$$

For $|x| \leq \delta$ and $\Re \mu \leq \eta$, this equation has the unique solution $\mu = \mu(x, \beta)$ near zero. For $|x| \geq \delta$, every such near-unit solution satisfies $\Re \mu \geq c$.

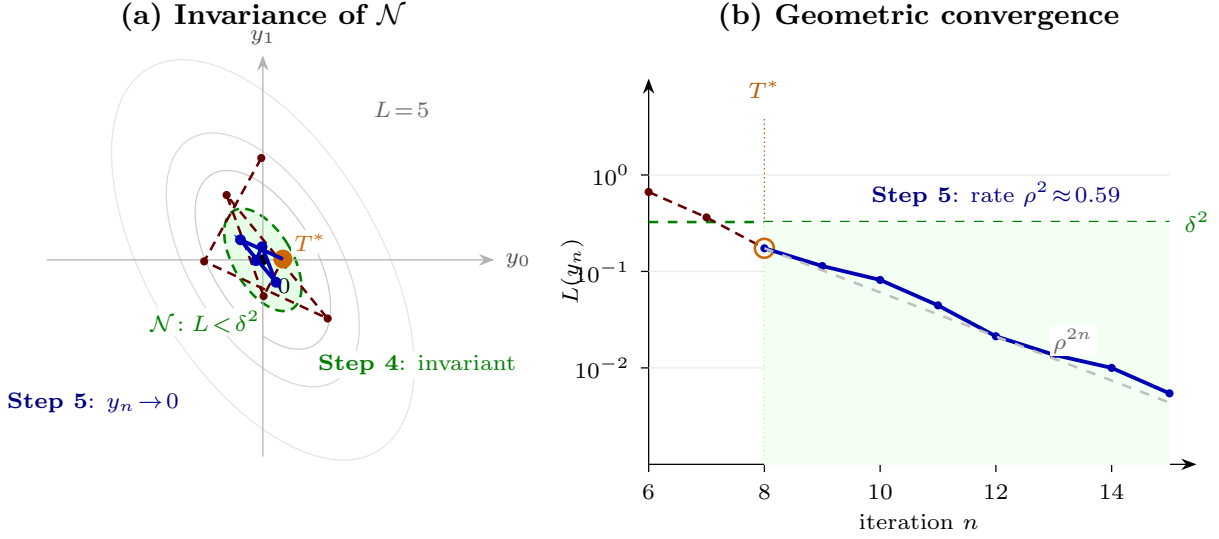


Figure 19 Steps 4–5 illustration (Example 11). (a) State-space trajectory in the (y_0, y_1) plane. The system enters $\mathcal{N} = \{L < \delta^2\}$ (green dashed ellipse) at $T^* = 8$ (orange) and remains inside (Step 4, invariance). Blue segments show the trajectory spiraling to the origin under pure linear dynamics (Step 5). (b) Energy $L(y_n)$ on log scale near T^* . Inside $\mathcal{N}$, the decay tracks the geometric rate $\rho^2 \approx 0.59$ (gray dashed).

Proof. First note that all roots of P_n lie in the closed unit disk. Indeed, if $R := |\lambda| > 1$, then

$$R^n = |p\lambda^{n-a} + q| \leq pR^{n-a} + q < pR^n + q,$$

which implies $R^n < 1$, a contradiction. Hence the logarithmic coordinates used below always satisfy $\Re \mu \geq 0$.

The representation is just the principal logarithm in the sector around a nearest n -th root of unity: once ω is chosen, set $\mu = -n \operatorname{Log}(\lambda/\omega)$, with $\Im \mu \in [-\pi, \pi]$. Since $\omega^n = 1$ and $\omega^{-a} = e^{ix}$, substituting $\lambda = \omega e^{-\mu/n}$ into $P_n(\lambda) = 0$ gives the displayed equation for H . If a root lies on a sector boundary and has two nearest n -th roots of unity, fix either one by an arbitrary deterministic tie-breaking rule. The estimates below are invariant under this choice. We use the chart only for coverage and uniform bounds; a boundary root that admits two labels is counted once as a root of P_n .

The only point that needs care is uniformity. At $(\mu, x) = (0, 0)$,

$$H_\mu(0, 0, \beta) = -(q + p\beta),$$

which is bounded away from zero uniformly over $\beta \in [0, 1]$, since $q > 0$. The implicit-function theorem therefore gives a unique branch $\mu(x, \beta)$ for $|x| \leq \delta$, with δ chosen uniformly in β . We also

need that this local branch accounts for all solutions in the near-unit strip when $|x| \leq \delta$. This follows by compactness. At $x = 0$ and $\Re \mu = 0$, writing

$$u = e^{-\mu}, \quad v = e^{-(1-\beta)\mu},$$

the equation $H(\mu, 0, \beta) = 0$ gives $u = pv + q$ with $|u| = |v| = 1$. Equality in the triangle inequality forces $u = v = 1$, and therefore $\mu = 0$ in the strip $\Im \mu \in [-\pi, \pi]$. Consequently, after choosing η and δ small enough, there are no solutions of $H(\mu, x, \beta) = 0$ with

$$0 \leq \Re \mu \leq \eta, \quad |x| \leq \delta, \quad \Im \mu \in [-\pi, \pi],$$

outside the fixed IFT neighborhood of $\mu = 0$. Thus the displayed branch is the unique near-unit solution for small phases.

If the last assertion failed, compactness would give a limit (μ_*, x_*, β_*) with $\Re \mu_* = 0$, $|x_*| \geq \delta$, and

$$e^{-\mu_*} - p e^{ix_*} e^{-(1-\beta_*)\mu_*} - q = 0.$$

Writing $u = e^{-\mu_*}$ and $v = e^{ix_*} e^{-(1-\beta_*)\mu_*}$, we have $|u| = |v| = 1$. The equation $u = pv + q$ can hold with $p, q > 0$ and $|u| = |v| = 1$ only when $u = v = 1$, by equality in the triangle inequality. Hence $x_* = 0 \pmod{2\pi}$, contradicting $|x_*| \geq \delta$ with $x_* \in (-\pi, \pi]$. Therefore phases bounded away from zero have $\Re \mu \geq c$ uniformly. $\square$

B.7.1. Proof of Proposition 1 We next evaluate the limiting spectral gap. The limiting roots of A are most transparent in logarithmic coordinates around the n -th roots of unity. For each $m \in \{0, \dots, n-1\}$, let

$$x_m := \text{the principal representative of } \frac{2\pi m}{n} \pmod{2\pi}, \quad x_m \in (-\pi, \pi].$$

Because $\gcd(a, n) = 1$, multiplication by a is invertible modulo n . Hence there is a unique $u_m \pmod{n}$ such that

$$au_m \equiv -m \pmod{n}.$$

Define

$$\omega_{m,n} := e^{2\pi i u_m / n}.$$

Then $\omega_{m,n}^n = 1$ and $\omega_{m,n}^{-a} = e^{ix_m}$.

For $m \neq 0$, the near-unit root chart in Lemma 12 parameterizes the root branch associated with this phase as

$$\lambda = \omega_{m,n} \exp(-\mu/n).$$

Substituting this expression into $P_n(\lambda) = 0$ gives

$$H(\mu, x_m, \beta_n) = 0, \quad \beta_n := a/n,$$

where

$$H(\mu, x, \beta) := e^{-\mu} - p e^{ix} e^{-(1-\beta)\mu} - q.$$

At phase $x = 0$, the solution is $\mu = 0$ for every $\beta \in [0, 1]$. Moreover,

$$H_\mu(0, 0, \beta) = -(q + p\beta) \neq 0.$$

Thus the implicit-function theorem gives a smooth local branch $\mu(x, \beta)$, uniformly for $\beta \in [0, 1]$, such that

$$H(\mu(x, \beta), x, \beta) = 0, \quad \mu(0, \beta) = 0.$$

Let $D(\beta) := q + p\beta$. Differentiating implicitly at $x = 0$ gives

$$\mu_x(0, \beta) = -i \frac{p}{D(\beta)}$$

and

$$\mu_{xx}(0, \beta) = \frac{pq}{D(\beta)^3}.$$

Consequently, near the boundary phase $x = 0$,

$$\Re \mu(x, \beta) = \frac{pq}{2D(\beta)^3} x^2 + O(|x|^3),$$

uniformly in β . Branches bounded away from $x = 0$ are separated from the unit circle by Lemma 12: for $|x| \geq \delta$, near-unit roots satisfy $\Re \mu \geq c_\delta > 0$, and therefore $|\lambda| \leq e^{-c_\delta/n} = 1 - \Theta(n^{-1})$. Roots outside the near-unit chart have an even larger radial gap.

Because a and n are coprime, the nonzero phase set is

$$\{\pm 2\pi/n, \pm 4\pi/n, \dots\}.$$

After shrinking δ , the Taylor remainder is at most one half of the quadratic term for $|x| \leq \delta$, uniformly in $\beta \in [0, 1]$. Hence among phases with $|x| \leq \delta$, the roots closest to the unit circle are exactly those with $|x| = 2\pi/n$. For these branches,

$$\Re \mu(2\pi/n, \beta_n) = \frac{2\pi^2 pq}{D(\beta_n)^3} n^{-2} + O(n^{-3}).$$

Therefore

$$\rho_\infty = \exp\left(-\frac{1}{n} \Re \mu(2\pi/n, \beta_n)\right) = 1 - \frac{2\pi^2 pq}{(q + p\beta_n)^3} n^{-3} + O(n^{-4}).$$

Returning to the original notation $\beta_n = l_A/l_B$ and $n = l_B$ gives

$$1 - \rho_\infty = \frac{2\pi^2 p(1-p)}{((1-p) + p\beta_n)^3} (l_B)^{-3} + O((l_B)^{-4}),$$

which is (15).

The same estimates also give the separation needed later. If $x_k = 2\pi k/n$ and $2 \leq k \leq \delta n/(2\pi)$, then

$$\Re \mu(x_k, \beta_n) - \Re \mu(x_1, \beta_n) \geq c(k^2 - 1)n^{-2},$$

so the corresponding moduli are below the dominant modulus by at least cn^{-3} . If $|x_k| \geq \delta$, Lemma 12 gives $1 - |\alpha_k| \geq c_\delta n^{-1}$. Thus every non-dominant limiting branch is separated from the dominant modulus by at least cn^{-3} .

B.7.2. Proof of Theorem 3 Let α be a simple root of A . The key point is that the finite-prompt correction enters the characteristic equation as a small radial perturbation of the limiting polynomial. Dividing (39) by $l_0 + n$, the finite- l_0 characteristic equation is

$$G(\lambda, \varepsilon) := A(\lambda) - \varepsilon \lambda A'(\lambda) = 0, \quad \varepsilon = \frac{1}{l_0 + n}.$$

At $(\lambda, \varepsilon) = (\alpha, 0)$, we have $G(\alpha, 0) = 0$ and $G_\lambda(\alpha, 0) = A'(\alpha) \neq 0$. The analytic implicit-function theorem therefore identifies a unique root branch $\lambda_\alpha(\varepsilon)$, analytic in ε , with $\lambda_\alpha(0) = \alpha$.

Differentiating $G(\lambda_\alpha(\varepsilon), \varepsilon) = 0$ at $\varepsilon = 0$ gives

$$\lambda'_\alpha(0) = \alpha.$$

Thus the first-order finite- l_0 effect pushes the root radially outward: the direction of motion is the root itself. To obtain the stated remainder, expand one order further. Write

$$\lambda_\alpha(\varepsilon) = \alpha + \alpha\varepsilon + c_2(\alpha)\varepsilon^2 + O(\varepsilon^3).$$

Substituting this expansion into $A(\lambda) - \varepsilon \lambda A'(\lambda) = 0$ and matching the ε^2 terms gives

$$c_2(\alpha) = \frac{\alpha}{2} \left(\frac{\alpha A''(\alpha)}{A'(\alpha)} + 2 \right).$$

Analyticity of the branch gives the stated $O(\varepsilon^3)$ remainder on a sufficiently small branch neighborhood.

To use this expansion at the threshold scale, we need a uniform bound on the second-order coefficient along the dominant roots. For such a root, write

$$\alpha = \omega_{m,n} \exp(-\mu(x_m, \beta_n)/n), \quad |x_m| = 2\pi/n.$$

Since $\mu(x_m, \beta_n) = O(n^{-1})$, we have

$$\alpha^a = e^{-ix_m} e^{-\beta_n \mu(x_m, \beta_n)} = 1 + O(n^{-1}).$$

Moreover,

$$P'_n(\alpha) = \alpha^{n-a-1} \{n\alpha^a - p(n-a)\},$$

and therefore

$$n\alpha^a - p(n-a) = n\{q + p\beta_n + O(n^{-1})\}.$$

Since $q + p\beta_n \geq q > 0$, $|P'_n(\alpha)| \geq cn$ for large n . The same calculation gives

$$P''_n(\alpha) = \alpha^{n-a-2} \{n(n-1)\alpha^a - p(n-a)(n-a-1)\} = O(n^2),$$

so

$$\left| \frac{P''_n(\alpha)}{P'_n(\alpha)} \right| = O(n)$$

uniformly on the dominant branches. Also

$$|1 - \alpha|^{-1} = O(n).$$

Indeed, $\omega_{m,n} \neq 1$, the nearest nontrivial n -th root of unity is at distance $2 \sin(\pi/n)$ from 1, and $\alpha - \omega_{m,n} = O(n^{-2})$ on the dominant branches. Since $A = -P_n/(1 - \lambda)$ by (38), and the constant sign cancels in logarithmic derivatives,

$$\frac{A''(\alpha)}{A'(\alpha)} = \frac{P''_n(\alpha)}{P'_n(\alpha)} + \frac{2}{1 - \alpha} = O(n).$$

Consequently, along each dominant branch,

$$|\lambda_\alpha(\varepsilon)| = |\alpha|(1 + \varepsilon + O(n\varepsilon^2)).$$

The same expansion also identifies the sign of the second-order modulus correction. Write

$$\lambda_\alpha(\varepsilon) = \alpha \left(1 + \varepsilon + d_\alpha \varepsilon^2 + O(\varepsilon^3) \right), \quad d_\alpha := \frac{c_2(\alpha)}{\alpha} = 1 + \frac{\alpha A''(\alpha)}{2A'(\alpha)}.$$

Then

$$|\lambda_\alpha(\varepsilon)| = |\alpha| \left(1 + \varepsilon + \Re(d_\alpha) \varepsilon^2 + O(\varepsilon^3) \right).$$

We check $\Re(d_\alpha) > 0$ on the dominant branches directly. Using $A = -P_n/(1 - \lambda)$ at a non-unit root,

$$d_\alpha = \frac{1}{1 - \alpha} + \frac{\alpha P_n''(\alpha)}{2P_n'(\alpha)}.$$

Let

$$r_\alpha := \alpha^a.$$

The derivative formulas above give the exact identity

$$\frac{\alpha P_n''(\alpha)}{P_n'(\alpha)} = n \frac{r_\alpha - p(1 - \beta_n)^2}{r_\alpha - p(1 - \beta_n)} - 1.$$

For dominant roots, $r_\alpha = e^{-ix_m} e^{-\beta_n \mu(x_m, \beta_n)}$ with $|x_m| = 2\pi/n$, so $r_\alpha = 1 + O(n^{-1})$. Hence

$$\frac{r_\alpha - p(1 - \beta_n)^2}{r_\alpha - p(1 - \beta_n)} = \frac{q + 2p\beta_n - p\beta_n^2}{q + p\beta_n} + O(n^{-1}).$$

The real part of the leading ratio is bounded below by a positive constant uniformly over $\beta_n \in [0, 1]$, because $q > 0$ and

$$q + 2p\beta_n - p\beta_n^2 \geq q.$$

The remaining term has nonnegative real part. Indeed, if $\alpha = re^{i\theta}$ with $r < 1$, then

$$\Re \frac{1}{1 - \alpha} = \frac{1 - r \cos \theta}{|1 - \alpha|^2} > 0.$$

Consequently the n -order positive term above cannot be offset by $(1 - \alpha)^{-1}$. There is a constant $c_* > 0$ such that

$$\Re(d_\alpha) \geq c_* n$$

on the dominant branches for all sufficiently large n . This proves that the second-order modulus correction is positive.

We now pass from the branch expansion to the spectral radius. For $0 < |x_m| \leq \delta$, the same logarithmic-coordinate calculation, formalized in Lemma 13 below, gives

$$\nu_m(\eta) = \mu(x_m, \beta_n) - \eta + O(\eta^2), \quad \eta = n\varepsilon,$$

uniformly in m, a, n . Hence

$$|\lambda_m(\varepsilon)| = |\alpha_m| \{1 + \varepsilon + O(n\varepsilon^2)\}$$

for all small-phase branches. For $|x_m| \geq \delta$, Lemma 12 gives $\Re \mu \geq c_\delta$, so the limiting roots are at radial distance $\Omega(n^{-1})$ from the unit circle. By Rouché's theorem on small circles around those limiting roots, the finite- ε roots remain at distance $\Omega(n^{-1})$ from the unit circle for $\varepsilon \leq En^{-3}$, after reducing E if needed. Combining these estimates with the non-dominant separation established in the proof of Proposition 1 yields

$$|\rho(\varepsilon) - \rho_\infty(1 + \varepsilon)| \leq K_\rho n \varepsilon^2$$

for $0 \leq \varepsilon \leq En^{-3}$, which is (13).

LEMMA 13 (Uniform finite-prompt perturbation in logarithmic coordinates). *Fix $p \in (0, 1)$. There are constants $\delta, \eta_0, C, N > 0$, depending only on p , such that the following holds for all coprime $1 \leq a < n$, $n \geq N$. For every nonzero phase $0 < |x_m| \leq \delta$, the finite-prompt equation has a unique logarithmic branch*

$$\lambda_{m,n}(\eta) = \omega_{m,n} \exp(-\nu_{m,n}(\eta)/n), \quad \eta := n\varepsilon,$$

with $\nu_{m,n}(0) = \mu(x_m, \beta_n)$, for $0 \leq \eta \leq \eta_0$, and

$$\nu_{m,n}(\eta) = \mu(x_m, \beta_n) - \eta + R_{m,n}(\eta), \quad |R_{m,n}(\eta)| \leq C\eta^2.$$

Proof. We keep the removable denominator in $A = -P_n/(1 - \lambda)$, because this is the term that must be controlled uniformly. In the chart $\lambda = \omega_{m,n}e^{-\nu/n}$, set $t := 1/n$ and

$$H(\nu, x, \beta) = e^{-\nu} - pe^{ix}e^{-(1-\beta)\nu} - q.$$

Since $A = -H/(1 - \lambda)$ and $\lambda A'(\lambda) = -n\partial_\nu A(\lambda)$, the equation

$$A(\lambda) - \varepsilon \lambda A'(\lambda) = 0$$

is equivalent, after multiplying by the nonzero factor $-(1 - \lambda)$, to

$$\mathcal{J}(\nu, \eta, x, \beta, t) := H + \eta H_\nu - t\eta \frac{\lambda}{1 - \lambda} H = 0, \quad \eta = n\varepsilon.$$

At $\eta = 0$, this reduces to $H(\nu, x, \beta) = 0$, whose solution is $\nu = \mu(x, \beta)$. Moreover,

$$\mathcal{J}_\nu(\mu(x, \beta), 0, x, \beta, t) = H_\nu(\mu(x, \beta), x, \beta),$$

and this derivative is bounded away from zero uniformly for $|x| \leq \delta$ and $\beta \in [0, 1]$, after shrinking δ , because $H_\nu(0, 0, \beta) = -(q + p\beta) \leq -q < 0$.

The only uniformity issue is the last term in $\mathcal{J}$. On the branches under consideration $m \neq 0$, and the principal logarithm has $|\Im \nu| \leq \pi$. Thus, on the unit-scale neighborhood used by the implicit-function theorem,

$$|1 - \lambda| \geq c/n, \quad \left| t \frac{\lambda}{1 - \lambda} \right| \leq C.$$

All derivatives of H are uniformly bounded on this neighborhood. The analytic implicit-function theorem therefore applies with constants independent of m, a, n .

Differentiating $\mathcal{J}(\nu_{m,n}(\eta), \eta, x_m, \beta_n, 1/n) = 0$ at $\eta = 0$ gives

$$\nu'_{m,n}(0) = -\frac{\mathcal{J}_\eta}{\mathcal{J}_\nu} = -\frac{H_\nu}{H_\nu} = -1,$$

where the term containing H vanishes because $H(\mu(x_m, \beta_n), x_m, \beta_n) = 0$. The same uniform bounds control the second η -derivative of $\nu_{m,n}$, so Taylor's theorem gives the stated $O(\eta^2)$ remainder. $\square$

LEMMA 14 (No stable islands before the asymptotic threshold). *Let $\Delta_n := 1 - \rho_\infty$. There are constants $C, N > 0$, depending only on p , such that for all coprime $1 \leq a < n$, $n \geq N$, the finite-prompt polynomial is unstable for every physical prompt value satisfying*

$$\Delta_n + Cn\Delta_n^2 \leq \varepsilon \leq \frac{1}{n}.$$

Consequently, the first stable integer prompt length is the same threshold identified by the local crossing of the two dominant branches.

Proof. The lemma rules out the only global pathology not excluded by the local expansion alone: stability could, in principle, reappear at a smaller prompt length after the dominant local crossing. We prove a stronger root-existence statement: throughout the physical range above, at least one root lies outside the unit disk.

Let $\eta = n\varepsilon$. We split the interval $[n\Delta_n + Cn^2\Delta_n^2, 1]$ into three ranges. On the small- η range, the dominant branch itself is outside. On an intermediate range, we construct an outside root in a fixed small phase window. On the large- η range, the product of the root moduli already exceeds one.

First consider $\eta \leq \eta_0$, where η_0 is the constant from Lemma 13, reduced if necessary so that the quadratic remainder below is at most $\eta/4$ whenever $0 \leq \eta \leq \eta_0$. On the dominant phase $x_1 = 2\pi/n$, Lemma 13 gives the continuation

$$\begin{aligned}\lambda_*(\eta) &= \omega_{1,n} \exp(-\nu_*(\eta)/n), \\ \nu_*(\eta) &= \mu(2\pi/n, \beta_n) - \eta + O(\eta^2)\end{aligned}$$

for $0 \leq \eta \leq \eta_0$. Since

$$\rho_\infty = \exp\{-\Re \mu(2\pi/n, \beta_n)/n\},$$

we have

$$\Re \mu(2\pi/n, \beta_n) = n\Delta_n + O(n\Delta_n^2).$$

Thus, after enlarging C , the excess $\eta - \Re \mu(2\pi/n, \beta_n)$ is larger than both the $O(n\Delta_n^2)$ conversion error above and the $O(\eta^2)$ remainder in $\nu_*(\eta)$. Therefore

$$\Re \nu_*(\eta) < 0 \quad \text{whenever} \quad \varepsilon \geq \Delta_n + Cn\Delta_n^2 \quad \text{and} \quad \eta \leq \eta_0.$$

The corresponding root satisfies $|\lambda_*(\eta)| = \exp(-\Re \nu_*(\eta)/n) > 1$, so the system is unstable throughout this part of the range.

We next cover an intermediate range that is bounded away from both endpoints. Set

$$\eta_1 := \frac{1+p}{2} < 1,$$

and reduce η_0 , if necessary, so that $0 < \eta_0 < \eta_1$. For $\eta \in [\eta_0, \eta_1]$, consider the reduced logarithmic equation at phase $x = 0$,

$$H(\nu, 0, \beta) + \eta H_\nu(\nu, 0, \beta) = 0.$$

Writing $z = e^{-\nu}$ and $\gamma := 1 - \beta$, this equation becomes

$$f_{\eta,\beta}(z) := (1-\eta)z - p\{1-\eta(1-\beta)\}z^{1-\beta} - q = 0.$$

For $z \geq 1$, the function $f_{\eta,\beta}$ is convex, because

$$f''_{\eta,\beta}(z) = p\{1-\eta(1-\beta)\}\beta(1-\beta)z^{-\beta-1} \geq 0.$$

Moreover,

$$f_{\eta,\beta}(1) = -\eta(q + p\beta) \leq -\eta_0 q < 0, \quad f_{\eta,\beta}(z) \rightarrow \infty \quad \text{as } z \rightarrow \infty,$$

uniformly over $(\eta, \beta) \in [\eta_0, \eta_1] \times [0, 1]$. To see the uniformity, use $z^{1-\beta} \leq z$ for $z \geq 1$ to get

$$f_{\eta,\beta}(z) \geq \{q(1-\eta) + p\eta\beta\}z - q \geq q(1-\eta_1)z - q.$$

Hence there is a largest root $z_*(\eta, \beta) > 1$. At this root,

$$f'_{\eta,\beta}(z_*) = (1-\eta)\beta + \frac{(1-\beta)q}{z_*} > 0,$$

where we used

$$p\{1-\eta(1-\beta)\}z_*^{1-\beta} = (1-\eta)z_* - q.$$

The positive derivative makes this root locally continuous in (η, β) by the implicit-function theorem, and the preceding uniform growth bound keeps it in a compact z -interval. Therefore compactness over $(\eta, \beta) \in [\eta_0, \eta_1] \times [0, 1]$ gives constants $r_0, m_0 > 0$ such that

$$z_*(\eta, \beta) \geq e^{r_0} > 1,$$

$$f'_{\eta,\beta}(z_*(\eta, \beta)) \geq m_0.$$

Equivalently, the reduced equation has a simple solution $v_0(\eta, \beta) := -\log z_*(\eta, \beta)$ with $\Re v_0 \leq -r_0$, uniformly over this compact parameter set.

We then realize this reduced outside root as an actual finite- n root. The derivative lower bound above lets us apply the implicit-function theorem uniformly once more, now with the phase x also allowed to vary. Thus, after choosing $x_0 > 0$ sufficiently small, the reduced equation

$$H(v, x, \beta) + \eta H_v(v, x, \beta) = 0$$

has a solution $v_0(\eta, \beta, x)$ for every $\eta \in [\eta_0, \eta_1]$, $\beta \in [0, 1]$, and $x \in [0, 2x_0]$, with

$$\Re v_0(\eta, \beta, x) \leq -\frac{3r_0}{4}.$$

Choose $I = [x_0, 2x_0] \subset (0, \delta)$, where δ is from Lemma 12. For all large n , the phase lattice contains at least $c_I n$ values $x_m \in I$. Since $m \mapsto u_m$ is a permutation modulo n , only $O(sn)$ of all phases can have $|\omega_{m,n} - 1| \leq s$. Taking $s > 0$ small enough, we can choose a phase $x_m \in I$ with $|\omega_{m,n} - 1| \geq s$. For this phase and for v in a fixed neighborhood of $v_0(\eta, \beta_n, x_m)$,

$$\left| \frac{1}{n} \frac{\lambda}{1-\lambda} \right| = O(n^{-1}), \quad \lambda = \omega_{m,n} e^{-v/n}.$$

Thus the full finite- n logarithmic equation

$$H + \eta H_\nu - \frac{\eta}{n} \frac{\lambda}{1 - \lambda} H = 0$$

is an $O(n^{-1})$ perturbation of the reduced equation, uniformly over $\eta \in [\eta_0, \eta_1]$. The implicit-function theorem gives a nearby solution $v_{m,n}(\eta)$ satisfying

$$\Re v_{m,n}(\eta) \leq -\frac{r_0}{2}.$$

The associated root has

$$|\lambda_{m,n}(\eta)| = \exp\{-\Re v_{m,n}(\eta)/n\} > 1.$$

Therefore the finite-prompt polynomial is unstable throughout $\eta \in [\eta_0, \eta_1]$.

For the large- η range, consider $\eta \in [\eta_1, 1]$. The finite-prompt polynomial has degree $n - 1$, constant coefficient q , and leading coefficient

$$1 - \varepsilon(n - 1) = 1 - \eta + \frac{\eta}{n}.$$

The product of the moduli of all $n - 1$ roots is therefore

$$\frac{q}{1 - \eta + \eta/n}.$$

For $\eta \geq \eta_1 = (1 + p)/2 = 1 - q/2$, the denominator is at most $q/2 + 1/n$, which is strictly smaller than q for all sufficiently large n . Hence the product of the root moduli is larger than one, and at least one root must lie outside the unit disk.

Combining the three ranges proves instability for every $\varepsilon \in [\Delta_n + Cn\Delta_n^2, 1/n]$. Stable islands before the asymptotic threshold are therefore impossible. $\square$

B.7.3. Proof of Corollary 1 Let $\Delta_n := 1 - \rho_\infty$. Proposition 1 gives $\Delta_n = \Theta(n^{-3})$. By Theorem 3,

$$\rho(\varepsilon) = \rho_\infty(1 + \varepsilon) + O(n\varepsilon^2).$$

The proof of Proposition 1 shows that every non-dominant limiting branch is below the dominant modulus by at least cn^{-3} . At the threshold scale $\varepsilon = O(\Delta_n) = O(n^{-3})$, the finite- l_0 perturbation error is

$$O(n\varepsilon^2) = O(n\Delta_n^2) = O(n^{-5}),$$

which is smaller than this separation. Thus the dominant branches alone determine the first crossing of the unit circle near the stability threshold. Therefore, for a sufficiently large constant C , the system is unstable whenever

$$\varepsilon \geq \Delta_n + Cn\Delta_n^2$$

because a dominant branch has modulus larger than one, while all roots lie strictly inside the unit disk whenever

$$\varepsilon \leq \Delta_n - Cn\Delta_n^2.$$

The stability boundary is therefore determined up to an $O(n\Delta_n^2)$ error by

$$1 = \rho_\infty(1 + \varepsilon) + O(n\varepsilon^2).$$

Since the boundary has $\varepsilon = \Theta(\Delta_n)$, this gives

$$\varepsilon = \Delta_n + O(n^{-5}), \quad \frac{\Delta_n}{\varepsilon} = 1 + O(n^{-2}).$$

Integer rounding in l_0 does not affect this order. If the real threshold is $s = l_0 + n$, replacing s by a neighboring integer changes $s\Delta_n$ by at most $\Delta_n = O(n^{-3})$, which is absorbed by the claimed $O(n^{-2})$ tolerance. Using $\varepsilon = (l_0^{\min} + n)^{-1}$, with Lemma 14 ruling out earlier stable islands, yields

$$(l_0^{\min} + n)(1 - \rho_\infty) = 1 + O(n^{-2}),$$

which is the claimed bound after enlarging the constant.

B.8. Extension to Multiple Request Types

We now extend the argument to $K \geq 3$ request types with decoding lengths $l_1 < \dots < l_K$, proportions $p_1, \dots, p_K$, and common prompt length l_0 . Write $L := l_K$. The proof follows the same embedded linearization as in the two-type case; the only new object is the survival weight α_s , which replaces the two-type piecewise normalization.

B.8.1. Cohort reduction Under per-iteration proportional admission $a_n^{(k)} = p_k a_n$ and proportional within-stage eviction, the full type-stage occupancy process collapses to an L -dimensional *cohort manifold*:

$$x_{n,s}^{(k)} = p_k Y_{n,s} \quad (s < l_k), \quad x_{n,s}^{(k)} = 0 \quad (s \geq l_k),$$

for a single vector $Y_n \in \mathbb{R}_+^L$. Shift preserves within-stage proportions, proportional eviction scales all types in a mixed stage by the same factor, and proportional admission injects a new mixed cohort.

From any initial state, the process reaches this manifold within L steps because all pre-existing requests have either completed or been evicted.

Define the *survival weight* and *effective stage weight*:

$$\alpha_s := \sum_{k:l_k > s} p_k, \quad V_s := \alpha_s(l_0 + s + 1), \quad s = 0, \dots, L-1.$$

The survival weight α_s is nonincreasing, with $\alpha_0 = 1$ and $\alpha_{L-1} = p_K > 0$; it drops exactly at the decoding lengths. Total memory is $\sum_{s=0}^{L-1} V_s Y_{n,s}$, and the eviction-free equilibrium is $Y^* = x^* \mathbf{1}$, where $x^* = M / \sum_s V_s$.

B.8.2. Characteristic polynomial On the cohort manifold, the linearized no-eviction dynamics have the same companion form as in the two-type case: $y_n = \Phi y_{n-1}$, with first-row entries $-V_{j+1}/V_0$. The nonzero eigenvalues are the roots of

$$F_{l_0}(\lambda) = \sum_{s=0}^{L-1} V_s \lambda^{L-1-s} = (l_0 + L)A(\lambda) - \lambda A'(\lambda),$$

where the limiting polynomial is

$$(1 - \lambda)A(\lambda) = -\lambda^L + \sum_{k=1}^K p_k \lambda^{L-l_k}. \quad (40)$$

B.8.3. Root structure of $A(\lambda)$ All roots of A lie in the closed unit disk. Indeed, if $|\lambda| > 1$, then

$$|\lambda|^L \leq \sum_k p_k |\lambda|^{L-l_k} < |\lambda|^L,$$

a contradiction. To locate the unit-circle roots, note that $\gcd(L-l_1, \dots, L-l_{K-1}, L) = g$: any divisor of L and $L-l_i$ also divides l_i . If $|\lambda| = 1$ and $A(\lambda) = 0$, equality in the triangle inequality forces $\lambda^{L-l_k} = 1$ for every k , and Bézout's identity gives $\lambda^g = 1$. Since $A(1) = \sum_k p_k l_k > 0$, only non-trivial g -th roots can be roots of A . At each such root ω , differentiating (40) gives

$$(1 - \omega)A'(\omega) = -\omega^{-1} \sum_k p_k l_k,$$

so $A'(\omega) \neq 0$.

B.8.4. Instability when $g > 1$ Each non-trivial g -th root ω satisfies $A(\omega) = 0$ and $A'(\omega) \neq 0$. Applying the implicit-function theorem to

$$G(\lambda, \varepsilon) = A(\lambda) - \varepsilon \lambda A'(\lambda), \quad \varepsilon = \frac{1}{l_0 + L},$$

gives $\lambda'(0) = \omega$, and hence

$$\lambda(\varepsilon) = \omega(1 + \varepsilon) + O(\varepsilon^2).$$

Thus the finite-prompt correction induces the same outward radial drift as in the two-type system. Each unit-circle root moves outside the unit disk for sufficiently small positive ε , producing $g - 1$ unstable eigenvalues.

B.8.5. Local stability when $g = 1$ When $g = 1$, all roots of A lie strictly inside the unit circle. Let $r_* := \max_j |\alpha_j| < 1$, and choose $\rho \in (r_*, 1)$. On $|\lambda| = \rho$, $|A(\lambda)| \geq m_\rho > 0$, while $|\varepsilon \lambda A'(\lambda)| \leq \varepsilon M_\rho$. For l_0 large enough that $\varepsilon M_\rho < m_\rho$, Rouché's theorem implies that $F_{l_0}/(l_0 + L)$ and A have the same number of roots in $|\lambda| < \rho$, namely all $L - 1$. Therefore $\rho(\Phi) \leq \rho < 1$.

B.8.6. Global convergence when $g = 1$ The Lyapunov argument of Section B.6 extends without a change in structure. Since $\alpha_s \geq p_K > 0$, the in-service population

$$N_n = \sum_s \alpha_s Y_{n,s} \leq \frac{M}{l_0 + 1}$$

is $O(1)$, uniformly in K . Each shift increases memory by at most N_n tokens. Restoring feasibility therefore requires evicting only $O(1/l_0)$ requests per step, so the perturbation from the no-eviction linearized dynamics satisfies $\|e_n\|_2 = O(1/l_0)$. The Lyapunov function $\mathcal{L}(y) = y^\top P y$ from Lemma 9, with the same uniform bound $I \preceq P \preceq C_P I$, satisfies

$$\mathcal{L}(y_n) - \mathcal{L}(y_{n-1}) \leq -\|y_{n-1}\|_2^2 + O(1/l_0).$$

For $\|y_{n-1}\|_2 \geq \delta$, this drift is strictly negative when l_0 is large enough. Since $\mathcal{L} \geq 0$, the embedded trajectory reaches the no-eviction neighborhood $\|y\|_2 < \delta$ in finite time. Thereafter $e_n = 0$, and the Schur-stable linear dynamics imply exponential convergence.

B.8.7. Spectral gap scaling for $K \geq 3$ The log-coordinate parameterization of Section B.7.1 extends to K types. For the polynomial (40), write $n := l_K$ and parameterize the u -th branch as $\lambda = \omega_u \exp(-\mu/n)$, where ω_u is an n -th root of unity. The branch equation becomes

$$\sum_{k=1}^K p_k e^{ix_k(u)} e^{\beta_{k,n}\mu} = 1, \quad x_k(u) := -\frac{2\pi u l_k}{n} \pmod{2\pi},$$

with $\beta_{k,n} := l_k/n$ and $x_K(u) = 0$. The implicit-function theorem gives a local branch $\mu(x)$ with

$$\Re \mu(x) = \frac{1}{2D} \sum_{k=1}^K p_k \left(x_k - \beta_k \bar{x} \right)^2 + O(\|x\|^3), \quad D := \sum_k p_k \beta_k, \quad \bar{x} := \frac{\sum_k p_k x_k}{D}.$$

The quadratic term is positive definite; in particular, the $k = K$ term contributes $p_K \bar{x}^2$. Thus the spectral gap is governed by the smallest nonzero value of $\Re \mu$ over the discrete phases $x(u)$, $u = 1, \dots, n-1$.

Fixed shorter lengths. If $l_1, \dots, l_{K-1}$ are fixed and $n \rightarrow \infty$, the branch u^* satisfying $\gcd(l_1, \dots, l_{K-1}) u^* \equiv 1 \pmod{n}$ yields phases $x_k = O(1/n)$. The resulting gap has the cubic scaling

$$1 - \rho_\infty = c_{\text{fix}} n^{-3} + O(n^{-4}), \quad c_{\text{fix}} = \frac{2\pi^2}{p_K} \sum_{k=1}^{K-1} p_k (a_k^\#)^2 + \frac{2\pi^2}{p_K^2} \left(\sum_{k=1}^{K-1} p_k a_k^\# \right)^2,$$

where $a_k^\# := l_k / \gcd(l_1, \dots, l_{K-1})$. For $K = 2$, this reduces to $2\pi^2 p / q^2$.

Bounded resonance. More generally, suppose there exist a fixed integer $\bar{u}$ and bounded integers $c_1, \dots, c_{K-1}$ such that $\bar{u} l_k \equiv c_k \pmod{n}$ for all k . Then

$$1 - \rho_\infty = c_{\text{res}} n^{-3} + O(n^{-4}), \quad c_{\text{res}} = \frac{2\pi^2}{D} \sum_{k=1}^K p_k \left(c_k - \beta_k \frac{\sum_j p_j c_j}{D} \right)^2,$$

with $c_K := 0$. This includes near-consecutive families such as $(n, n+1, \dots, n+K-1)$.

Generic ratios. When no bounded resonance exists, the minimizing branch index grows with n . The gap exponent then depends on the simultaneous Diophantine approximation properties of $(\beta_1, \dots, \beta_{K-1})$. The numerical evidence is consistent with the generic exponent $1 + 2/(K-1)$, giving $\alpha = 2$ for $K = 3$ and $\alpha = 5/3$ for $K = 4$. A proof of this generic exponent remains open.

B.9. Heterogeneous Prompt Lengths

We now justify Remark 2. When $l_0^A \neq l_0^B$, the characteristic polynomial gains a correction term:

$$F(\lambda) = -\lambda A'(\lambda) + [l_0^A p + l_0^B q + l_B] A(\lambda) - p q (l_0^A - l_0^B) \cdot C(\lambda)$$

where $C(\lambda) = (\lambda^{l_B - l_A} - 1) / (\lambda - 1)$.

Case $g > 1$: At every non-trivial g -th root ω , we have $\omega^{l_B - l_A} = 1$ because $g \mid l_A$ and $g \mid l_B$. Hence $C(\omega) = 0$. The heterogeneous-prompt correction vanishes on the critical unit-circle modes, and the instability argument of Theorem 4 applies unchanged.

Case $g = 1$: The correction term perturbs the root locations by order $O(|l_0^A - l_0^B|)$. Stability is preserved when this perturbation is negligible relative to the leading prompt scale. A sufficient condition is

$$\frac{|l_0^A - l_0^B|}{(l_0^A + 1)p + (l_0^B + 1)q} \rightarrow 0 \quad \text{as } l_0^A, l_0^B \rightarrow \infty,$$

which holds, for example, when $|l_0^A - l_0^B| = o(\min(l_0^A, l_0^B))$.

REMARK 3 (STABLE INITIAL CONDITIONS IN NON-COPRIME SYSTEMS). When $g > 1$, some initial conditions remain stable because the instability is spectral rather than global across all directions. The general solution admits a Vandermonde decomposition into the equilibrium mode $\lambda_0 = 1$, unstable modes $\lambda_1, \dots, \lambda_{g-1}$ with $|\lambda_j| > 1$, and stable modes $\lambda_g, \dots, \lambda_{l_B-1}$ with $|\lambda_j| < 1$. Stability holds exactly when the initial perturbation has no projection onto the unstable subspace. The equilibrium $X^* = x^*(1, \dots, 1)^\top$ itself lies outside that unstable subspace.

B.10. Pulse Cycle Under Strict Per-Iteration Proportions

This subsection records a complementary finite- l_0 calculation for a stricter mixing protocol. Specifically, we require

$$a_n^{(A)} = p a_n, \quad a_n^{(B)} = q a_n$$

at every iteration. If LPF partially evicts a mixed stage, it removes types A and B in proportion to their occupancies in that stage. This protocol is more restrictive than the long-run proportion constraint used in Theorem 2. It is useful for identifying an explicit eviction cycle and the spectrum of its local return map, but it is not needed for the proof of Theorem 2.

LEMMA 15 (Cohort reduction for the strict mixing rule). *Consider the two-type common-prompt system with decoding lengths $l_A < l_B$, and write $a := l_A$ and $b := l_B$. Under strict per-iteration proportional admission and proportional within-stage eviction, the type-stage process admits a scalar cohort representation $y_n^{(j)}$ such that*

$$x_n^{(A,j)} = \begin{cases} p y_n^{(j)}, & j < a, \\ 0, & j \geq a, \end{cases} \quad x_n^{(B,j)} = q y_n^{(j)}, \quad 0 \leq j \leq b-1.$$

Moreover, the normalized variables of the coprime proof satisfy

$$Y_{n,j} = y_n^{(j)}, \quad 0 \leq j \leq b-1,$$

and the resulting cohort dynamics have effective weights

$$V_j = \begin{cases} l_0 + 1 + j, & 0 \leq j \leq a-1, \\ q(l_0 + 1 + j), & a \leq j \leq b-1. \end{cases}$$

Proof. Admission creates a fresh mixed cohort in the exact proportions $p : q$. The execute step advances each surviving cohort by one age; type A leaves after age $a-1$, while type B leaves after

age $b - 1$. Proportional within-stage eviction rescales all types in a mixed stage by the same factor and therefore preserves the within-stage composition. Induction over n gives the displayed cohort representation. For $j < a$, the total occupancy is $x_n^{(A,j)} + x_n^{(B,j)} = y_n^{(j)}$; for $j \geq a$, only type B remains, so the total occupancy is $qy_n^{(j)}$. Hence the normalized variables used above satisfy $Y_{n,j} = y_n^{(j)}$, with the effective stage weights stated in the lemma. $\square$

PROPOSITION 5 (Exact pulse cycle). Assume $g = \gcd(a, b) > 1$, and write $h := b/g$. For $r = 0, \dots, g - 1$, define

$$J_r = \{r, r + g, \dots, r + (h - 1)g\}, \quad \beta = \frac{M}{\sum_{m=0}^{h-1} V_{g-1+mg}},$$

and

$$\alpha_r = \frac{M - \beta \sum_{m=1}^{h-1} V_{r+mg}}{V_r}.$$

Let $Z^{(r)} \in \mathbb{R}^b$ be the state whose only nonzero coordinates are

$$Z_r^{(r)} = \alpha_r, \quad Z_{r+mg}^{(r)} = \beta, \quad m = 1, \dots, h - 1.$$

Then

$$Z^{(0)} \rightarrow Z^{(1)} \rightarrow \dots \rightarrow Z^{(g-1)} \rightarrow Z^{(0)}$$

is an exact period- g orbit. Its average throughput is

$$\bar{T}_{\text{pulse}} = \frac{2M}{a(2l_0 + a + g) + q(b - a)(2l_0 + a + b + g)}.$$

Proof. Write $u = l_0 + 1$. For $r = 0, \dots, g - 1$, let

$$S_r := \sum_{m=1}^{h-1} V_{r+mg}.$$

Then $\alpha_r = (M - \beta S_r)/(u + r)$. Within each branch of the weight profile, $V_{j+g} - V_j$ is constant, so

$$S_r = S_0 + cr, \quad c = \frac{a}{g} - 1 + q\left(\frac{b}{g} - \frac{a}{g}\right) \geq 0.$$

Hence

$$\alpha_r - \alpha_{r+1} = \frac{M - \beta S_0 + \beta cu}{(u + r)(u + r + 1)} > 0 \quad \text{for } r = 0, \dots, g - 2,$$

because $M - \beta S_0 = V_0 \alpha_0 > 0$. Also,

$$\alpha_{g-1} = \frac{M - \beta \sum_{m=1}^{h-1} V_{g-1+mg}}{V_{g-1}} = \beta.$$

Fix $r \leq g - 2$. Starting from $Z^{(r)}$, the execute step shifts the support from J_r to J_{r+1} . The youngest occupied age $r + 1$ carries mass α_r , and all older occupied ages in J_{r+1} carry mass β . The post-execute memory exceeds M by exactly

$$V_{r+1}(\alpha_r - \alpha_{r+1}) > 0.$$

This overflow is smaller than the full youngest-stage load $V_{r+1}\alpha_r$. Hence LPF trims only the youngest occupied age, reducing its mass from α_r to α_{r+1} . No new admission occurs, so the next state is $Z^{(r+1)}$.

At phase $r = g - 1$, every occupied age already carries mass β . Execute removes the age- $(b - 1)$ cohort, shifts the remaining $h - 1$ cohorts to ages $g, 2g, \dots, (h - 1)g$, and creates slack

$$M - \sum_{m=1}^{h-1} V_{mg}\beta = V_0\alpha_0.$$

No eviction is needed. Admission therefore inserts a new age-0 cohort of mass α_0 , reconstructing $Z^{(0)}$.

Finally,

$$\sum_{m=0}^{h-1} V_{g-1+mg} = \frac{a(2l_0 + a + g) + q(b - a)(2l_0 + a + b + g)}{2g}.$$

Since one cohort of mass β completes every g iterations,

$$\bar{T}_{\text{pulse}} = \frac{\beta}{g} = \frac{2M}{a(2l_0 + a + g) + q(b - a)(2l_0 + a + b + g)}.$$

□

PROPOSITION 6 (Exact return map and compressed polynomial). *In a neighborhood of the pulse cycle from Proposition 5, the phase maps are affine. For $r = 0, \dots, g - 2$, if*

$$z = (z_0, \dots, z_{h-1})$$

denotes the masses on the support J_r , then

$$\Phi_r(z_0, \dots, z_{h-1}) = \left(\frac{M - \sum_{m=1}^{h-1} V_{r+1+mg} z_m}{V_{r+1}}, z_1, \dots, z_{h-1} \right).$$

At the wrap phase,

$$\Phi_{g-1}(z_0, \dots, z_{h-1}) = \left(\frac{M - \sum_{m=1}^{h-1} V_{mg} z_{m-1}}{V_0}, z_0, \dots, z_{h-2} \right).$$

Let $\mathcal{R} = \Phi_{g-1} \circ \dots \circ \Phi_0$ be the g -step return map at phase 0. Then $D\mathcal{R}$ has one exact zero eigenvalue, and its remaining eigenvalues are the roots of

$$\widehat{F}(\lambda) = \sum_{m=0}^{a/g-1} \left(\frac{l_0}{g} + 1 + m \right) \lambda^{h-1-m} + q \sum_{m=a/g}^{h-1} \left(\frac{l_0}{g} + 1 + m \right) \lambda^{h-1-m}. \quad (41)$$

Proof. The strict inequalities $\alpha_r > \alpha_{r+1}$ for $r \leq g-2$, together with $\alpha_0 > 0$, persist in a small neighborhood of each phase state. On that neighborhood, the LPF trim pattern is fixed: for $r \leq g-2$, only the youngest occupied age is partially trimmed; for $r = g-1$, no eviction occurs and admission fills the slack at age 0. The displayed affine maps follow directly from execute, LPF trim, and admission.

Now write

$$x = (x_0, \dots, x_{h-2}) := (z_1, \dots, z_{h-1}), \quad W_m := V_{g-1+mg}, \quad m = 0, \dots, h-1.$$

Every non-wrap phase deletes the current youngest coordinate and leaves the older coordinates unchanged. Therefore the g -step return map is determined by x alone:

$$\mathcal{R}_{\text{red}}(x_0, \dots, x_{h-2}) = \left(\frac{M - \sum_{m=1}^{h-1} W_m x_{m-1}}{W_0}, x_0, \dots, x_{h-3} \right).$$

Its Jacobian is the companion matrix

$$K = \begin{pmatrix} -W_1/W_0 & -W_2/W_0 & \dots & -W_{h-1}/W_0 \\ 1 & 0 & \dots & 0 \\ 0 & 1 & \ddots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{pmatrix},$$

so the full Jacobian has block form

$$D\mathcal{R} = \begin{pmatrix} 0 & * \\ 0 & K \end{pmatrix},$$

This block-triangular form gives one exact zero eigenvalue. The remaining $h-1$ eigenvalues are the roots of the characteristic polynomial of K .

To identify that polynomial, set

$$\bar{a} := a/g, \quad \bar{b} := b/g = h, \quad \ell_0 := l_0/g.$$

Since

$$\frac{W_m}{g} = \frac{V_{g-1+mg}}{g} = \begin{cases} \ell_0 + 1 + m, & 0 \leq m \leq \bar{a} - 1, \\ q(\ell_0 + 1 + m), & \bar{a} \leq m \leq \bar{b} - 1, \end{cases}$$

the companion polynomial of K is exactly

$$\widehat{F}(\lambda) = \sum_{m=0}^{\bar{a}-1} (\ell_0 + 1 + m) \lambda^{\bar{b}-1-m} + q \sum_{m=\bar{a}}^{\bar{b}-1} (\ell_0 + 1 + m) \lambda^{\bar{b}-1-m},$$

which is (41). $\square$

COROLLARY 2 (Local pulse stability). *The pulse cycle from Proposition 5 is locally asymptotically stable if and only if the polynomial $\widehat{F}$ in (41) is Schur stable. In particular, for generic $q \in (0, 1)$ and sufficiently large l_0 , the pulse cycle is locally asymptotically stable.*

Proof. The local return map is affine in a neighborhood of the pulse cycle, so local asymptotic stability is equivalent to $\rho(D\mathcal{R}) < 1$. Proposition 6 reduces this condition to $\rho(K) < 1$, equivalently Schur stability of $\widehat{F}$. The final claim follows because the compressed pair $(\bar{a}, \bar{b}) = (a/g, b/g)$ is coprime, and the proof of Theorem 5 depends only on the coefficient profile of the polynomial. Applying that argument to $\widehat{F}$ gives Schur stability for generic q and sufficiently large $\ell_0 = l_0/g$, hence for sufficiently large l_0 . $\square$

EXAMPLE 12 (PULSE RETURN MAP FOR (2, 4)). Let $a = 2$, $b = 4$, and $g = 2$. At phase 0, the state is $z = (x, y)$ on ages 0 and 2, and

$$\Phi_0(x, y) = \left(\frac{M - V_3 y}{V_1}, y \right) = \left(\frac{M - q(l_0 + 4)y}{l_0 + 2}, y \right).$$

At phase 1, the state is (u, v) on ages 1 and 3, and

$$\Phi_1(u, v) = \left(\frac{M - V_2 u}{V_0}, u \right) = \left(\frac{M - q(l_0 + 3)u}{l_0 + 1}, u \right).$$

Therefore

$$D(\Phi_1 \circ \Phi_0) = \begin{pmatrix} 0 & \frac{V_2 V_3}{V_0 V_1} \\ 0 & -\frac{V_3}{V_1} \end{pmatrix},$$

so the eigenvalues are

$$\lambda_1 = 0, \quad \lambda_2 = -\frac{V_3}{V_1} = -\frac{q(l_0 + 4)}{l_0 + 2}.$$

Hence the exact local stability condition is

$$q < \frac{l_0 + 2}{l_0 + 4}.$$

For the symmetric case $p = q = 1/2$, this inequality holds for every $l_0 \geq 1$.

B.11. Summary

This appendix establishes Theorem 2 and its extensions:

1. The characteristic polynomial decomposes as $F(\lambda) = (l_0 + l_B)A(\lambda) - \lambda A'(\lambda)$.
2. Roots of $A(\lambda)$ on the unit circle are exactly the non-trivial g -th roots of unity (Lemmas 3–6).
3. For $g > 1$: IFT shows these roots drift outside the unit circle, causing instability (Theorem 4).
4. For $g = 1$: All roots remain inside the unit circle; Lyapunov analysis proves global convergence from any initial condition (Theorems 5 and 6).
5. Under strict per-iteration proportional admission, the non-coprime two-type system also admits an exact period- g pulse cycle whose local return-map eigenvalues are given by a compressed coprime polynomial (Section B.10).
6. The results extend to $K \geq 3$ request types (Section B.8) and heterogeneous prompt lengths (Remark 2).

Appendix C: Analysis of Rate-Limited Admission Policy

The complete analysis of the rate-limited admission policy will appear here.

Appendix D: Numerical Supplement

This appendix contains the numerical experiments behind Section 6. The main text keeps one representative figure per mechanism. The supplementary figures give the supporting model-based simulations, Vidur simulations, real-GPU experiments, and parameter sweeps.

Throughout this appendix, “model-based simulator” denotes a direct implementation of the discrete-time memory and admission recursions in Section 2. It keeps the model’s state variables, capacity constraint, greedy admission rule, and LPF eviction rule, and adds stochastic arrivals only in experiments that explicitly study open-system behavior. These runs use the paper’s state equations as the simulator and isolate the mathematical mechanism.

“Vidur” denotes the large-scale LLM-serving simulator of Agrawal et al. (2024), which reports simulation errors below 5% for latency and throughput. Unless otherwise stated, our Vidur experiments use the Sarathi scheduler (Agrawal et al. 2023), Meta-Llama-3-8B, and a single NVIDIA A100 GPU. Vidur simulates serving iterations and tracks request-level KV cache memory, so it serves as an intermediate simulation environment between the model-based simulator and physical GPU experiments. In the Vidur simulations, pending requests are considered in arrival order and admitted when memory is available; the global population cap suggested by the eviction-free equilibrium is added only in the rate-limited variant. When resident KV cache later exceeds capacity,

Vidur records a restart: the request loses its current KV cache and must be recomputed from prefill. We report both restart counts and evicted-request counts; in our experiments these are the system-level counterparts of model evictions. The eviction priority follows the LPF abstraction used in the model, removing least-progressed active requests first.

D.1. Model-based GCD simulations

The model-based simulator implements the discrete-time memory and admission dynamics of Section 2. Time is measured in scheduling iterations, memory is measured in the same normalized KV cache units as the model, and a newly admitted request contributes its prefill length before growing by one unit per decode step. The first two figures isolate the deterministic GCD mechanism under saturated input: coprime decode lengths desynchronize completions and converge to the eviction-free equilibrium, whereas non-coprime decode lengths align completions and generate eviction cycles.

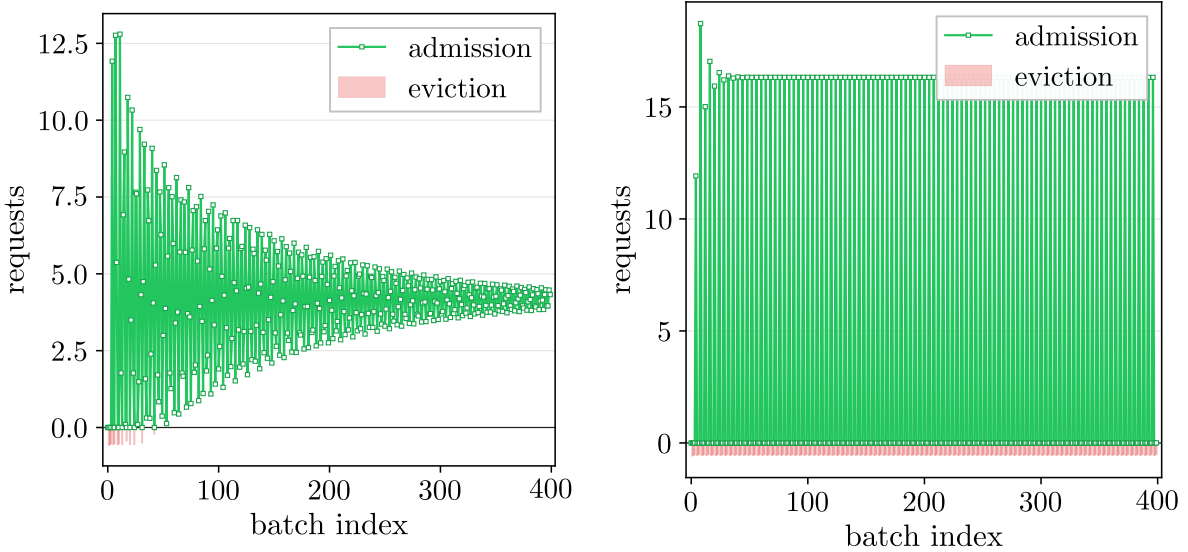


Figure 20 Two-class model-based GCD comparison under greedy admission with $l_0 = 40$ and $p = 1/2$. The coprime pair $(4, 7)$ converges to an eviction-free equilibrium, while the non-coprime pair $(4, 8)$ exhibits persistent oscillations and eviction.

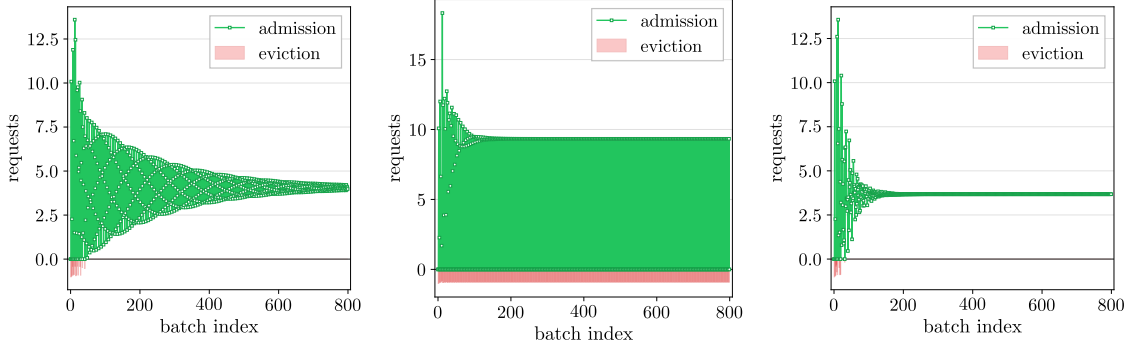


Figure 21 Three-class model-based GCD comparison under greedy admission with $l_0 = 30$ and equal type proportions. The coprime configurations (2, 7, 12) and (2, 9, 12) converge to eviction-free equilibria; the non-coprime configuration (2, 8, 12) exhibits persistent oscillations.

D.2. Mixing mechanisms and additional metrics

Section 6.2 gives the representative Vidur/Sarathi mixing result. The supplementary figures and tables separate the model-based mechanism from the metric changes induced by coprime mixing and partial-GCD reduction. In the model-based runs, routing changes only the set of decode lengths pooled on a node; in the Vidur runs, the same routing change is tested under Sarathi’s serving simulation, which tracks request-level KV cache memory.

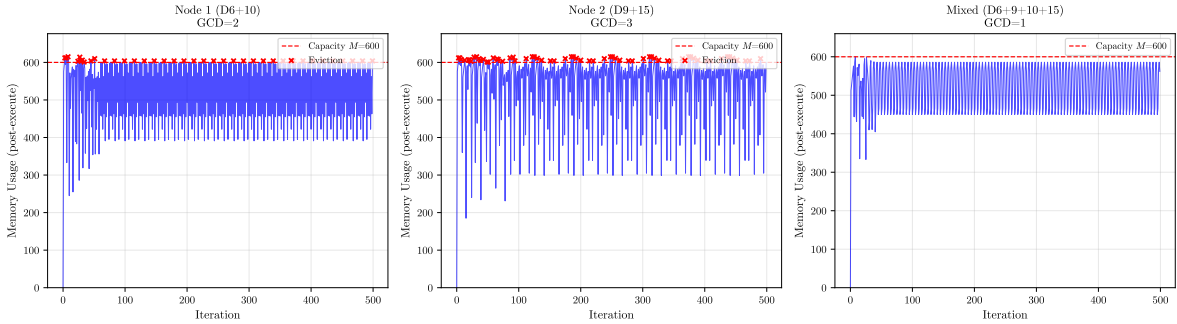


Figure 22 Request mixing dynamics in the model-based simulator with $M = 600$ and $l_0 = 30$. Segregated routing assigns D6+10 to one node and D9+15 to another, producing periodic eviction because the two nodes have non-coprime GCDs. Uniformly mixing D6+9+10+15 gives effective GCD = 1, desynchronizes completions, and converges to an eviction-free equilibrium.

Table 4 Request mixing effect (model-based simulator with $M = 600$, $l_0 = 30$). Segregating types on separate nodes causes periodic eviction due to non-coprime GCDs (2 and 3). Uniformly mixing types creates effective GCD= 1, achieving near-zero eviction through desynchronization. Mixing cuts restarts by 100% (88→0), improves latency by 4.0%, and increases throughput by

5.3% versus the segregated average.

Routing Strategy	Restarts	Latency (s)	Throughput (req/s)	Configuration
Segregated (D6+10)	63	42.50	48.2	D6+10
Segregated (D9+15)	112	45.30	46.8	D9+15
Segregated (average)	88	43.90	47.5	—
Mixed (all types)	0	42.10	50.0	D6+9+10+15

Table 5 Partial GCD reduction effect (model-based simulator with $M = 1000$, $l_0 = 30$). Segregating types on separate nodes with high GCDs (20 and 25) causes severe periodic eviction. Uniformly mixing types reduces effective GCD to 5, achieving 52% fewer restarts through partial desynchronization. This demonstrates that mixing benefits extend beyond coprime configurations.

Routing Strategy	Restarts	Latency (s)	Throughput (req/s)	Configuration
Segregated (D20+40)	248	68.20	38.5	D20+40, GCD=20
Segregated (D25+50)	190	71.40	37.2	D25+50, GCD=25
Segregated (average)	219	69.80	37.9	—
Mixed (all types)	105	65.30	42.8	D20+25+40+50, GCD=5

Table 6 Request mixing effect (Vidur simulation with Sarathi scheduler, Meta-Llama-3-8B, NVIDIA A100 GPU, 20k requests, $l_0 = 512$, overloaded). Segregating types on separate nodes causes periodic eviction due to non-coprime GCDs (25 and 7). Uniformly mixing types creates effective GCD= 1, achieving 98.7% fewer restarts through coprime desynchronization. Mixing cuts restarts by 1,338 (1,356→18), improves latency by 17.4%, and increases throughput by 19.0% versus the segregated average.

Routing Strategy	Restarts	Latency (s)	Throughput (req/s)	Configuration
Segregated (D50+75)	1,059	136.3	73.4	D50+75, GCD=25
Segregated (D77+84)	1,640	183.2	54.8	D77+84, GCD=7
Segregated (average)	1,349	159.8	64.1	—
Mixed (all types)	18	131.9	76.2	D50+75+77+84, GCD=1

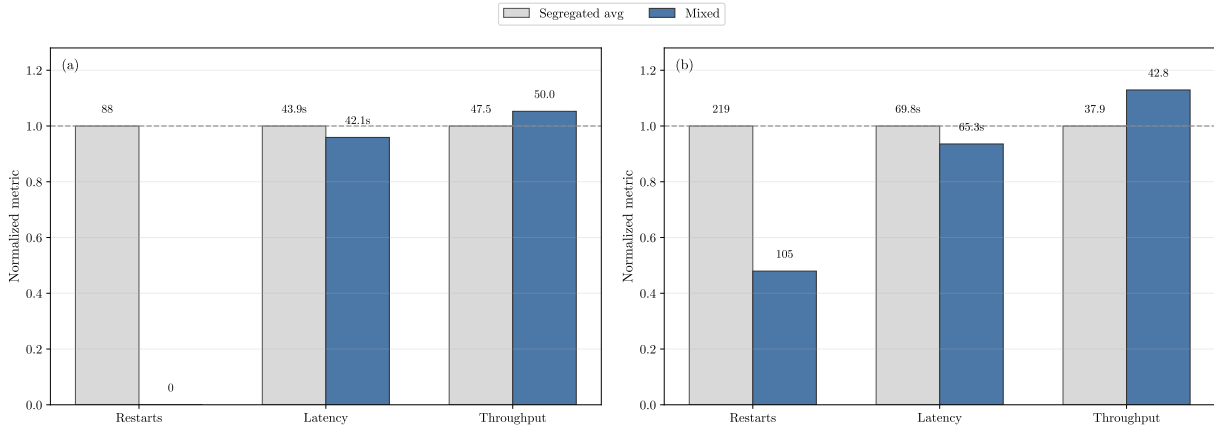


Figure 23 Model-based simulator summary under Poisson arrivals: mixed routing versus the segregated average. Coprime mixing eliminates restarts in this setup, while partial-GCD reduction still lowers restart pressure, latency, and throughput loss.

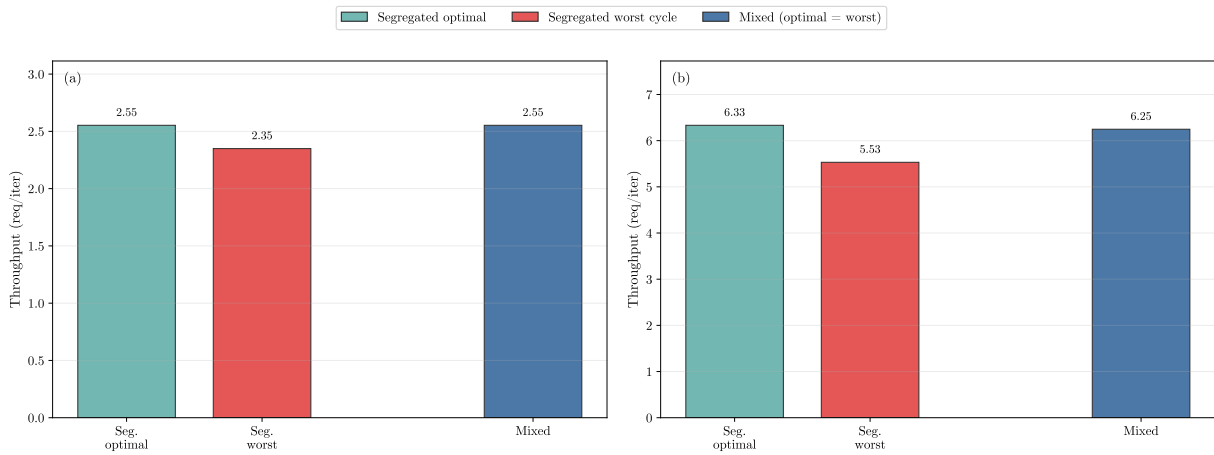


Figure 24 Throughput mechanism for request mixing. Mixing removes the gap between the segregated optimal throughput and the synchronized worst-cycle throughput. The gain comes from preventing synchronization from driving the system into a lower-throughput cycle.

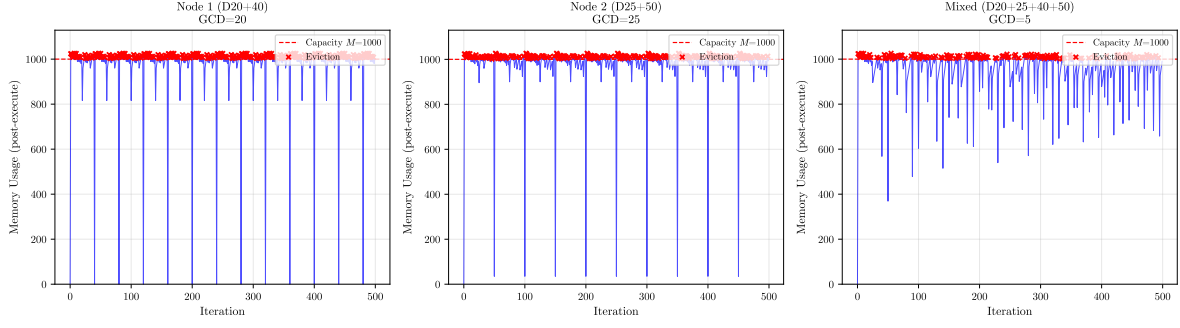


Figure 25 Partial GCD reduction in the model-based simulator with $M = 1000$ and $l_0 = 30$. Segregated D20+40 and D25+50 nodes have high GCDs and severe periodic eviction; mixing all four types reduces the effective GCD to 5 and cuts eviction pressure.

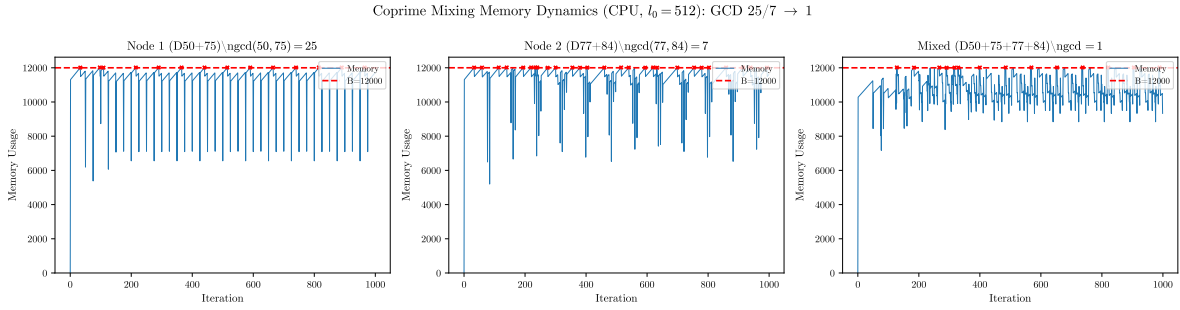


Figure 26 Coprime mixing memory dynamics in the Vidur/Sarathi setting with $l_0 = 512$. Mixing D50+75 and D77+84 produces combined $\text{GCD} = 1$ and removes the periodic overflow observed under segregated routing.

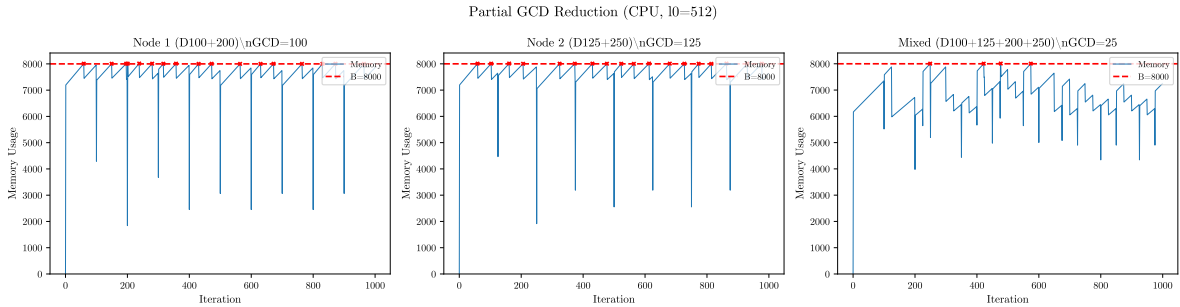


Figure 27 Partial-GCD mixing memory dynamics in the Vidur/Sarathi setting with $l_0 = 512$. Mixing D100+200 and D125+250 reduces the effective GCD from 100/125 to 25 and weakens synchronization.

Table 7 Partial GCD reduction effect (Vidur simulation with Sarathi scheduler, Meta-Llama-3-8B, NVIDIA A100 GPU, 20k requests, $l_0 = 512$). Segregating types on separate nodes with high GCDs (100 and 125) causes severe periodic eviction. Uniformly mixing types reduces effective GCD to 25, achieving 34.0% fewer restarts through partial desynchronization. Mixing cuts restarts by 1,259 (3,708→2,449), improves latency by 7.2%, and increases throughput by 5.8% versus the segregated average. This

demonstrates that mixing benefits extend beyond coprime configurations.

Routing Strategy	Restarts	Latency (s)	Throughput (req/s)	Configuration
Segregated (D100+200)	3296	367.9	27.3	D100+200, GCD=100
Segregated (D125+250)	4121	473.4	21.2	D125+250, GCD=125
Segregated (average)	3708	420.6	24.2	—
Mixed (all types)	2449	390.4	25.6	D100+125+200+250, GCD=25

D.3. Vidur and real-GPU experiments

The following experiments ask whether the same synchronization mechanism is visible after reintroducing serving-system details. Vidur simulates iteration times and request-level KV cache memory for Meta-Llama-3-8B on a single NVIDIA A100 GPU. The real-GPU experiments use an implemented serving stack. These experiments test whether the same completion synchronization, memory growth, and eviction patterns persist beyond the stylized simulator.

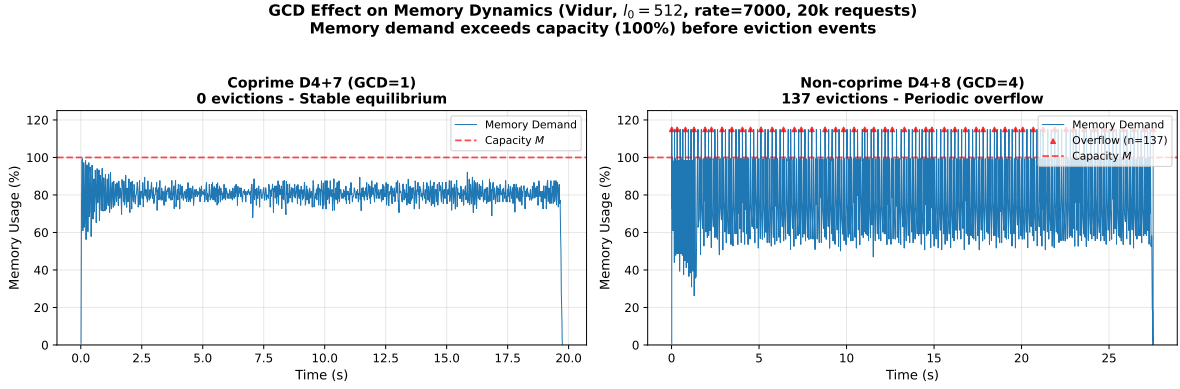


Figure 28 GCD effect on memory dynamics in Vidur with Sarathi, Meta-Llama-3-8B, and an NVIDIA A100 GPU ($l_0 = 512$, rate = 7000, 20k requests). Coprime D4+7 converges to a stable memory equilibrium with zero eviction; non-coprime D4+8 exhibits periodic overflow and eviction.

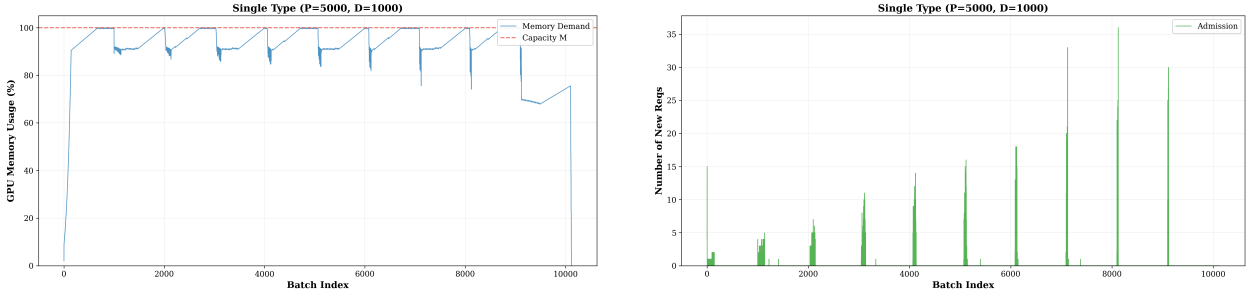


Figure 29 Homogeneous real-GPU experiment ($P = 5000$, $D = 1000$). The figure records physical memory usage and admitted requests over time. Memory repeatedly saturates capacity, triggers eviction, and drops; admissions mirror the same cycle.

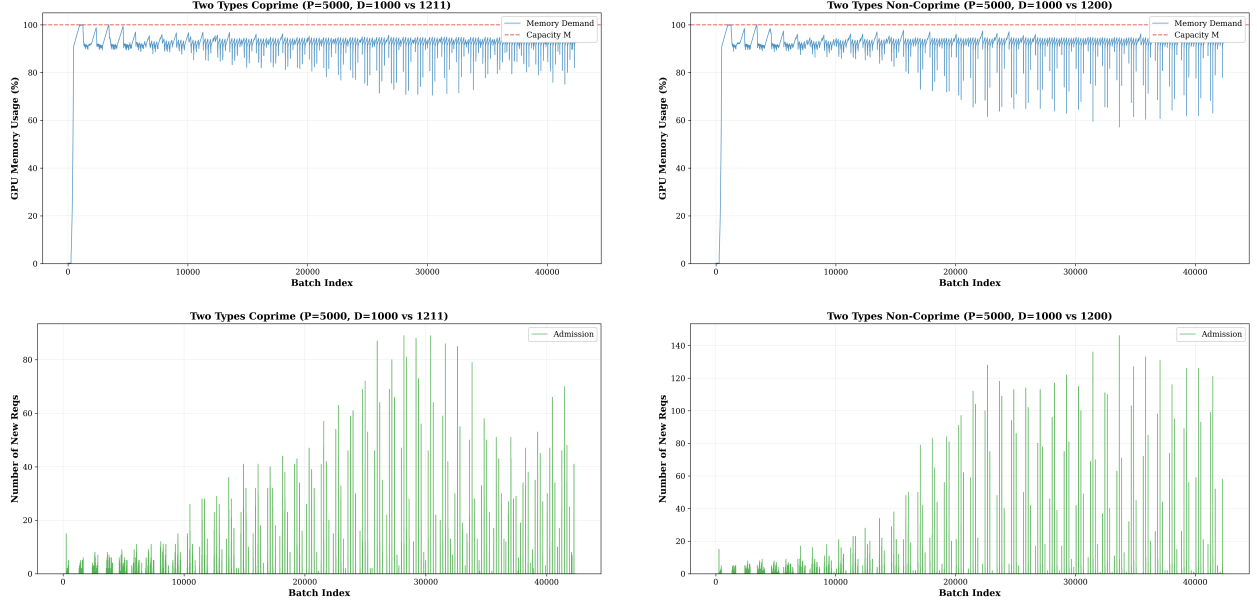


Figure 30 Two-type real-GPU experiments with $P = 5000$. Columns compare a coprime pair (1000, 1211) with a non-coprime pair (1000, 1200); rows show physical memory usage and admitted requests. The non-coprime case exhibits stronger synchronized memory pressure.

D.4. Rate-limit mechanism and sensitivity

Section 6.3 gives the Vidur rate-limited admission result. The supplementary figures isolate the same mechanism in the model-based simulator and show the throughput-eviction tradeoff across admission limits.

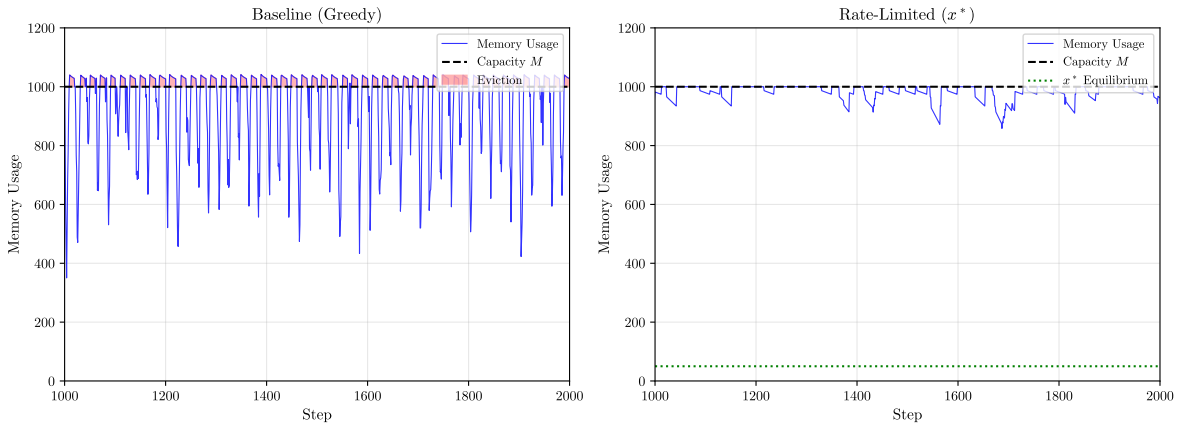


Figure 31 Rate-limited admission dynamics in the model-based simulator with $M = 1000$, $l_0 = 20$, and $l_1 = 20$. Greedy admission produces periodic memory overflow and eviction spikes; threshold admission keeps memory below capacity and eliminates eviction.

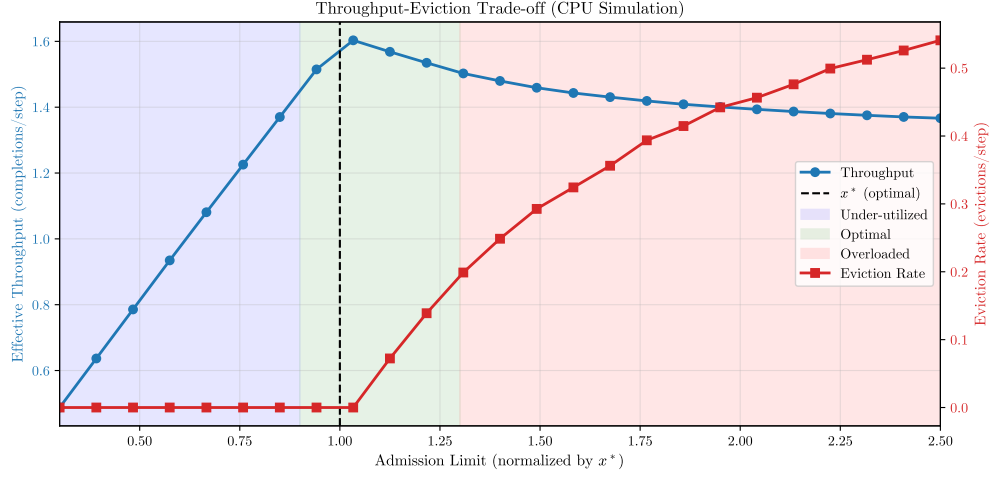


Figure 32 Throughput-eviction tradeoff in the model-based simulator. Throughput peaks near the eviction-free equilibrium x^* and declines beyond it as eviction overhead grows.

Table 8 Rate-limited admission eliminates eviction while improving throughput (model-based simulator with

$M = 1000, l_0 = 20, l_1 = 20$).

Metric	Baseline (Greedy)	Rate-Limited (x^*)	Improvement
Total Evictions (4000 steps)	2662	0	100%
Effective Throughput (req/step)	1.33	1.61	20.7%
Mean Eviction Rate	0.666	0.000	—
Stability	Unstable (periodic eviction)	Stable (zero eviction)	—

D.5. Burst-robustness sweeps

Section 6.4 uses the finite-window ablation as the representative burst result. The following sweeps show the policy ordering across Vidur and model-based settings.

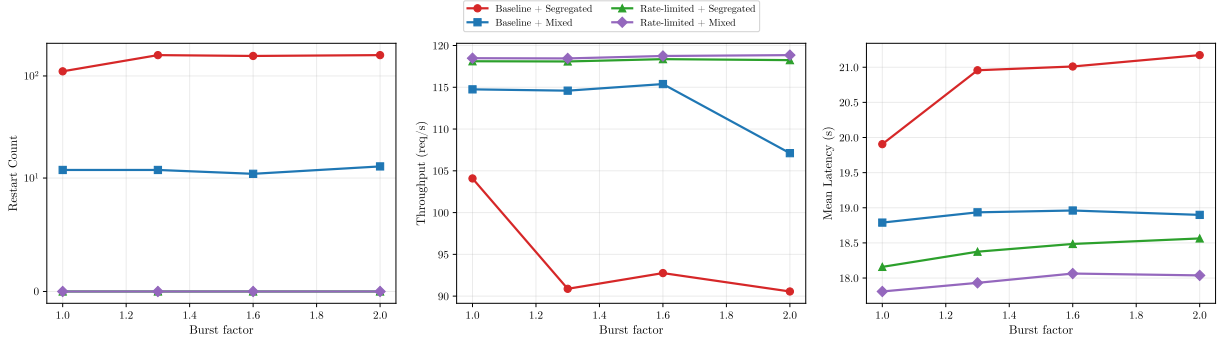


Figure 33 Robustness to periodic burst arrivals: coprime mixing in Vidur. The sweep compares baseline segregated routing, baseline mixed routing, rate-limited segregated routing, and rate-limited mixed routing.

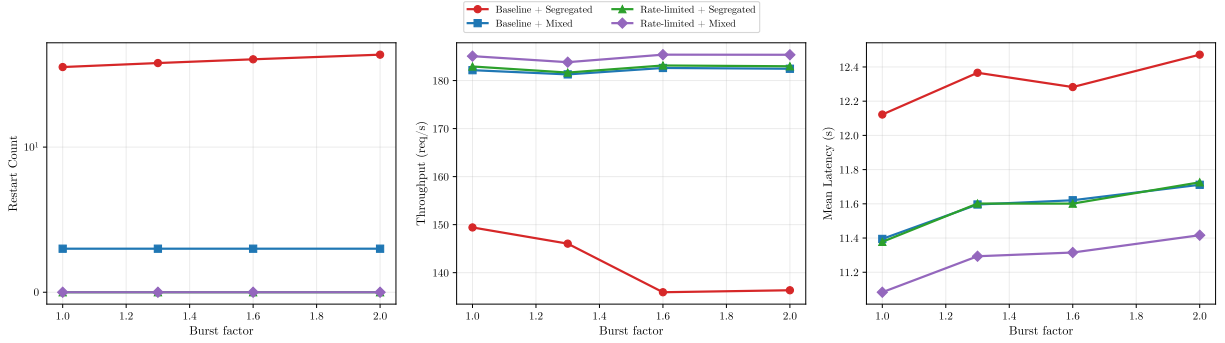


Figure 34 Robustness to periodic burst arrivals: partial-GCD reduction in Vidur. Mixed routing reduces the effective GCD to a value above one; rate limiting remains the policy that eliminates restarts.

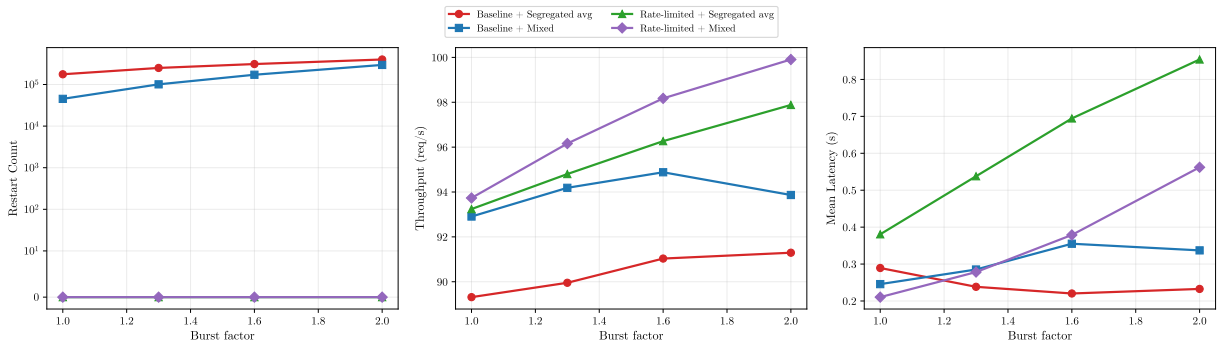


Figure 35 Model-based robustness under periodic burst arrivals: coprime mixing. Mixing reduces restart pressure under greedy admission, while rate-limited policies remain at zero restart.

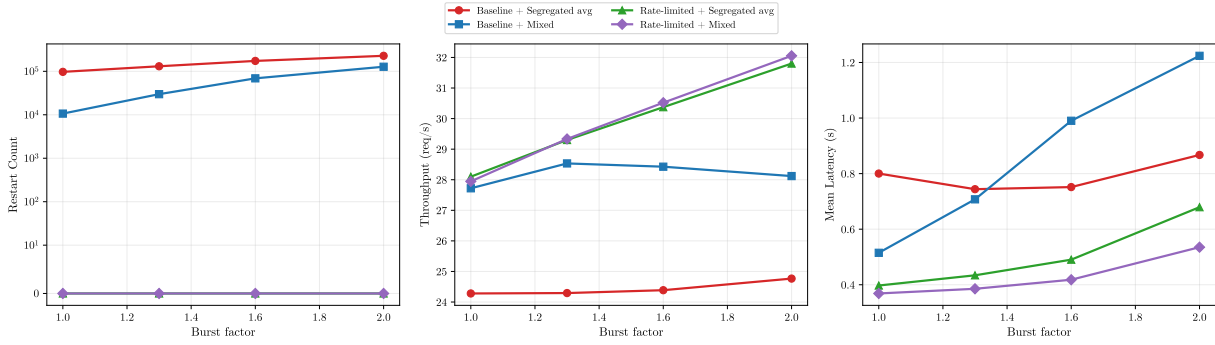


Figure 36 Model-based robustness under periodic burst arrivals: partial-GCD reduction. Partial mixing improves the greedy baseline, while rate limiting remains the hard stability mechanism.